

MULTI-AGENT REINFORCEMENT LEARNING

FOUNDATIONS AND
MODERN APPROACHES



Stefano V. Albrecht
Filippos Christianos
Lukas Schäfer

Praise for *Multi-Agent Reinforcement Learning*

“The authors meticulously bring reinforcement learning together with game theory to provide a foundation for research and application of multi-agent reinforcement learning. This book is the perfect starting point for a grounding in the field.”

– **Andrew Barto**, Professor Emeritus, University of Massachusetts Amherst

“Multi-agent reinforcement learning is well positioned to be the next white-hot area of artificial intelligence and this book provides the essential background, concepts, and insights for understanding this exciting and important area of research.”

– **Michael L. Littman**, Professor of Computer Science, Brown University

“This book is the first complete reference for the growing area of multi-agent reinforcement learning. It provides both an essential resource for newcomers to the field; and a valuable perspective for established researchers.”

– **Peter Stone**, Professor of Computer Science, The University of Texas at Austin

“A landmark textbook to multiagent reinforcement learning, combining game-theoretic foundations with state-of-the-art deep learning. This essential textbook delivers fundamental insights for newcomers, experts and practitioners, featuring real-world applications and advanced algorithms.”

– **Karl Tuyls**, Director of AI Research, Meta AI and Professor of Computer Science, University of Liverpool

“This will become the standard text of the emerging field of multiagent reinforcement learning. It builds from foundational ideas, incorporating recent breakthroughs in deep learning. This book will help accelerate theoretical and practical progress.”

– **Mykel J. Kochenderfer**, Professor of Aeronautics and Astronautics, Stanford University

Multi-Agent Reinforcement Learning

Multi-Agent Reinforcement Learning

Foundations and Modern Approaches

Stefano V. Albrecht, Filippas Christianos, and Lukas Schäfer

The MIT Press
Cambridge, Massachusetts
London, England

© 2024 Massachusetts Institute of Technology

This work is subject to a Creative Commons CC-BY-NC-ND license.

This license applies only to the work in full and not to any components included with permission. Subject to such license, all rights are reserved. No part of this book may be used to train artificial intelligence systems without permission in writing from the MIT Press.



The MIT Press would like to thank the anonymous peer reviewers who provided comments on drafts of this book. The generous work of academic experts is essential for establishing the authority and quality of our publications. We acknowledge with gratitude the contributions of these otherwise uncredited readers.

This book was set in Times New Roman by Stefano V. Albrecht, Filippas Christianos, and Lukas Schäfer. Printed and bound in the United States of America.

Library of Congress Cataloging-in-Publication Data

Names: Albrecht, Stefano V., author. | Christianos, Filippas, author. | Schäfer, Lukas, author.

Title: Multi-agent reinforcement learning : foundations and modern approaches / Stefano V. Albrecht, Filippas Christianos, Lukas Schäfer, The University of Edinburgh, United Kingdom.

Description: Cambridge, Massachusetts : The MIT Press, [2024] | Includes bibliographical references and index.

Identifiers: LCCN 2024002669 (print) | LCCN 2024002670 (ebook) | ISBN 9780262049375 (hardcover) | ISBN 9780262380508 (epub) | ISBN 9780262380515 (pdf)

Subjects: LCSH: Reinforcement learning. | Intelligent agents (Computer software)

Classification: LCC Q325.6 .A43 2024 (print) | LCC Q325.6 (ebook) | DDC 006.3/1—dc23/eng/20240412

LC record available at <https://lccn.loc.gov/2024002669>

LC ebook record available at <https://lccn.loc.gov/2024002670>

Für A.H.A. und R.J.A. — träumt groß, strebt nach euren Zielen, geht voraus.

For A.E.A. — this book would not be here without you.

—S.V.A.

Στη Δάφνη — για την αστείρευτη αγάπη και την υποστήριξη της στη συγγραφή ενός βιβλίου που πιθανώς να μην διαβάσει ποτέ.

—F.C.

Für Jonas, Daniel, Annette und Stefan — für eure Liebe, Unterstützung und die Neugierde, die ihr in mir geweckt habt.

—L.S.

Contents

Summary of Notation	xiii
List of Figures	xvii
Preface	xxiii
1 Introduction	1
1.1 Multi-Agent Systems	2
1.2 Multi-Agent Reinforcement Learning	6
1.3 Application Examples	9
1.3.1 Multi-Robot Warehouse Management	9
1.3.2 Competitive Play in Board Games and Video Games	10
1.3.3 Autonomous Driving	11
1.3.4 Automated Trading in Electronic Markets	11
1.4 Challenges of MARL	12
1.5 Agendas of MARL	13
1.6 Book Contents and Structure	15
I FOUNDATIONS OF MULTI-AGENT REINFORCEMENT LEARNING	17
2 Reinforcement Learning	19
2.1 General Definition	20
2.2 Markov Decision Processes	22
2.3 Expected Discounted Returns and Optimal Policies	24
2.4 Value Functions and Bellman Equation	26
2.5 Dynamic Programming	29
2.6 Temporal-Difference Learning	32
2.7 Evaluation with Learning Curves	36

2.8	Equivalence of $\mathcal{R}(s, a, s')$ and $\mathcal{R}(s, a)$	39
2.9	Summary	40
3	Games: Models of Multi-Agent Interaction	43
3.1	Normal-Form Games	44
3.2	Repeated Normal-Form Games	46
3.3	Stochastic Games	47
3.4	Partially Observable Stochastic Games	49
3.4.1	Belief States and Filtering	53
3.5	Modeling Communication	55
3.6	Knowledge Assumptions in Games	56
3.7	Dictionary: Reinforcement Learning $\leftrightarrow$ Game Theory	58
3.8	Summary	58
4	Solution Concepts for Games	61
4.1	Joint Policy and Expected Return	62
4.2	Best Response	65
4.3	Minimax	65
4.3.1	Minimax Solution via Linear Programming	67
4.4	Nash Equilibrium	68
4.5	ϵ -Nash Equilibrium	70
4.6	(Coarse) Correlated Equilibrium	71
4.6.1	Correlated Equilibrium via Linear Programming	74
4.7	Conceptual Limitations of Equilibrium Solutions	75
4.8	Pareto Optimality	76
4.9	Social Welfare and Fairness	78
4.10	No-Regret	81
4.11	The Complexity of Computing Equilibria	83
4.11.1	PPAD Complexity Class	84
4.11.2	Computing ϵ -Nash Equilibrium Is PPAD-Complete	86
4.12	Summary	87
5	Multi-Agent Reinforcement Learning in Games: First Steps and Challenges	89
5.1	General Learning Process	90
5.2	Convergence Types	92
5.3	Single-Agent RL Reductions	95
5.3.1	Central Learning	95
5.3.2	Independent Learning	97
5.3.3	Example: Level-Based Foraging	99

5.4	Challenges of MARL	102
5.4.1	Non-Stationarity	102
5.4.2	Equilibrium Selection	104
5.4.3	Multi-Agent Credit Assignment	106
5.4.4	Scaling to Many Agents	108
5.5	What Algorithms Do Agents Use?	109
5.5.1	Self-Play	110
5.5.2	Mixed-Play	111
5.6	Summary	112
6	Multi-Agent Reinforcement Learning: Foundational Algorithms	115
6.1	Dynamic Programming for Games: Value Iteration	116
6.2	Temporal-Difference Learning for Games: Joint-Action Learning	118
6.2.1	Minimax Q-Learning	121
6.2.2	Nash Q-Learning	123
6.2.3	Correlated Q-Learning	124
6.2.4	Limitations of Joint-Action Learning	125
6.3	Agent Modeling	127
6.3.1	Fictitious Play	128
6.3.2	Joint-Action Learning with Agent Modeling	131
6.3.3	Bayesian Learning and Value of Information	134
6.4	Policy-Based Learning	140
6.4.1	Gradient Ascent in Expected Reward	141
6.4.2	Learning Dynamics of Infinitesimal Gradient Ascent	142
6.4.3	Win or Learn Fast	145
6.4.4	Win or Learn Fast with Policy Hill Climbing	147
6.4.5	Generalized Infinitesimal Gradient Ascent	149
6.5	No-Regret Learning	151
6.5.1	Unconditional and Conditional Regret Matching	151
6.5.2	Convergence of Regret Matching	153
6.6	Summary	156
II	MULTI-AGENT DEEP REINFORCEMENT LEARNING: ALGORITHMS AND PRACTICE	159
7	Deep Learning	161
7.1	Function Approximation for Reinforcement Learning	161
7.2	Linear Function Approximation	163
7.3	Feedforward Neural Networks	165

7.3.1	Neural Unit	166
7.3.2	Activation Functions	167
7.3.3	Composing a Network from Layers and Units	168
7.4	Gradient-Based Optimization	169
7.4.1	Loss Function	170
7.4.2	Gradient Descent	171
7.4.3	Backpropagation	174
7.5	Convolutional and Recurrent Neural Networks	175
7.5.1	Learning from Images—Exploiting Spatial Relationships in Data	176
7.5.2	Learning from Sequences with Memory	178
7.6	Summary	180
8	Deep Reinforcement Learning	183
8.1	Deep Value Function Approximation	184
8.1.1	Deep Q-Learning—What Can Go Wrong?	184
8.1.2	Moving Target Problem	187
8.1.3	Breaking Correlations	188
8.1.4	Putting It All Together: Deep Q-Networks	192
8.1.5	Beyond Deep Q-Networks	193
8.2	Policy Gradient Algorithms	195
8.2.1	Advantages of Learning a Policy	195
8.2.2	Policy Gradient Theorem	197
8.2.3	REINFORCE: Monte Carlo Policy Gradient	200
8.2.4	Actor-Critic Algorithms	202
8.2.5	A2C: Advantage Actor-Critic	204
8.2.6	PPO: Proximal Policy Optimization	207
8.2.7	Policy Gradient Algorithms in Practice	209
8.2.8	Concurrent Training of Policies	210
8.3	Observations, States, and Histories in Practice	215
8.4	Summary	217
9	Multi-Agent Deep Reinforcement Learning	219
9.1	Training and Execution Modes	220
9.1.1	Centralized Training and Execution	220
9.1.2	Decentralized Training and Execution	221
9.1.3	Centralized Training with Decentralized Execution	222
9.2	Notation for Multi-Agent Deep Reinforcement Learning	222
9.3	Independent Learning	223
9.3.1	Independent Value-Based Learning	224

9.3.2	Independent Policy Gradient Methods	226
9.3.3	Example: Deep Independent Learning in a Large Task	228
9.4	Multi-Agent Policy Gradient Algorithms	230
9.4.1	Multi-Agent Policy Gradient Theorem	231
9.4.2	Centralized Critics	232
9.4.3	Centralized Action-Value Critics	236
9.4.4	Counterfactual Action-Value Estimation	237
9.4.5	Equilibrium Selection with Centralized Action-Value Critics	239
9.5	Value Decomposition in Common-Reward Games	242
9.5.1	Individual-Global-Max Property	244
9.5.2	Linear Value Decomposition	246
9.5.3	Monotonic Value Decomposition	249
9.5.4	Value Decomposition in Practice	255
9.5.5	Beyond Monotonic Value Decomposition	261
9.6	Agent Modeling with Neural Networks	266
9.6.1	Joint-Action Learning with Deep Agent Models	267
9.6.2	Learning Representations of Agent Policies	271
9.7	Environments with Homogeneous Agents	274
9.7.1	Parameter Sharing	276
9.7.2	Experience Sharing	278
9.8	Policy Self-Play in Zero-Sum Games	281
9.8.1	Monte Carlo Tree Search	283
9.8.2	Self-Play MCTS	286
9.8.3	Self-Play MCTS with Deep Neural Networks: AlphaZero	288
9.9	Population-Based Training	290
9.9.1	Policy Space Response Oracles	292
9.9.2	Convergence of PSRO	295
9.9.3	Grandmaster Level in <i>StarCraft II</i> : AlphaStar	298
9.10	Summary	301
10	Multi-Agent Deep Reinforcement Learning in Practice	305
10.1	The Agent-Environment Interface	305
10.2	MARL Neural Networks in PyTorch	307
10.2.1	Seamless Parameter Sharing Implementation	309
10.2.2	Defining the Models: An Example with IDQN	310
10.3	Centralized Value Functions	312
10.4	Value Decomposition	313

10.5	Practical Tips for MARL Algorithms	313
10.5.1	Stacking Time Steps vs. Recurrent Network vs. Neither	314
10.5.2	Standardizing Rewards	314
10.5.3	Centralized Optimization	315
10.6	Presentation of Experimental Results	316
10.6.1	Learning Curves	316
10.6.2	Hyperparameter Search	318
11	Multi-Agent Environments	319
11.1	Criteria for Choosing Environments	320
11.2	Structurally Distinct 2×2 Matrix Games	321
11.2.1	No-Conflict Games	321
11.2.2	Conflict Games	322
11.3	Complex Environments	323
11.3.1	Level-Based Foraging	324
11.3.2	Multi-Agent Particle Environment	326
11.3.3	StarCraft Multi-Agent Challenge	327
11.3.4	Multi-Robot Warehouse	328
11.3.5	Google Research Football	329
11.3.6	Hanabi	330
11.3.7	Overcooked	331
11.4	Environment Collections	332
11.4.1	Melting Pot	333
11.4.2	OpenSpiel	334
11.4.3	Petting Zoo	335
A	Surveys on Multi-Agent Reinforcement Learning	337
	References	341
	Index	363

Summary of Notation

Sets are denoted with capital letters.

Elements of sets are denoted with lower-case letters.

Time index t (or τ) is shown in superscript (e.g., s^t denotes state at time t).

Agent index is shown in subscript (e.g., a_i denotes action of agent i).

The most common symbols used in the book are listed below. Specific sections may use additional notation.

General

$\mathbb{R}$	set of real numbers
$\propto$	proportional to
$x^\top$	transpose of a vector x
$X^\top$	transpose of a matrix X
$\Pr$	probability
$\Pr(x y)$	conditional probability of x given y
$\mathbb{E}_p[x]$	expectation of x under probability distribution p
$x \sim p$	x sampled according to probability distribution p
$x \leftarrow y$	assign value y to variable x
$\mathcal{D}$	training data set
$\frac{\partial f}{\partial x}$	derivative of function f with respect to x
∇	gradient
$\langle a, b, c, \dots \rangle$	concatenation of inputs $a, b, c, \dots$ into tuple $(a, b, c, \dots)$
$[x]_1$	indicator function: returns 1 if x is true, otherwise returns 0

Game Model

I	set of agents
i, j	agent subscripts
$-i$	subscript to denote the tuple (all agents except agent i)
$S, \bar{S}$	state space, set of terminal states
s	state
O, O_i	(joint-) observation space, observation space of agent i
o, o_i	(joint) observation, observation of agent i
A, A_i	(joint-) action space, action space of agent i
a, a_i	(joint) action, action of agent i

r, r_i	(joint) reward, reward of agent i
μ	initial state distribution
$\mathcal{T}$	state transition function
$\hat{\mathcal{T}}$	simulation/sampling model of state transitions
$\mathcal{O}, \mathcal{O}_i$	observation function (of agent i)
$\mathcal{R}, \mathcal{R}_i$	reward function (of agent i)
Γ_s	normal-form game for state s

Policies, Returns, Values

Π, Π_i	(joint-) policy space, policy space of agent i
π, π_i	(joint) policy, policy of agent i
π^*	optimal policy, or equilibrium joint policy
$H, \hat{H}$	set of histories, set of full histories
h, h_i	joint-observation history, observation history of agent i
$\hat{h}$	full history containing states, joint observations, joint actions
$\sigma(\hat{h})$	function returning joint-observation history from full history $\hat{h}$
γ	discount factor
u, u_i	discounted return (for agent i)
U, U_i	expected discounted return (for agent i)

(Multi-Agent) Reinforcement Learning

$\mathbb{L}$	learning algorithm
α	learning rate
ϵ	exploration rate
$\bar{\pi}_i$	empirical action distribution, or averaged policy, of agent i
$\hat{\pi}_j$	agent model for agent j
BR_i	set of best-response actions or policies for agent i
V^π, V_i^π	state-value function (of agent i) under policy π
Q^π, Q_i^π	action-value function (of agent i) under policy π
V^*, Q^*	optimal state/action-value function
Value_i	return an equilibrium value for agent i in a normal-form game

Deep Learning

θ	network parameters
$f(x; \theta)$	function f for input x with parameters θ
$\mathcal{L}(\theta)$	loss function over parameters θ
$\mathcal{B}$	batch of data
B	batch size, i.e., number of samples in a batch

(Multi-Agent) Deep Reinforcement Learning

θ, θ_i	value function parameters (of agent i)
--------------------	---

ϕ, ϕ_i	policy parameters (of agent i)
$\bar{\theta}$	target network parameters
$\mathcal{D}, \mathcal{D}_i$	experience replay buffer (of agent i)
$\mathcal{H}$	entropy
z	centralized information, e.g. the state of the environment

List of Figures

1.1	Schematic of a multi-agent system.	2
1.2	A level-based foraging task.	4
1.3	Schematic of multi-agent reinforcement learning.	6
1.4	Dimensions in multi-agent reinforcement learning.	8
2.1	Definition of a reinforcement learning problem.	20
2.2	Basic reinforcement learning loop for a single-agent system.	21
2.3	Mars Rover Markov decision process (MDP).	23
2.4	Sarsa and Q-learning algorithms in the Mars Rover problem.	37
3.1	Hierarchy of game models.	44
3.2	Three normal-form games with two agents (i.e., matrix games).	46
3.3	Three game models as directed cyclic graphs.	50
3.4	Level-based foraging environment with partial observability.	53
3.5	Synonymous terms in reinforcement learning and game theory.	59
4.1	Definition of a multi-agent reinforcement learning problem.	62
4.2	Matrix game in which ϵ -Nash equilibrium can be far from Nash equilibrium.	71
4.3	Chicken matrix game.	73
4.4	Feasible joint rewards and Pareto frontier in Chicken game.	77
4.5	Feasible joint rewards and fairness-optimal outcomes in Battle of the Sexes game.	80
4.6	Ten episodes between two agents in the non-repeated Prisoner's Dilemma matrix game.	82
4.7	Instances of END-OF-LINE problem.	85
5.1	General learning process in multi-agent reinforcement learning.	90

5.2	Convergence of “infinitesimal” independent Q-learning (IQL).	99
5.3	A level-based foraging task with two agents.	100
5.4	Central and independent Q-learning in a level-based foraging task.	101
5.5	Evolving policies of two agents in Rock-Paper-Scissors with WoLF-PHC.	103
5.6	Matrix games with multiple equilibria.	105
5.7	A level-based foraging task with three agents.	107
6.1	Simplified grid-world soccer game.	122
6.2	Minimax Q-learning won episodes and episode length in soccer game.	122
6.3	NoSDE (“No Stationary Deterministic Equilibrium”) game.	126
6.4	General agent model.	128
6.5	Evolving policies of two agents in Rock-Paper-Scissors with fictitious play.	129
6.6	First ten episodes in the non-repeated Rock-Paper-Scissors game when both agents use fictitious play.	130
6.7	Joint-action learning with agent modeling in a level-based foraging task.	133
6.8	Two agent models for Prisoner’s Dilemma.	135
6.9	Value of information in the Prisoner’s Dilemma example.	136
6.10	Dirichlet distributions in the Rock-Paper-Scissors non-repeated game.	137
6.11	Joint policy (α, β) learned by Infinitesimal Gradient Ascent (IGA) in unconstrained space.	144
6.12	General form of joint policy (α, β) trajectory when using WoLF-IGA.	146
6.13	Evolving policies of two agents in Rock-Paper-Scissors with WoLF-PHC.	149
6.14	Evolving policies of two agents in Rock-Paper-Scissors using unconditional regret matching.	155
6.15	Unconditional regrets in Rock-Paper-Scissors using unconditional regret matching.	156
7.1	Single-agent maze environment.	162
7.2	Feedforward neural network with three layers.	165

7.3	Single unit of a feedforward neural network.	166
7.4	Summary of commonly used activation functions for input $x \in \mathbb{R}$.	167
7.5	Common non-linear activation functions defined in Figure 7.4.	168
7.6	Training loop for gradient-based optimization of a neural network.	169
7.7	Comparison of gradient-based optimization algorithms.	173
7.8	Convolutional neural network.	177
7.9	Recurrent neural network.	179
8.1	Neural network architecture for action-value functions.	185
8.2	Correlations of consecutive experiences.	190
8.3	Single-agent level-based foraging environment and learning curves for value-based deep RL algorithms.	192
8.4	Comparison of ϵ -greedy and softmax policies.	196
8.5	Variance and bias trade-off of N -step returns.	204
8.6	Learning curves for policy gradient RL algorithms in single-agent level-based foraging environment.	210
8.7	Synchronous data collection for parallel training of the agent.	211
8.8	Learning curves for A2C with synchronous data collection in single-agent level-based foraging environment.	213
8.9	Asynchronous training for parallel optimization of the agent.	214
9.1	Learning curves of IA2C in the level-based foraging environment.	230
9.2	Architecture of a centralized critic.	233
9.3	Speaker-listener game and learning curve for A2C with and without a centralized critic.	235
9.4	Architecture of a centralized action-value critic.	237
9.5	The Stag Hunt and Climbing matrix games.	240
9.6	Learning curves comparing A2C with centralized critic and Pareto actor-critic.	242
9.7	A coordination graph.	244
9.8	Network architectures of VDN and QMIX.	250
9.9	Value decomposition format.	255
9.10	Linearly-decomposable matrix game with the learned value decompositions of VDN and QMIX.	256
9.11	Monotonically-decomposable matrix game with the learned value decompositions of VDN and QMIX.	257

9.12	QMIX mixing function in the monotonic matrix game.	257
9.13	Two-step common-reward stochastic game.	258
9.14	Learned value decompositions of VDN and QMIX in two-step common-reward stochastic game.	258
9.15	Learned value decomposition of VDN and QMIX in the Climbing game.	259
9.16	Level-based foraging task and learning curves comparing IDQN, VDN, and QMIX.	260
9.17	QTRAN value decomposition in the linear matrix game.	264
9.18	QTRAN value decomposition in the monotonic matrix game.	264
9.19	QTRAN value decomposition in the Climbing game.	265
9.20	Level-based foraging environment and learning curve for IDQN and deep JAL-AM.	270
9.21	Encoder-decoder architecture to learn representations of other agents' policies.	271
9.22	Level-based foraging environment and learning curve for centralized A2C with and without representation-based agent models.	273
9.23	Environments with strongly and weakly homogeneous agents.	276
9.24	Learning curves for the independent actor-critic algorithm in the level-based foraging environment.	277
9.25	Tree expansion and backpropagation in MCTS.	285
9.26	State transformation in chess.	287
9.27	AlphaZero match results.	290
9.28	Policy space response oracles (PSRO) steps.	294
9.29	PSRO in the non-repeated Rock-Paper-Scissors game.	297
10.1	Architecture for an independent Deep Q-networks algorithm.	310
10.2	Exemplary learning curves in single-agent and multi-agent reinforcement learning.	317
11.1	List of multi-agent environments with their properties.	324
11.2	Two level-based foraging environments.	325
11.3	Three multi-agent particle environment tasks.	326
11.4	StarCraft Multi-Agent Challenge tasks.	328
11.5	Three multi-robot warehouse tasks.	329

11.6	Two Google Research Football tasks.	330
11.7	Cooking Zoo.	332
11.8	Four Melting Pot environments.	333
11.9	Three Petting Zoo environments.	334

Preface

Multi-agent reinforcement learning (MARL) is a varied and highly active field of research. With the introduction of deep learning to MARL in the mid-2010s, the field has seen an explosive growth of activity, and now all major artificial intelligence and machine learning conferences routinely feature papers that develop new MARL algorithms or apply MARL in some way. This steep growth is also documented by the increasing number of survey papers that have been published since, of which we list many in Chapter A.

In the wake of this growth, it became clear that the field needed a textbook to provide a principled introduction to MARL. The present book is in part based on the tutorial “Multiagent Learning: Foundations and Recent Trends” given by Stefano V. Albrecht and Peter Stone at the 2017 International Joint Conference on Artificial Intelligence in Melbourne, Australia. The book was written to provide a basic introduction to the models, solution concepts, algorithmic ideas, and technical challenges in MARL, and to describe modern approaches in MARL that integrate deep learning techniques to produce powerful new algorithms. In essence, we believe that the materials covered in this book should be known by every MARL researcher. In addition, the book aims to give practical guidance for researchers and practitioners when using MARL algorithms. To this end, the book comes with its own codebase written in the Python programming language, which contains implementations of several MARL algorithms discussed in this book. The primary purpose of the codebase is to provide algorithm code that is self-contained and easy to read, to aid the reader’s understanding. A dedicated chapter (Chapter 10) describes of how these MARL algorithms are implemented in the codebase.

This book assumes that readers have an undergraduate-level background in basic mathematics, including statistics, probability theory, linear algebra, and calculus. A basic familiarity with programming concepts is required to understand and use the codebase. In general, we recommend reading the book chapters in the given sequence. For readers unfamiliar with reinforcement

learning and deep learning, we provide the basics in Chapters 2, 7, and 8, respectively. Readers who are already familiar with reinforcement learning and deep (reinforcement) learning, and who want to quickly get going with deep learning-based MARL algorithms, may read Chapter 3 and then skip to Chapter 9 and onward. To aid lecturers in adopting this book, we have developed lecture slides (available from the book's website) that can be modified as required to suit the course's needs.

MARL has become a large field of research, and this book does not cover all aspects of MARL. For instance, there is a growing body of work on using communication in MARL, which is not covered in this book. This includes questions such as how agents can learn to communicate robustly when communication channels are noisy and unreliable; and how agents may use MARL to learn specialized communication protocols or languages for a given task. While this book does not focus on communication in MARL, the models introduced in this book are general enough to also represent communication actions (as described in Section 3.5). There has also been research on using evolutionary game theory for multi-agent learning, which is not covered in this book. (We recommend the excellent survey of Bloembergen et al. (2015).) Finally, with the steep rise of activity in MARL in recent years, it would be futile to write a book that tries to keep up with new algorithms. We instead focus on the foundational concepts and ideas in MARL, and refer to survey papers (including those listed in Chapter A) for a more complete list of algorithm developments.

Acknowledgements: We are grateful to many people who have worked with us or provided feedback during the writing of this book. Special thanks go to Elizabeth Swayze and Matthew Valades from the MIT Press who diligently guided us through the publication process. Many colleagues have provided valuable feedback and suggestions, and we are grateful to all of them (in alphabetical order): Christopher Amato, Ignacio Carlucho, Georgios Chalkiadakis, Francesco Da Dalt, Sam Dauncey, Alex Davey, Bertrand Decoster, Mhairi Dunion, Kousha Etessami, Aris Filos-Ratsikas, Elliot Fosong, Amy Greenwald, Dongge Han, Josiah Hanna, Leonard Hinceldey, Sarah Keren, Mykel Kochenderfer, Marc Lanctot, Stefanos Leonardos, Michael Littman, Luke Marris, Elle McFarlane, Trevor McInroe, Mahdi Kazemi Moghaddam, Frans Oliehoek, Georgios Papoudakis, Tabish Rashid, Michael Rovatsos, Rahat Santosh, Raul Steleac, Massimiliano Tamborski, Kale-ab Tessera, Callum Tilbury, Jeroen van Riel, and Zhu Zheng. We also thank the anonymous reviewers who reviewed the book for the MIT Press. The Mars Rover MDP from Figure 2.3 is based on a similar MDP created by Elliot Fosong and Adam Jelley for the reinforcement learning course at

the University of Edinburgh. The images in Figure 4.4 and Figure 4.5(b) were created for this book by Mahdi Kazemi Moghaddam. We are grateful to Karl Tuyls for announcing this book in his keynote talk at the AAMAS 2023 conference in London (29 May–2 June, 2023), during which the first preprint draft of the book was released.

Errata: Despite our best efforts, it is possible that some typos or imprecisions may have gone unnoticed. If you detect any errors, we would be much obliged if you could report them to us via e-mail at issues@marl-book.com.

Book website, codebase, slides: The full PDF version of this book and links to accompanying materials, including the codebase and lecture slides, can be found on the book website at www.marl-book.com.

February, 2024

Stefano V. Albrecht
Filippos Christianos
Lukas Schäfer

1 Introduction

Imagine a scenario in which a collective of autonomous agents, each capable of making its own decisions, must interact in a shared environment to achieve certain goals. The agents may have a shared goal, such as a fleet of mobile robots whose task is to collect and deliver goods within a large warehouse or a team of drones tasked with monitoring a power plant. The agents may also have conflicting goals, such as agents trading goods in a virtual market in which each agent seeks to maximize its own gains. Since we may not know how the agents should interact to achieve their goals, we tell them to figure it out on their own. Thus, the agents begin to try actions in their environment and collect experiences about how the environment changes as a result of their actions, as well as how the other agents behave. In time, the agents begin to learn various concepts, such as skills needed to solve their task and, importantly, how to *coordinate* their actions with other agents. They may even learn to develop a shared language to enable communication between agents. Finally, the agents reach a certain level of proficiency and have become experts at interacting optimally to achieve their goals.

This exciting vision is, in a nutshell, what *multi-agent reinforcement learning* (MARL) aims to achieve. MARL is based on *reinforcement learning* (RL), in which agents learn optimal decision policies by trying actions and receiving rewards, with the goal of choosing actions to maximize the sum of received rewards over time. While in single-agent RL the focus is on learning an optimal policy for a single agent, in MARL the focus is on learning optimal policies for multiple agents and the unique challenges that arise in this learning process.

In this first chapter, we will begin to outline some of the underlying concepts and challenges in MARL. We begin by introducing the concept of a multi-agent system, which is defined by an environment, the agents in the environment, and their goals. We then discuss how MARL operates in such systems to learn optimal policies for the agents, which we illustrate with a number of examples of potential applications. Next we discuss some of the key challenges in MARL,

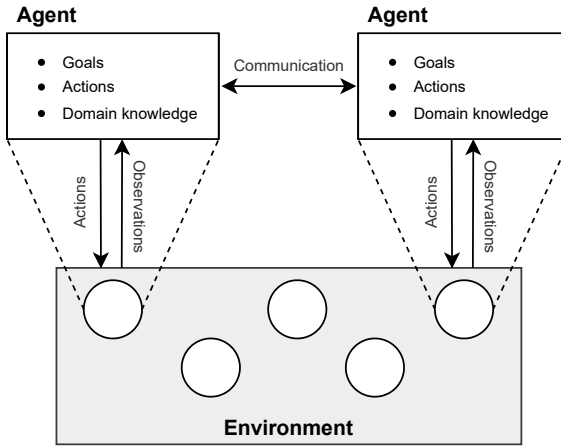


Figure 1.1: Schematic of a multi-agent system. A multi-agent system consists of an environment and multiple decision-making agents (shown as circles inside the environment). The agents can observe information about the environment and take actions to achieve their goals.

such as the non-stationarity and equilibrium selection problems, as well as several “agendas” of MARL that describe different ways in which MARL can be used. At the end of this chapter, we give an overview of the topics covered in the two parts of this book.

1.1 Multi-Agent Systems

A multi-agent system consists of an *environment* and multiple decision-making *agents* that interact in the environment to achieve certain goals. Figure 1.1 shows a general schematic of a multi-agent system, and we describe the basic components in the following.

Environment An environment is a physical or virtual world whose state evolves over time and is influenced by the actions of the agents that exist within the environment. The environment specifies the actions that agents can take at any point in time as well as the observations that individual agents receive about the state of the environment. The states of the environment may be defined as discrete or continuous quantities, or a combination of both. For example, in a 2D-maze environment, the state may be defined as the combination of the discrete integer positions of all agents together

with their continuous orientations in radians. Similarly, actions may be discrete or continuous, such as moving up/down/left/right in the maze or turning around by a specified continuous angle. Multi-agent environments are often characterized by the fact that agents only have a limited and imperfect view of the environment. This means that individual agents may only observe some partial information about the state of the environment, and different agents may receive different observations about the environment.

Agents An agent is an entity that receives information about the state of the environment and can choose different actions in order to influence the state. Agents may have different prior knowledge about the environment, such as the possible states that the environment can be in and how states are affected by the actions of the agents. Importantly, agents are goal-directed in the sense that agents have specified *goals* and choose their actions in order to achieve their goals. These goals could be to reach a certain environment state, or to maximize certain quantities such as monetary revenues. In MARL, such goals are defined by *reward functions* that specify scalar reward signals that agents receive after taking certain actions in certain states. The term *policy* refers to a function used by the agent to select actions (or assign probabilities to selecting each action) given the current state of the environment. If the environment is only partially observed by the agent, then the policy may be conditioned on the current and past observations of the agent.

As a concrete example of the above concepts, consider the level-based foraging example shown in Figure 1.2.¹ In this example, multiple robots are tasked with collecting items that are spatially distributed in a grid-world environment. Each robot and item has an associated skill level, and a group of one or more robots can collect an item if the robots are located next to the item and the sum of the robots' levels is greater than or equal to the item's level. The state of this environment at a given time is completely described by variables containing the x/y-positions of the robots and items, and binary variables for each item indicating whether the item still exists or not.² In this example, we use three independent agents to control each of the three robots. At any given time, each agent can observe the complete state of the environment and choose an action

1. The level-based foraging example will appear throughout this book. We provide an open-source implementation of this environment at <https://github.com/uoel-agents/lb-foraging>. See Section 11.3.1 for more details on this environment.

2. Note that the skill levels of robots and items are not included in the state since these are assumed constant. However, if the skill levels can change between different episodes, then the state may additionally include the skill levels of the robots and items.

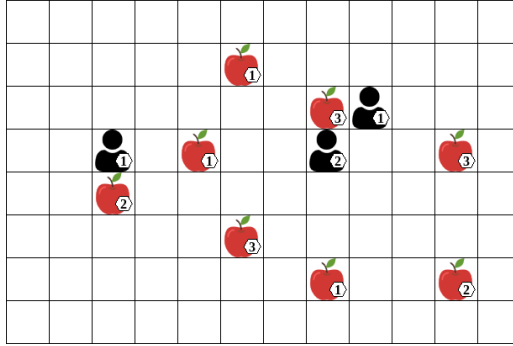


Figure 1.2: A level-based foraging task in which a group of three robots, each controlled by an agent, must collect all items (shown as apples). Each robot and item has an associated skill level, shown inset. A group of one or more robots can collect an item if they are located next to the item, and the sum of the robots’ levels is greater than or equal to the item’s level.

from the set $\{up, down, left, right, collect, noop\}$ to control its robot. The first four actions in this action set modify the x/y -position of the robot in the state by moving the robot into the respective direction (unless the robot is already at the edge of the grid-world, in which case the action has no effect). The *collect* action attempts to collect an item that is located adjacent to the robot; if the item is collected, the action modifies the binary existence variable corresponding to the item. The *noop* (no operation) action has no effect on the state.

Notice that in the previous description, we used the terms “robot” and “agent” to refer to two distinct concepts. In level-based foraging, the word “robot” is a label for an object that is explicitly represented via the x/y -position variables. Similarly, the word “item” refers to an object in level-based foraging that is represented by its x/y -position variables and its binary existence variable. In contrast to these object labels, the term “agent” refers to an abstract decision-making entity that observes some information from the environment and chooses values for certain action variables, in this case, the action for a robot. If there is a direct one-to-one correspondence between agents and certain objects, such as agents and robots in level-based foraging, then it can be convenient to use the terms interchangeably. For example, in level-based foraging, we may say “the skill level of agent i ” when referring to the skill level of the robot controlled by agent i . Unless the distinction is relevant, in this book, we will often use the term “agent” synonymously with the object that it controls.

The defining characteristic of a multi-agent system is that the agents must coordinate their actions with (or against) each other to achieve their goals. In a

fully cooperative scenario, the agents' goals are perfectly aligned so that the agents need to collaborate toward achieving a shared goal. For example, in level-based foraging, all agents may receive a reward of +1 whenever an item is successfully collected by any of the agents. In a competitive scenario, the agents' goals may be diametrically opposed so that the agents are in direct competition with each other. An example of such a competitive scenario is two agents playing a game of chess, in which the winning player gets a reward of +1 and the losing player gets a reward of -1 (or both get 0 for a drawn outcome). In between these two extremes, the agents' goals may align in some respects while differing in other respects, which can lead to complex multi-agent interaction problems that involve both cooperation and competition to varying degrees. For example, in the actual implementation of level-based foraging we use in this book (described in Section 11.3.1), only those agents that were involved in the collection of an item (rather than all agents) will receive a positive normalized reward. Thus, the agents have a motivation to maximize their own returns (sum of rewards), which can lead them to try and collect items before other agents can do so; but they may also need to collaborate with other agents at certain times in order to collect an item.

The previously described concepts of states, actions, observations, and rewards are formally defined within *game models*. Different types of game models exist, and Chapter 3 introduces the most common game models used in MARL, including normal-form games, stochastic games, and partially observable stochastic games. A solution for a game model consists of a set of policies for the agents that satisfies certain desired properties. As we will see in Chapter 4, there exist a range of solution concepts in the general case. Most solution concepts are anchored in some notion of *equilibrium*, which means that no individual agent can deviate from its policy in the solution to improve its outcome.

Research in multi-agent systems has a long history in artificial intelligence and spans a vast range of technical problems (Shoham and Leyton-Brown 2008; Wooldridge 2009). These include questions such as how to design algorithms that enable agents to choose optimal actions toward their specified goals; how to design environments to incentivize certain long-term behaviors in agents; how information is communicated and propagated among agents; and how norms, conventions, and roles may emerge in a collective of agents. This book is concerned with the first of these questions, with a focus on using RL techniques to optimize and coordinate the policies of agents in order to maximize the rewards they accumulate over time.

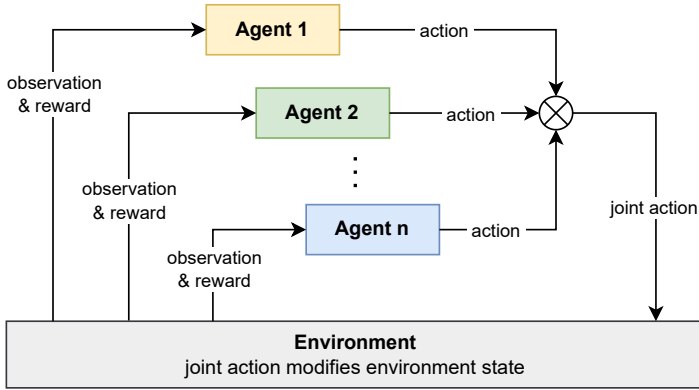


Figure 1.3: Schematic of multi-agent reinforcement learning. A set of n agents receive individual observations about the state of the environment, and choose actions to modify the state of the environment. Each agent then receives a scalar reward and a new observation, and the loop repeats.

1.2 Multi-Agent Reinforcement Learning

Multi-agent reinforcement learning (MARL) algorithms learn optimal policies for a set of agents in a multi-agent system.³ As in the single-agent counterpart, the policies are learned via a process of trial-and-error to maximize the agents' cumulative rewards, or *returns*. Figure 1.3 shows a basic schematic of the MARL training loop. A set of n agents choose individual actions, which together are referred to as the *joint action*. The joint action changes the state of the environment according to the environment dynamics, and the agents receive individual rewards as a result of this change as well as individual observations about the new environment state. This loop continues until a terminal criterion is satisfied (such as one agent winning a game of chess) or indefinitely. A complete run of this loop from the initial state to the terminal state is called an *episode*. The generated data produced from multiple independent episodes — that is, the experienced observations, actions, and rewards in each episode — are used to continually improve the agents' policies.

In the level-based foraging environment introduced in the previous section, each agent $i \in \{1, 2, 3\}$ observes the full environment state and chooses an action $a_i \in \{up, down, left, right, collect, noop\}$. Given the joint action (a_1, a_2, a_3) , the

3. This book uses a literal definition of the MARL term in which we learn policies for *multiple* agents. This is in contrast to learning a policy for a single agent that operates in a multi-agent system in which we have no control over the other agents.

environment state transitions into a new state by modifying the x/y-positions of the robots and the binary existence variables of the items, depending on the chosen actions in the joint action. Then each agent receives a reward, such as +1 if any of the items has been collected and 0 otherwise, and observes the new state of the environment. An episode in level-based foraging terminates once all items have been collected, or after a maximum number of allowed time steps. Initially, each agent starts with a random policy that selects actions randomly. As the agents keep trying different actions in different states and observe the resulting rewards and new states, they will change their policies to select actions in each state that will maximize the sum of received rewards.

The MARL loop shown in Figure 1.3 is analogous to the single-agent RL loop (which will be covered in Chapter 2) and extends it to multiple agents. There are several important use cases in which MARL can have significant benefits over single-agent RL. One use case for MARL is to decompose a large, intractable decision problem into smaller, more tractable decision problems. To illustrate this idea, consider again the level-based foraging example shown in Figure 1.2. If we view this as a single-agent RL problem, then we have to train a single central agent that selects actions for each of the three robots. Thus, an action of the central agent is defined by the tuple (a_1, a_2, a_3) , in which a_i specifies what robot i does. This results in a decision problem with $6^3 = 216$ possible actions for the central agent in every time step. Even in this toy example, most standard single-agent RL algorithms do not scale easily to action spaces this large. However, we can decompose this decision problem by introducing three independent agents, one for each robot, such that each agent faces a decision problem with only six possible actions in each time step. Of course, this decomposition introduces a new challenge, which is that the agents need to coordinate their actions in order to be successful. MARL algorithms may use various approaches to facilitate the learning of coordinated agent policies.

Even if we were able to solve the previous example using single-agent RL to train a central agent, this approach rests on the implicit assumption that the environment allows for centralized control. However, in many applications of multi-agent systems, it may not be possible to control and coordinate the actions of multiple agents from a central place. Examples include autonomous driving in urban environments, where each car requires its own local policy to drive, and a team of mobile robots used in search-and-rescue missions, where it may not be possible to communicate with a central coordinator and so each agent (robot) may need to act fully independently. In such applications, the agents may need to learn *decentralized* policies, where each agent executes its own policy locally based on its own observations. For such applications, MARL algorithms can learn agent policies that can be executed in a decentralized fashion.

Dimension	Questions
Size	How many agents exist in the environment? Is the number of agents fixed, or can it change? How many states and actions does the environment specify? Are states/actions discrete or continuous? Are actions defined as single values or multi-valued vectors? <i>(Chapter 3)</i>
Knowledge	Do agents know what actions are available to themselves and to other agents? Do they know their own reward functions, and the reward functions of other agents? Do agents know the state transition probabilities of the environment? <i>(Chapter 3)</i>
Observability	What can agents observe about their environment? Can agents observe the full environment state, or are their observations partial and noisy? Can they observe the actions and/or the rewards of other agents? <i>(Chapter 3)</i>
Rewards	Are the agents opponents, with zero-sum rewards? Or are agents teammates, with common (shared) rewards? Or do agents have to compete <i>and</i> cooperate in some way? <i>(Chapter 3)</i>
Objective	Is the agents' goal to learn an equilibrium joint policy? What type of equilibrium? Is performance during learning important, or only the final learned policies? Is the goal to perform well against certain classes of other agents? <i>(Chapters 4 and 5)</i>
Centralization & Communication	Can agents coordinate their actions via a central controller or coordinator? Or do agents learn fully independent policies? Can agents share/communicate information during learning and/or after learning? Is the communication channel reliable, or noisy and unreliable? <i>(Chapters 3, 5, 6, and 9)</i>

Figure 1.4: Dimensions in MARL and relevant book chapters for further details.

MARL algorithms can be categorized based on a number of dimensions, as shown in Figure 1.4. For example, this includes assumptions about the agents' rewards (e.g., fully cooperative, competitive, or mixed), what type of solution concept the algorithm is designed to achieve (e.g., Nash equilibrium), and what agents can observe about their environment. Algorithms can also be categorized based on assumptions made during the learning of agent policies ("training") versus assumptions made after learning ("execution"). Centralized training and execution assumes that both stages have access to some centrally shared mechanism or information, such as sharing all observations between agents. For example, a single central agent may receive information from all other agents and dictate the actions to the agents. Such centralization can help to improve coordination between agents and alleviate issues such as non-stationarity (discussed in Section 1.4). In contrast, decentralized training and execution assumes no such centrally shared information, and instead requires that the learning of an agent's policy as well as the policy itself only use the local information of that agent. The third major category, centralized training with decentralized execution, aims to combine the benefits of the two aforementioned approaches, by assuming that centralization is feasible during training (e.g., in simulation) while producing policies that can be executed in a fully decentralized way. These ideas will be further discussed in Chapter 9.

1.3 Application Examples

We provide several examples to illustrate the MARL training loop shown in Figure 1.3 and its different constituent elements, such as agents, observations, actions, and rewards. Each example is based on a potential real-world application, and we give pointers to works that have used MARL to develop such applications.

1.3.1 Multi-Robot Warehouse Management

Imagine a large warehouse consisting of many aisles of storage racks that contain all manner of items. There is a constant stream of orders, which specify certain items and quantities to be picked up from the storage racks and delivered to a work station for further processing. Suppose we have one hundred mobile robots that can move along the aisles and pick items from the storage racks. We can use MARL to train these robots to collaborate optimally to service the incoming orders, with the goal of completing the orders as quickly and efficiently as possible. In this application, each robot could be controlled by an independent agent, so we would have 100 agents. Each agent might observe

information about its own location and current heading within the warehouse, the items it is carrying, and the current order it is servicing. It might also observe information about other agents, such as their locations, items, and orders. The actions of an agent may include physical movements, such as rotating toward a certain direction and accelerating/braking as well as picking items. Actions might also include sending communication messages to other robots, which could for example contain information about the travel plans of the communicating agent. Lastly, each agent might receive an individual positive reward when completing an order, which includes picking all items at the specified quantities in the order. Alternatively, the agents may all receive a collective reward when any order has been completed by any robot. The latter case, when all agents receive identical rewards, is called *common reward* (or shared reward) and is an important special case in MARL, discussed further in Chapter 3. Krnjaic et al. (2024) used MARL algorithms for multi-robot warehouse applications. A simple simulator of a multi-robot warehouse is described in Section 11.3.4.

1.3.2 Competitive Play in Board Games and Video Games

MARL can be used to train agents to achieve strong competitive play in board and card games (e.g., Backgammon, Chess, Go, Poker) and multi-player video games (e.g., shooting games, racing games, and other games). Each agent assumes the role of one of the players in the game. Agents may have actions available to move individual pieces or units to specific positions, placing specific cards, shooting target units, and other actions. Agents may observe the full game state, such as the entire game board with all pieces, or they may receive only a partial observation, such as their own cards but not the cards of other players or a partial view of the game map. Depending on the rules and mechanics of the game, the agents may or may not observe the chosen actions of other agents. In fully competitive games with two agents, one agent’s reward is the negative of the other agent’s reward. Thus, an agent may receive a reward of +1 for winning the game, in which case the other losing agent will receive a reward of -1, and vice versa. This property is referred to as *zero-sum reward* and is another important special case in MARL. With this setup, during MARL training, the agents will learn to exploit each other’s weaknesses and improve their play to eliminate their own weaknesses, leading to strong competitive play. Many different types of board games, card games, and video games have been tackled using MARL approaches (Tesauro 1994; Silver et al. 2018; Vinyals et al. 2019; Bard et al. 2020; Meta Fundamental AI Research Diplomacy Team et al. 2022; Pérolat et al. 2022).

1.3.3 Autonomous Driving

Autonomous driving in urban environments and highways involves frequent interactions with other vehicles. Using MARL, we could train control policies for multiple vehicles to navigate through complicated interaction scenarios, such as driving through busy junctions and roundabouts and merging onto highways. The actions of an agent might be the continuous controls for a vehicle, such as steering and acceleration/braking, or discrete actions such as deciding between different maneuvers to execute (e.g., change lane, turning, overtaking). An agent may receive observations about its own controlled vehicle (e.g., position on lane, orientation, and speed) as well as observations about other nearby vehicles. Observations about other vehicles may be uncertain due to sensor noise, and they may be *incomplete* due to partial observability caused by occlusions (e.g., other vehicles blocking the agent's view). The reward of each agent can involve multiple factors. At a basic level, agents must avoid collisions and so any collision would result in a large negative reward. In addition, we want the agents to produce efficient and natural driving behavior, so there may be positive rewards for minimizing driving times, and negative rewards for abrupt acceleration/braking and frequent lane changes. Therefore, in contrast to the multi-robot warehouse (agents have the same goal) and game playing (agents have opposed goals), here we have a mixed-motive scenario in which agents collaborate to avoid collisions but are also self-interested based on their desire to minimize driving times and drive smoothly. This case is referred to as *general-sum reward* and is among the most challenging tasks in MARL. MARL algorithms have been applied to a range of autonomous driving tasks (e.g., Shalev-Shwartz, Shammah, and Shashua 2016; Peake et al. 2020; Zhou, Luo, et al. 2020; Dinneweth et al. 2022; Zhou et al. 2022).

1.3.4 Automated Trading in Electronic Markets

Software agents can be developed to assume the roles of traders in electronic markets (Wellman, Greenwald, and Stone 2007). The typical objective of agents in a market is to maximize their own returns by placing buying and selling orders. Thus, agents have actions to buy or sell commodities at specified times, prices, and quantities. Agents receive observations about price developments in the market and other key performance indicators, and possibly some information about the current state of the order book. In addition, the agents may need to model and monitor external events and processes based on diverse types of observed information, such as news pertaining to certain companies, or energy demand and usage of own managed households in peer-to-peer energy markets. The reward of an agent could be defined as a function of gains and losses

made over a certain period of time, for example at the end of each trading day, quarter, or year. Thus, trading in electronic markets is another example of a mixed-motive scenario, since the agents need to collaborate in some sense to agree on sell-buy prices while aiming to maximize their own individual gains. MARL algorithms have been proposed for different types of electronic markets, including financial markets and energy markets (Roesch et al. 2020; Qiu et al. 2021; Shavandi and Khedmati 2022).

1.4 Challenges of MARL

Various challenges exist in multi-agent reinforcement learning that stem from aspects such as that agents may have conflicting goals, that agents may have different partial views of their environment, and that agents are learning concurrently to optimize their policies. Next we outline some of the main challenges, which will be discussed in more detail in Chapter 5.

Non-stationarity caused by learning agents An important characteristic of MARL is non-stationarity caused by the continually changing policies of the agents during their learning processes. This non-stationarity can lead to a moving target problem because each agent adapts to the policies of other agents whose policies in turn also adapt to changes in other agents, thereby potentially causing cyclic and unstable learning dynamics. This problem is further exacerbated by the fact that the agents may learn different behaviors at different rates as a result of their different rewards and local observations. Thus, the ability to handle such non-stationarity in a robust way is often a crucial aspect in MARL algorithms and has been the subject of much research.

Optimality of policies and equilibrium selection When are the policies of agents in a multi-agent system *optimal*? In single-agent RL, a policy is optimal if it achieves maximum expected returns in each state. However, in MARL, the returns of one agent's policy also depend on the other agents' policies, and thus we require more sophisticated notions of optimality. Chapter 4 presents a range of solution concepts, such as equilibrium-type solutions in which each agent's policy is in some specific sense optimal with respect to the other agents' policies. In addition, while in the single-agent case all optimal policies yield the same expected return for the agent, in a multi-agent system (where agents may receive different rewards) there may be multiple equilibrium solutions, and each equilibrium may entail different returns for different agents. Thus, there is an additional challenge of agents having to essentially negotiate during learning which equilibrium

to converge to (Harsanyi and Selten 1988). A central goal of MARL research is to develop learning algorithms that can learn agent policies that robustly converge to a particular solution type.

Multi-agent credit assignment Temporal credit assignment in RL is the problem of determining which past actions contributed to a received reward. In MARL, this problem is compounded by the additional problem of determining *whose* action contributed to the reward. To illustrate, consider the level-based foraging example shown in Figure 1.2 and assume all agents choose the “collect” action, following which they receive a collective reward of +1. Given only this state/action/reward information, it can be highly non-trivial to disentangle the contribution of each agent to the received reward, in particular that the agent on the left did not contribute to the reward since its action had no effect (the agent’s level was not large enough). While ideas based on counterfactual reasoning can address this problem in principle, it is still an open problem how to resolve multi-agent credit assignment in an efficient and scalable way.

Scaling in number of agents In a multi-agent system, the total number of possible action combinations between agents may grow exponentially with the number of agents. This is particularly the case if each added agent comes with its own additional action variables. For example, in level-based foraging, each agent controls a robot and adding another agent comes with its own associated action variable to control a robot. (But see Section 5.4.4 for a counter-example without exponential growth.) In the early days of MARL research, it was common to use only two agents to avoid issues with scaling. Even with today’s deep learning-based MARL algorithms, it is common to use a number of agents between 2 and 10. How to handle many more agents in an efficient and robust way is an important goal in MARL research.

1.5 Agendas of MARL

An influential article by Shoham, Powers, and Grenager (2007) titled “*If multi-agent learning is the answer, what is the question?*” describes several distinct agendas that have been pursued in MARL research.⁴ The agendas differ in their

4. The article of Shoham, Powers, and Grenager (2007) was part of a special issue called “Foundations of Multi-Agent Learning” which was published in the *Artificial Intelligence* journal (Vohra and Wellman 2007). This special issue contains many interesting articles from some of the early contributors in MARL research, including responses to Shoham et al.’s article.

motivations and goals for using MARL, as well as the criteria by which progress and success are measured. In their article, Shoham et al. made the important point that one must be clear about the purpose and goals when using MARL. We describe the main agendas as follows:

Computational The computational agenda uses MARL as an approach to compute solutions for game models. A solution consists of a collection of decision policies for the agents that satisfy certain properties (e.g., Nash equilibrium and other solution concepts, discussed in Chapter 4). Once computed, a solution could be deployed in an application of the game to control the agents (see Section 1.3 for some examples), or it may be used to conduct further analysis of the game. Thus, in the computational agenda, MARL algorithms compete with other direct methods to compute game solutions (Nisan et al. 2007; Roughgarden 2016). Such direct methods can be significantly more efficient than MARL algorithms for certain types of games (such as the linear programming methods discussed in Sections 4.3.1 and 4.6.1), but they typically require full knowledge of the game, including the reward functions of all agents. In contrast, MARL algorithms are usually designed to learn solutions without full knowledge of the game.

Prescriptive The prescriptive agenda focuses specifically on the behaviors and performance of agents *during learning* and asks how they should learn to achieve a given set of criteria. Different criteria have been proposed in this regard. One possible criterion is that the average reward received by a learning agent should not fall below a certain threshold during learning, regardless of how other agents may be learning. An additional criterion might be that the learning agent should learn optimal actions if the other agents come from a certain class of agents (such as static, non-learning agents) and otherwise should not fall below a certain performance threshold (Powers and Shoham 2004). Such criteria focus on the behaviors of agents during learning, and may leave open whether the collective learning process converges to a particular equilibrium. Thus, convergence to a particular solution concept (e.g., equilibrium) is not necessarily the goal in this agenda.

Descriptive The descriptive agenda uses MARL to study the behaviors of agents, including natural agents such as humans and animals, when learning in a population. This agenda often begins by proposing a certain MARL algorithm that uses an idealized description of how the studied agents adapt their actions based on past interactions. Methods from social sciences and behavioral economics can be used to test how closely the MARL algorithm matches the behavior of the agents, such as via controlled experimentation

in a laboratory setting (Mullainathan and Thaler 2000; Camerer 2011; Drouvelis 2021). This is followed by an analysis, such as via methods based on evolutionary game theory (Bloembergen et al. 2015), of whether a population of such agents will converge to a certain kind of equilibrium solution if all agents use the proposed MARL algorithm.

Our perspective in this book is to view MARL as a method to optimize the decision policies of agents toward defined criteria. As such, the book primarily covers ideas and algorithms from the computational and prescriptive agendas. In particular, the computational agenda is closest to our perspective, which is reflected in the structure of the book by first introducing game models and solution concepts, followed by algorithms designed to learn such solutions. Using MARL to study the learning behaviors of natural and other agents, as in the descriptive agenda, is outside the scope of this book.

1.6 Book Contents and Structure

This book provides an introduction to the theory and practice of multi-agent reinforcement learning, suitable for university students, researchers, and practitioners. Following this introductory chapter, the remainder of the book is divided into two parts.

Part I of the book provides foundational knowledge about the basic models and concepts used in MARL. Specifically, Chapter 2 gives an introduction to the theory and tabular algorithms of single-agent RL. Chapter 3 introduces the basic game models to define concepts such as states, actions, observations, and rewards in a multi-agent environment. Chapter 4 then introduces a series of solution concepts that define what it means to solve these game models, that is, what it means for agents to act optimally. The final two chapters in this part of the book explore a range of MARL approaches for computing solutions in games: Chapter 5 presents basic concepts such as central and independent learning, and discusses core challenges in MARL. Chapter 6 introduces different classes of foundational algorithms developed in MARL research and discusses their learning properties.

Part II of the book focuses on contemporary research in MARL that leverages deep learning techniques to create new powerful MARL algorithms. We start by providing introductions to deep learning and deep reinforcement learning in Chapters 7 and 8, respectively. Building on the previous two chapters, Chapter 9 introduces several of the most important MARL algorithms developed in recent years, including ideas such as centralized training with decentralized

execution, value decomposition, parameter sharing, and population-based training. Chapter 10 provides practical guidance when implementing and using MARL algorithms and how to evaluate the learned policies. Finally, Chapter 11 describes examples of multi-agent environments that have been developed in MARL research.

One of the goals of this book is to provide a starting point for readers who want to use the MARL algorithms discussed in this book in practice, as well as develop their own algorithms. Thus, the book comes with its own MARL codebase (downloadable from the book's website) that was developed in the Python programming language, providing implementations of many existing MARL algorithms that are self-contained and easy to read. Chapter 10 uses code snippets from the codebase to explain implementation details of the important concepts underlying the algorithms presented in the earlier chapters. We hope that the provided code will be useful to readers in understanding MARL algorithms as well as getting started with using them in practice.

I FOUNDATIONS OF MULTI-AGENT REINFORCEMENT LEARNING

Part I of this book covers the foundations of multi-agent reinforcement learning. The chapters in this part focus on basic questions, including how to represent the mechanics of a multi-agent system via game models; how to define a learning objective in games to specify optimal agent behaviors; and how reinforcement learning methods can be used to learn optimal agent behaviors, as well as the complexities and challenges involved in multi-agent learning.

Chapter 2 provides an introduction to the basic models and algorithmic concepts of reinforcement learning, including Markov decision processes, dynamic programming, and temporal-difference learning. Chapter 3 then introduces game models to represent interaction processes in a multi-agent system, including basic normal-form games, stochastic games, and partially observable stochastic games. Chapter 4 introduces a range of solution concepts from game theory to define optimal agent policies in games, including equilibrium-type solutions such as minimax, Nash, and correlated equilibrium, as well as other concepts such as Pareto optimality, welfare/fairness, and no-regret. We provide examples for each solution concept and also discuss important conceptual limitations. Together, a game model and a solution concept define a learning problem in multi-agent reinforcement learning.

Building on the previous chapters, Chapters 5 and 6 look at how to use reinforcement learning techniques to learn optimal agent policies in a game. Chapter 5 begins by defining the general learning process in games and different convergence types, and introduces the basic concepts of central learning and independent learning that reduce the multi-agent learning problem to a single-agent learning problem. The chapter then discusses the central challenges in multi-agent reinforcement learning, including non-stationarity, equilibrium selection, multi-agent credit assignment, and scaling to many agents. Chapter 6 introduces several classes of foundational algorithms for multi-agent reinforcement learning that go beyond the basic approaches introduced in the prior chapter, and discusses their convergence properties.

2 Reinforcement Learning

Multi-agent reinforcement learning (MARL) is, in essence, reinforcement learning (RL) applied to multi-agent game models to learn optimal policies for the agents. Thus, MARL is deeply rooted in both RL theory and game theory. This chapter provides a basic introduction to the theory and algorithms of RL when there is only a single agent for which we want to learn an optimal policy. We will begin by providing a general definition of RL, following which we will introduce the Markov decision process (MDP) as the foundational model used in RL to represent single-agent decision processes. Based on the MDP model, we will define basic concepts such as expected returns, optimal policies, value functions, and Bellman equations. The goal in an RL problem is to learn an optimal policy that chooses actions to achieve some objective, such as maximizing the expected (discounted) return in each state of the environment (Figure 2.1).

We will then introduce two basic families of algorithms to compute optimal policies for MDPs: dynamic programming and temporal-difference learning. Dynamic programming requires complete knowledge of the MDP specification and uses this knowledge to compute optimal value functions and policies. In contrast, temporal-difference learning does not require complete knowledge of the MDP; instead, it learns optimal value functions and policies by interacting with the environment and generating experiences. Most of the MARL algorithms introduced in Chapter 6 build on these families of algorithms and essentially extend them to game models.

Part I of this book focuses on the basic models and concepts used in MARL, and as such this chapter focuses only on the basic RL concepts that are required to understand the following chapters of this part of the book. In particular, topics such as value and policy function approximation are not covered in this chapter. These latter topics will be covered in Chapters 7 and 8 in Part II of the book.

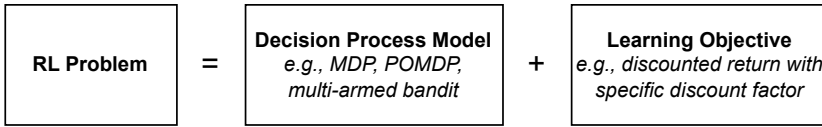


Figure 2.1: An RL problem is defined by the combination of a decision process model that defines the mechanics of the agent-environment interaction and a learning objective that specifies the properties of the optimal policy to be learned (e.g., maximize expected discounted return in each state).

2.1 General Definition

We begin by providing a general definition of reinforcement learning:

Reinforcement learning (RL) algorithms learn solutions for sequential decision processes via repeated interaction with an environment.

This definition raises three main questions:

- What is a sequential decision process?
- What is a solution to the process?
- What is learning via repeated interaction?

A sequential decision process is defined by an agent that makes decisions over multiple time steps within an environment to achieve a specified goal. In each time step, the agent receives an observation from the environment and chooses an action. In a fully observable setting, as we assume in this chapter, the agent observes the full state of the environment; but in general, observations may be incomplete and noisy. Given the chosen action, the environment may change its state according to some transition dynamics and send a scalar reward signal to the agent. Figure 2.2 summarizes this process.

A solution to the decision process is an optimal decision policy for the agent, which chooses actions in each state to achieve some specified learning objective. Typically, the learning objective is to maximize the expected return for the agent in each possible state.¹ The return in a state when following a given policy is defined as the sum of rewards received over time from that state onward. Thus, RL assumes that the goal in the decision process can, in principle, be framed as the maximization of expected returns.

1. Other learning objectives can be specified, such as maximizing the average reward (Sutton and Barto 2018) and different types of “risk-sensitive” objectives (Mihatsch and Neuneier 2002).

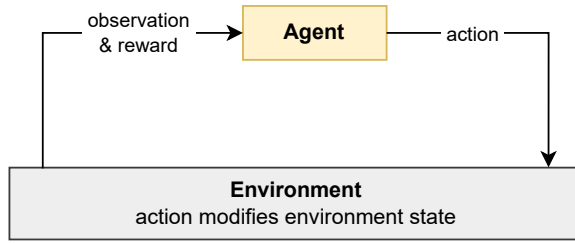


Figure 2.2: Basic reinforcement learning loop for a single-agent system.

Finally, RL algorithms learn such optimal policies by trying different actions in different states and observing the outcomes. This way of learning is sometimes described as “trial and error” since the actions may lead to positive or negative outcomes that are not known beforehand and, therefore, must be discovered by trying the actions. A central problem in this learning process, often called the *exploration-exploitation dilemma*, is how to balance exploring the outcomes of different actions versus sticking with actions that are currently believed to be best. Exploration may discover better actions but can accrue low rewards in the process, while exploitation achieves a certain level of returns but may not discover the optimal actions.

RL is a type of machine learning that differs from other types such as supervised learning and unsupervised learning. In supervised learning, we have access to a set of labeled input-output pairs $\{x_i, y_i\}$ of some unknown function $f(x_i) = y_i$, and the goal is to learn this function using the data. In unsupervised learning, we have access to some unlabeled data $\{x_i\}$ and the goal is to identify some useful structure within the data. RL is neither of these: RL is not supervised learning because the reward signals do not tell the agent which action to take in each state x_i , and thus do not act as a supervision signal y_i . This is because some actions may give lower immediate reward but may lead to states from which the agent can eventually receive higher rewards. RL also differs from unsupervised learning because the rewards, while not a supervised signal, act as a proxy from which to learn an optimal policy.

In the following sections, we will formally define these concepts — sequential decision processes, optimal policies, and learning by interaction — within a framework called the Markov decision process.

2.2 Markov Decision Processes

The standard model used in RL to define the sequential decision process is the *Markov decision process*:

Definition 1 (Markov decision process) A finite Markov decision process (MDP) consists of:

- Finite set of states S , with subset of terminal states $\bar{S} \subset S$
- Finite set of actions A
- Reward function $\mathcal{R} : S \times A \times S \rightarrow \mathbb{R}$
- State transition probability function $\mathcal{T} : S \times A \times S \rightarrow [0, 1]$ such that

$$\forall s \in S, a \in A : \sum_{s' \in S} \mathcal{T}(s, a, s') = 1 \quad (2.1)$$

- Initial state distribution $\mu : S \rightarrow [0, 1]$ such that

$$\sum_{s \in S} \mu(s) = 1 \quad \text{and} \quad \forall s \in \bar{S} : \mu(s) = 0 \quad (2.2)$$

An MDP starts in an initial state $s^0 \in S$, which is sampled from μ . At time t , the agent observes the current state $s^t \in S$ of the MDP and chooses an action $a^t \in A$ with probability given by its policy, $\pi(a^t | s^t)$, which is conditioned on the state. Given the state s^t and action a^t , the MDP transitions into a next state $s^{t+1} \in S$ with probability given by $\mathcal{T}(s^t, a^t, s^{t+1})$, and the agent receives a reward $r^t = \mathcal{R}(s^t, a^t, s^{t+1})$. We also write this probability as $\mathcal{T}(s^{t+1} | s^t, a^t)$ to emphasize that it is conditioned on the state-action pair s^t, a^t . These steps are repeated until the process reaches a terminal state $s^t \in \bar{S}$ or after completing a maximum number of T time steps,² after which the process terminates; or the steps may be repeated for an infinite number of time steps if the MDP is non-terminating. Each independent run of this process is referred to as an *episode*.

A finite MDP can be compactly represented as a finite state machine. Figure 2.3 shows an example MDP, in which a Mars rover has collected some samples and must return to the base station. From the *Start* state, there are two paths that lead to the *Base* target state. The rover can travel down a steep mountain slope (action *right*) that will take it directly to the base station. However, there is a high probability of 0.5 that the rover will fall down the cliff and be destroyed (-10 reward). Or the rover can travel through a longer path (action *left*, then two times action *right*), which passes by two sites (*Site A* and *Site B*)

2. Technically, if we assume that the agent can observe the time step t , then termination after T time steps can be modeled by including t in the information contained in the state s^t and defining the set of terminal states $\bar{S}$ to contain all states s^t with $t \geq T$.

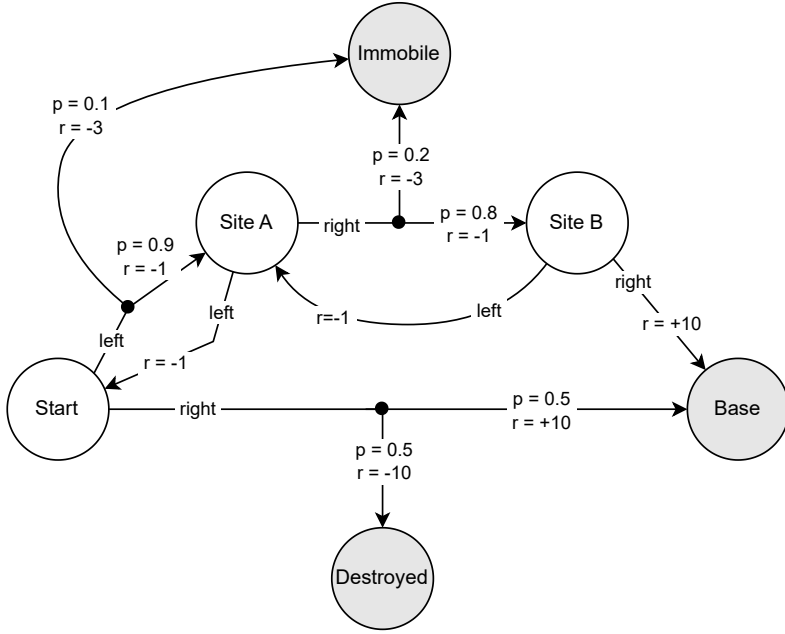


Figure 2.3: Mars Rover MDP. States are shown as circles, where *Start* is the initial state. Each non-terminal state (white) has two possible actions, *right* and *left*, that are shown as directed edges with associated transition probabilities (p) and rewards (r). Gray shaded circles mark terminal states.

before arriving at the base station. However, it takes a day to arrive at each site (-1 reward), and there is a probability of 0.3 that the rover will get stuck on the rocky ground and be immobilized (-3 reward). Arriving at the base station gives a reward of $+10$.

The term “Markov” comes from the *Markov property*, which states that the future state and reward are conditionally independent of past states and actions, given the current state and action:

$$\Pr(s^{t+1}, r^t | s^t, a^t, s^{t-1}, a^{t-1}, \dots, s^0, a^0) = \Pr(s^{t+1}, r^t | s^t, a^t) \quad (2.3)$$

This means that the current state provides sufficient information to choose optimal actions in an MDP — past states and actions are not relevant.

The most common assumption in RL is that the dynamics of the MDP, in particular the transition and reward functions $\mathcal{T}$ and $\mathcal{R}$, are a priori unknown to the agent. Typically, the only parts of the MDP that are assumed to be known are the action space A and the state space S .

While Definition 1 defines MDPs with finite, or *discrete*, states and actions, MDPs can also be defined with continuous states and actions, or a mixture of both discrete and continuous elements. Moreover, while the reward function $\mathcal{R}$ as defined here is deterministic, MDPs can also define a probabilistic reward function such that $\mathcal{R}(s, a, s')$ gives a distribution over possible rewards.

The *multi-armed bandit problem* (or simply “bandit problem”) is an important special case of the MDP in which each episode terminates after one time step (i.e., $T = 1$), there is only a single state in S and no terminal states (i.e., $|S| = 1$ and $\bar{S} = \emptyset$), and the reward function $\mathcal{R}$ is probabilistic and unknown to the agent. Thus, in such a bandit problem, each action produces a reward from some unknown reward distribution, and the goal is effectively to find the action that yields the highest expected reward as quickly as possible. Bandit problems have been used as a basic model to study the exploration-exploitation dilemma (Lattimore and Szepesvári 2020).

An inherent assumption in MDPs is that the agent can fully observe the state of the environment in each time step. In many applications, an agent only observes partial and noisy information about the environment. *Partially observable Markov decision processes* (POMDPs) (Kaelbling, Littman, and Cassandra 1998) generalize MDPs by defining a decision process in which the agent receives observations o^t rather than directly observing the state s^t , and these observations depend in a probabilistic or deterministic way on the state. Thus, in general, the agent will need to take into account the history of past observations $o^0, o^1, \dots, o^t$ in order to infer the possible current states s^t of the environment. POMDPs are a special case of the partially observable stochastic game (POSG) model introduced in Chapter 3; namely, a POMDP is a POSG in which there is only one agent. We refer to Section 3.4 for a more detailed definition and discussion of partial observability in decision processes.

We have defined the MDP as a model of decision processes, but we have not yet defined the learning objective in an MDP. The next section will introduce the most common learning objective used in MDPs: maximizing expected discounted returns. As discussed earlier (Figure 2.1), together, an MDP and learning objective specify an RL problem.

2.3 Expected Discounted Returns and Optimal Policies

Given a policy π that specifies action probabilities in each state, and assuming that each episode in the MDP terminates after T time steps, the total *return* in an episode is the cumulative reward received over time

$$r^0 + r^1 + \dots + r^{T-1}. \quad (2.4)$$

Due to the stochastic nature of the MDP, it may not be possible to maximize this return in all episodes. This is because some actions may lead to different outcomes with certain probabilities, and these outcomes are outside the control of the agent. Therefore, the agent can maximize the *expected return* given by

$$\mathbb{E}_\pi [r^0 + r^1 + \dots + r^{T-1}] \quad (2.5)$$

where the expectation $\mathbb{E}_\pi$ assumes that the initial state is sampled from the initial state distribution (i.e., $s^0 \sim \mu$), that the agent follows policy π to select actions (i.e., $a^t \sim \pi(\cdot | s^t)$), and that successor states are governed by the state transition probabilities (i.e., $s^{t+1} \sim \mathcal{T}(\cdot | s^t, a^t)$).

The above definition of total returns is guaranteed to be finite for a terminating MDP. However, in non-terminating MDPs the total return may be infinite, in which case returns may not be informative to distinguish the performance of different policies that achieve infinite returns. The standard approach to ensuring finite returns in non-terminating MDPs is to use a *discount factor* $\gamma \in [0, 1]$, based on which we define the *discounted return*³

$$\mathbb{E}_\pi [r^0 + \gamma r^1 + \gamma^2 r^2 + \dots] = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r^t \right]. \quad (2.6)$$

For $\gamma < 1$ and assuming that rewards are constrained to lie in a finite range $[r_{\min}, r_{\max}]$, the discounted return is guaranteed to be finite

$$\sum_{t=0}^{\infty} \gamma^t r^t \leq r_{\max} \sum_{t=0}^{\infty} \gamma^t = r_{\max} \frac{1}{1 - \gamma} \quad (2.7)$$

where the right-hand fraction in the above equation is the closed form of the geometric series given by $\sum_{t=0}^{\infty} \gamma^t$.

The discount factor has two equivalent interpretations. One interpretation is that $(1 - \gamma)$ is the probability with which the MDP terminates after each time step. Thus, the probability that the MDP terminates after a total of $T > 0$ time steps is $\gamma^{T-1}(1 - \gamma)$, where γ^{T-1} is the probability of continuing (i.e., not terminating) in the first $T - 1$ time steps, multiplied by the probability $(1 - \gamma)$ of terminating in the following time step. For example, in the Mars Rover MDP shown in Figure 2.3, we can specify a discount factor of $\gamma = 0.95$ to model the fact that the rover's battery may fail with a probability of 0.05 after each state transition. The second interpretation is that the agent gives “weight” γ^t to reward r^t received at time t . Thus, a γ close to 0 leads to a myopic agent that cares more about near-term rewards, while a γ close to 1 leads to a farsighted agent that also values distant rewards. In either case, it is important to note that

3. Note that r^t refers to the reward at time t , while γ^t denotes the exponentiation operation (γ to the power of t). This is a standard notational convention in RL literature.

the discount rate is part of the learning objective and not a tunable algorithm parameter; γ is a fixed parameter specified by the learning objective. In the remainder of this book, whenever we refer to returns, we specifically mean discounted returns with some discount factor γ .

Finally, the above definition of discounted returns can work for both terminating and non-terminating MDPs via the convention of *absorbing states*. If an episode reaches a terminal state or a maximum number of time steps, we define the final state in the episode to be absorbing in that any subsequent actions in this state will transition the MDP into the same state with probability 1 and give a reward of 0 to the agent. Thus, once the MDP reaches an absorbing state, it will forever remain in that state and the agent will not accrue any more rewards.

Based on the above definition of discounted returns and absorbing states, we can now define the solution to the MDP as the *optimal policy* π^* that maximizes the expected discounted return.

2.4 Value Functions and Bellman Equation

Based on the Markov property defined in Equation 2.3, we know that given the current state and action, the future states and rewards are independent of the past states and actions. This implies independence of past rewards, since rewards are a function of the states and actions, $r^t = \mathcal{R}(s^t, a^t, s^{t+1})$. Therefore, in an MDP, the expected return can be defined individually for each state $s \in S$. This gives rise to the concept of *value functions*, which are central to much of RL theory and many algorithms.

First, note that the discounted reward sequence can be written in recursive form as

$$u^t = r^t + \gamma r^{t+1} + \gamma^2 r^{t+2} + \dots \quad (2.8)$$

$$= r^t + \gamma u^{t+1}. \quad (2.9)$$

Given a policy π , the *state-value function* $V^\pi(s)$ gives the “value” of state s under π , which is the expected return when starting in state s and following policy π to select actions, formally

$$V^\pi(s) = \mathbb{E}_\pi[u^t \mid s^t = s] \quad (2.10)$$

$$= \mathbb{E}_\pi[r^t + \gamma u^{t+1} \mid s^t = s] \quad (2.11)$$

$$= \sum_{a \in A} \pi(a \mid s) \sum_{s' \in S} \mathcal{T}(s' \mid s, a) [\mathcal{R}(s, a, s') + \gamma \mathbb{E}_\pi[u^{t+1} \mid s^{t+1} = s']] \quad (2.12)$$

$$= \sum_{a \in A} \pi(a \mid s) \sum_{s' \in S} \mathcal{T}(s' \mid s, a) [\mathcal{R}(s, a, s') + \gamma V^\pi(s')] \quad (2.13)$$

where $V^\pi(s) = 0$ for terminal (i.e., absorbing) states $s \in \bar{S}$. The recursive equation given in Equation 2.13 is called the *Bellman equation* in honor of Richard Bellman and his pioneering work (Bellman 1957).

The Bellman equation for V^π defines a system of m linear equations with m variables, where $m = |S|$ is the number of states in the finite MDP:

$$V^\pi(s_1) = \sum_{a \in A} \pi(a | s_1) \sum_{s' \in S} \mathcal{T}(s' | s_1, a) [\mathcal{R}(s_1, a, s') + \gamma V^\pi(s')] \quad (2.14)$$

$$V^\pi(s_2) = \sum_{a \in A} \pi(a | s_2) \sum_{s' \in S} \mathcal{T}(s' | s_2, a) [\mathcal{R}(s_2, a, s') + \gamma V^\pi(s')] \quad (2.15)$$

$$\vdots$$

$$V^\pi(s_m) = \sum_{a \in A} \pi(a | s_m) \sum_{s' \in S} \mathcal{T}(s' | s_m, a) [\mathcal{R}(s_m, a, s') + \gamma V^\pi(s')] \quad (2.16)$$

where $V^\pi(s_k)$ for $k = 1, \dots, m$ are the variables in the equation system and $\pi(a | s_k)$, $\mathcal{T}(s' | s_k, a)$, $\mathcal{R}(s_k, a, s')$, and γ are constants. This equation system has a unique solution given by the value function V^π for policy π . If all elements of the MDP are known, then one can solve this equation system to obtain V^π using any method to solve linear equation systems (such as Gauss elimination).

Analogous to state-value functions, we can define *action-value functions* $Q^\pi(s, a)$ which give the expected return when selecting action a in state s and then following policy π to select actions subsequently,

$$Q^\pi(s, a) = \mathbb{E}_\pi [u^t | s^t = s, a^t = a] \quad (2.17)$$

$$= \mathbb{E}_\pi [r^t + \gamma u^{t+1} | s^t = s, a^t = a] \quad (2.18)$$

$$= \sum_{s' \in S} \mathcal{T}(s' | s, a) [\mathcal{R}(s, a, s') + \gamma V^\pi(s')] \quad (2.19)$$

$$= \sum_{s' \in S} \mathcal{T}(s' | s, a) [\mathcal{R}(s, a, s') + \gamma \sum_{a' \in A} \pi(a' | s') Q^\pi(s', a')] \quad (2.20)$$

where $Q^\pi(s, a) = 0$ for terminal states $s \in \bar{S}$. For the jump from Equation 2.19 to 2.20, notice in Equation 2.13 that the second sum (over states s') is an application of Equation 2.19, and hence it can be replaced by $Q^\pi(s', a')$. Equation 2.20 admits a system of linear equations that has a unique solution given by Q^π .

A policy π is optimal in the MDP if the policy's (state or action) value function is the *optimal value function* of the MDP, defined as

$$V^*(s) = \max_{\pi'} V^{\pi'}(s), \quad \forall s \in S \quad (2.21)$$

$$Q^*(s, a) = \max_{\pi'} Q^{\pi'}(s, a), \quad \forall s \in S, a \in A \quad (2.22)$$

We use π^* to denote any optimal policy with optimal value function V^* or Q^* .

Because of the Bellman equation, this means that for any optimal policy π^* we have

$$\forall \pi' \forall s : V^*(s) \geq V^{\pi'}(s). \quad (2.23)$$

Thus, maximizing the expected return in an MDP amounts to maximizing the expected return in each possible state $s \in S$.

In fact, we can write the optimal value functions without reference to the policy using the *Bellman optimality equations*:

$$V^*(s) = \max_{a \in A} \sum_{s' \in S} \mathcal{T}(s' | s, a) [\mathcal{R}(s, a, s') + \gamma V^*(s')] \quad (2.24)$$

$$Q^*(s, a) = \sum_{s' \in S} \mathcal{T}(s' | s, a) \left[\mathcal{R}(s, a, s') + \gamma \max_{a' \in A} Q^*(s', a') \right] \quad (2.25)$$

The Bellman optimality equations define a system of m non-linear equations, where m is again the number of states in the finite MDP. The non-linearity is due to the max-operator used in the equations. The unique solution to the system is the optimal value function V^*/Q^* .

Once we know the optimal action-value function Q^* , the optimal policy π^* is simply derived by choosing actions with maximum value (i.e., expected return) in each state,

$$\pi^*(s) = \arg \max_{a \in A} Q^*(s, a) \quad (2.26)$$

where we use the $\arg \max$ -notation in Equation 2.26 as a convenient shorthand to denote the deterministic policy that assigns probability 1 to the $\arg \max$ -action in state s . If multiple actions have the same maximum value under Q^* , then the optimal policy can assign arbitrary probabilities to these actions (the probabilities must sum to 1). Therefore, while the optimal value function of the MDP is always unique, there may be multiple⁴ optimal policies that have the same optimal (unique) value function. However, as shown in Equation 2.26, there always exist deterministic optimal policies in MDPs.

In the next sections, we will present two algorithm families to compute optimal value functions and policies. The first, dynamic programming, uses the Bellman (optimality) equations in an iterative way to estimate value functions, and it requires knowledge of the complete MDP such as the reward function and state transition probabilities. The second, temporal-difference learning, does not require complete knowledge of the MDP and instead uses sampled experiences from interactions with the environment to update value estimates.

4. In fact, if multiple actions have maximum value, then there will be an *infinite* number of optimal policies since the set of possible probability assignments to these actions spans a continuous (infinite) space.

2.5 Dynamic Programming

Dynamic programming (DP)⁵ is a family of algorithms to compute value functions and optimal policies in MDPs (Bellman 1957; Howard 1960). DP algorithms use the Bellman equations as operators to iteratively estimate the value function and optimal policy. Thus, DP algorithms require complete knowledge of the MDP model, including the reward function $\mathcal{R}$ and state transition probabilities $\mathcal{T}$. While DP algorithms do not “interact” with the environment as per our definition in Section 2.1, the basic concepts underlying DP are an important building block in RL theory, including temporal-difference learning presented in Section 2.6.

The basic DP approach is *policy iteration*, which alternates between two tasks:

- **Policy evaluation:** compute value function V^π for current policy π
- **Policy improvement:** improve current policy π with respect to V^π

Thus, policy iteration produces a sequence of policy and value function estimates, starting with some initial policy π^0 (e.g., uniform-random policy) and value function V^0 (e.g., zero vector):

$$\pi^0 \rightarrow V^{\pi^0} \rightarrow \pi^1 \rightarrow V^{\pi^1} \rightarrow \pi^2 \rightarrow \dots \rightarrow V^* \rightarrow \pi^* \quad (2.27)$$

As we will show later in this section, this sequence converges to the optimal value function, V^* , and optimal policy, π^* , under *greedy policy improvements*.

We first consider the policy evaluation task. Recall that the Bellman equation for V^π defines a system of linear equations, which can be solved to obtain V^π . However, using Gauss elimination (the de-facto standard solution approach for linear equation systems) has time complexity $O(m^3)$, where m is the number of states of the MDP. A number of alternative methods for policy evaluation have been developed in the DP literature, which often operate in iterative ways to produce successive approximations of value functions (see Puterman (2014) and Sutton and Barto (2018) for overviews). *Iterative policy evaluation* is one such method that iteratively applies the Bellman equation for V^π to produce successive estimates of V^π . The algorithm first initializes a vector $V^0(s) = 0$ for all $s \in \mathcal{S}$. It then repeatedly performs update sweeps for all states $s \in \mathcal{S}$:

$$V^{k+1}(s) \leftarrow \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} \mathcal{T}(s'|s, a) [\mathcal{R}(s, a, s') + \gamma V^k(s')] \quad (2.28)$$

5. The word “programming” in DP refers to a mathematical optimization problem, analogous to how the term is used in linear programming and non-linear programming. Regarding the adjective “dynamic,” Bellman (1957) states in his preface that it “indicates that we are interested in processes in which time plays a significant role, and in which the order of operations may be crucial.”

This sequence $V^0, V^1, V^2, \dots$ converges to the value function V^π . Hence, in practice, we may stop the updates once there are no more changes between V^k and V^{k+1} after performing an update sweep. Notice that Equation 2.28 updates the value estimate for a state s using value estimates for other states s' . This property is called *bootstrapping* and is a core property of many RL algorithms.

As an example, we apply iterative policy evaluation in the Mars Rover problem from Section 2.2 with $\gamma = 0.95$. First, consider a policy π which in the initial state *Start* chooses action *right* with probability 1. (We ignore the other states since they will never be visited under this policy.) After we run iterative policy evaluation to convergence, we obtain a value of $V^\pi(\text{Start}) = 0$. From the MDP specification in Figure 2.3, it can be easily verified that this value is correct, since $V^\pi(\text{Start}) = -10 \cdot 0.5 + 10 \cdot 0.5 = 0$. Now, consider a policy π , which chooses both actions with equal probability 0.5 in the initial state and action *right* with probability 1 in states *Site A* and *Site B*. In this case, the converged values are $V^\pi(\text{Start}) = 2.05$, $V^\pi(\text{Site A}) = 6.2$, and $V^\pi(\text{Site B}) = 10$. For both policies, the values for the terminal states are all 0, since these are absorbing states as defined in Section 2.3.

To see why iterative policy evaluation converges to the value function V^π , one can show that the Bellman operator defined in Equation 2.28 is a *contraction mapping*. A mapping $f: \mathcal{X} \rightarrow \mathcal{X}$ on a $\|\cdot\|$ -normed complete vector space $\mathcal{X}$ is a γ -contraction, for $\gamma \in [0, 1)$, if for all $x, y \in \mathcal{X}$:

$$\|f(x) - f(y)\| \leq \gamma \|x - y\| \quad (2.29)$$

By the Banach fixed-point theorem, if f is a contraction mapping, then for any initial vector $x \in \mathcal{X}$ the sequence $f(x), f(f(x)), f(f(f(x))), \dots$ converges to a unique fixed point $x^* \in \mathcal{X}$ such that $f(x^*) = x^*$. Using the max-norm $\|x\|_\infty = \max_i |x_i|$, it can be shown that the Bellman equation indeed satisfies Equation 2.29. First, rewrite the Bellman equation as follows:

$$\begin{aligned} V^\pi(s) &= \sum_{a \in A} \pi(a|s) \sum_{s' \in S} \mathcal{T}(s'|s, a) [\mathcal{R}(s, a, s') + \gamma V^\pi(s')] \\ &= \sum_{a \in A} \sum_{s' \in S} \pi(a|s) \mathcal{T}(s'|s, a) \mathcal{R}(s, a, s') + \sum_{a \in A} \sum_{s' \in S} \pi(a|s) \mathcal{T}(s'|s, a) \gamma V^\pi(s') \end{aligned} \quad (2.30)$$

$$(2.31)$$

This can be written as an operator $f^\pi(v)$ over a value vector $v \in \mathbb{R}^{|S|}$

$$f^\pi(v) = r^\pi + \gamma M^\pi v \quad (2.32)$$

where $r^\pi \in \mathbb{R}^{|S|}$ is a vector with elements

$$r_s^\pi = \sum_{a \in A} \sum_{s' \in S} \pi(a|s) \mathcal{T}(s'|s, a) \mathcal{R}(s, a, s') \quad (2.33)$$

and $M^\pi \in \mathbb{R}^{|S| \times |S|}$ is a matrix with elements

$$M_{s,s'}^\pi = \sum_{a \in A} \pi(a|s) \mathcal{T}(s'|s, a). \quad (2.34)$$

Then, we have for any two value vectors v, u :

$$\|f^\pi(v) - f^\pi(u)\|_\infty = \|(r^\pi + \gamma M^\pi v) - (r^\pi + \gamma M^\pi u)\|_\infty \quad (2.35)$$

$$= \gamma \|M^\pi(v - u)\|_\infty \quad (2.36)$$

$$\leq \gamma \|v - u\|_\infty. \quad (2.37)$$

Therefore, the Bellman operator is a γ -contraction under the max-norm, and repeated application of the Bellman operator converges to a unique fixed point, which is V^π . (The inequality in (2.37) holds because for each $s \in S$ we have $\sum_{s'} M_{s,s'}^\pi = 1$ and, therefore, $\|M^\pi x\|_\infty \leq \|x\|_\infty$ for any value vector x .)

Now that we have a method for policy evaluation, we next consider the policy improvement task in policy iteration. Once we have computed the value function V^π , the policy improvement task modifies the policy π by making it *greedy* with respect to V^π for all $s \in S$:

$$\pi' = \arg \max_{a \in A} \mathcal{T}(s'|s, a) [\mathcal{R}(s, a, s') + \gamma V^\pi(s')] \quad (2.38)$$

$$= \arg \max_{a \in A} Q^\pi(s, a) \quad (2.39)$$

By the *policy improvement theorem* (Sutton and Barto 2018), we know that if for all $s \in S$

$$\sum_{a \in A} \pi'(a|s) Q^\pi(s, a) \geq \sum_{a \in A} \pi(a|s) Q^\pi(s, a) \quad (2.40)$$

$$= V^\pi(s) \quad (2.41)$$

then π' must be as good as or better than π :

$$\forall s: V^{\pi'}(s) \geq V^\pi(s). \quad (2.42)$$

If after the policy improvement task, the greedy policy π' did not change from π , then we know that $V^{\pi'} = V^\pi$ and it follows for all $s \in S$:

$$V^{\pi'}(s) = \max_{a \in A} \mathbb{E}_\pi [r^t + \gamma V^\pi(s^{t+1}) | s^t = s, a^t = a] \quad (2.43)$$

$$= \max_{a \in A} \mathbb{E}_{\pi'} [r^t + \gamma V^{\pi'}(s^{t+1}) | s^t = s, a^t = a] \quad (2.44)$$

$$= \max_{a \in A} \sum_{s' \in S} \mathcal{T}(s'|s, a) [\mathcal{R}(s, a, s') + \gamma V^{\pi'}(s')] \quad (2.45)$$

$$= V^*(s) \quad (2.46)$$

Algorithm 1 Value iteration for MDPs

1: Initialize: $V(s) = 0$ for all $s \in S$

2: Repeat until V converged:

$$\forall s \in S: V(s) \leftarrow \max_{a \in A} \sum_{s' \in S} \mathcal{T}(s' | s, a) [\mathcal{R}(s, a, s') + \gamma V(s')] \quad (2.48)$$

3: Return optimal policy π^* with

$$\forall s \in S: \pi^*(s) \leftarrow \arg \max_{a \in A} \sum_{s' \in S} \mathcal{T}(s' | s, a) [\mathcal{R}(s, a, s') + \gamma V(s')] \quad (2.49)$$

Therefore, if π' did not change from π after policy improvement, we know that π' must be the optimal policy of the MDP, and policy iteration is complete.

The above version of policy iteration, which uses iterative policy evaluation, makes multiple complete update sweeps over the entire state space S . *Value iteration* is a DP algorithm that combines one sweep of iterative policy evaluation and policy improvement in a single update equation by using the Bellman optimality equation as an update operator:

$$V^{k+1}(s) \leftarrow \max_{a \in A} \sum_{s' \in S} \mathcal{T}(s' | s, a) [\mathcal{R}(s, a, s') + \gamma V^k(s')], \quad \forall s \in S \quad (2.47)$$

Following a similar argument as for the Bellman operator, it can be shown that the Bellman optimality operator defined in Equation 2.47 is a γ -contraction mapping. Thus, repeated application of value iteration converges to the unique fixed point that is the optimal value function V^* . Algorithm 1 provides the complete pseudocode for the value iteration algorithm.

Returning to the Mars Rover example, if we run value iteration to convergence, we obtain the optimal state values $V^*(Start) = 4.1$, $V^*(Site A) = 6.2$, and $V^*(Site B) = 10$. The optimal policy π^* chooses action *left* with probability 1 in the initial state and action *right* with probability 1 in states *Site A* and *Site B*.

2.6 Temporal-Difference Learning

Temporal-difference learning (TD) is a family of RL algorithms that learn value functions and optimal policies based on experiences from interactions with the environment. The experiences are generated by following the agent-environment interaction loop shown in Figure 2.2: in state s^t , sample an action $a^t \sim \pi(\cdot | s^t)$ from policy π , then observe reward $r^t = \mathcal{R}(s^t, a^t, s^{t+1})$ and the new state $s^{t+1} \sim \mathcal{T}(\cdot | s^t, a^t)$. Like DP algorithms, TD algorithms learn value functions based on the Bellman equations and bootstrapping — estimating the value of a

state or action using value estimates of other states/actions. However, unlike DP algorithms, TD algorithms do not require complete knowledge of the MDP, such as the reward function and state transition probabilities. Instead, TD algorithms perform the policy evaluation and policy improvement tasks based solely on the experiences collected while interacting with the environment.

TD algorithms use the following general update rule to learn action-value functions:

$$Q(s^t, a^t) \leftarrow Q(s^t, a^t) + \alpha [\mathcal{X} - Q(s^t, a^t)] \quad (2.50)$$

where $\mathcal{X}$ is the *update target*, and $\alpha \in (0, 1]$ is the *learning rate* (or step size). The update target $\mathcal{X}$ is constructed based on experience samples (s^t, a^t, r^t, s^{t+1}) collected from interactions with the environment. A wide range of options exist to specify the update target, and here we present two basic variants.

Recall the Bellman equation for the action-value function Q^π for policy π :

$$Q^\pi(s, a) = \sum_{s' \in S} \mathcal{T}(s' | s, a) \left[\mathcal{R}(s, a, s') + \gamma \sum_{a' \in A} \pi(a' | s') Q^\pi(s', a') \right] \quad (2.51)$$

Sarsa (Sutton and Barto 2018) is a TD algorithm that constructs an update target based on Equation 2.51 by replacing the summations over successor states s' and actions a' as well as the reward function $\mathcal{R}(s, a, s')$ with their corresponding elements from the experience tuple $(s^t, a^t, r^t, s^{t+1}, a^{t+1})$ (hence the name Sarsa):

$$\mathcal{X} = r^t + \gamma Q(s^{t+1}, a^{t+1}) \quad (2.52)$$

Here, the action a^{t+1} is sampled from the policy π in the successor state s^{t+1} , $a^{t+1} \sim \pi(\cdot | s^{t+1})$. The complete Sarsa update rule is specified as

$$Q(s^t, a^t) \leftarrow Q(s^t, a^t) + \alpha [r^t + \gamma Q(s^{t+1}, a^{t+1}) - Q(s^t, a^t)]. \quad (2.53)$$

If π stays fixed, then it can be shown under certain conditions that Sarsa will learn the action-value function $Q = Q^\pi$. The first condition is that all state-action combinations $(s, a) \in S \times A$ must be tried an infinite number of times during learning. The second condition is given by the “standard stochastic approximation conditions,” which state that the learning rate α must be reduced over time in a way that satisfies the following conditions:

$$\forall s \in S, a \in A: \sum_{k=1}^{\infty} \alpha_k(s, a) \rightarrow \infty \quad \text{and} \quad \sum_{k=1}^{\infty} \alpha_k(s, a)^2 < \infty \quad (2.54)$$

where $\alpha_k(s, a)$ denotes the learning rate used when applying the update rule in Equation 2.53 after the k th selection of action a in state s . The left-hand sum in Equation 2.54 ensures that the learning rate is large enough to overcome initial learning conditions, while the right-hand sum ensures that the sequence will converge at a certain rate. Therefore, $\alpha_k(s, a) = \frac{1}{k}$ satisfies these conditions on the

Algorithm 2 Sarsa for MDPs (with ϵ -greedy policies)

- 1: Initialize: $Q(s, a) = 0$ for all $s \in S, a \in A$
 - 2: Repeat for every episode:
 - 3: Observe initial state s^0
 - 4: With probability ϵ : choose random action $a^0 \in A$
 - 5: Otherwise: choose action $a^0 \in \arg \max_a Q(s^0, a)$
 - 6: **for** $t = 0, 1, 2, \dots$ **do**
 - 7: Apply action a^t , observe reward r^t and next state s^{t+1}
 - 8: With probability ϵ : choose random action $a^{t+1} \in A$
 - 9: Otherwise: choose action $a^{t+1} \in \arg \max_a Q(s^{t+1}, a)$
 - 10: $Q(s^t, a^t) \leftarrow Q(s^t, a^t) + \alpha [r^t + \gamma Q(s^{t+1}, a^{t+1}) - Q(s^t, a^t)]$
-

learning rate, while a constant learning rate $\alpha_k(s, a) = c$ does not. Nonetheless, in practice it is common to use a constant learning rate since learning rates that meet the above conditions, while theoretically sound, can lead to slow learning.

In order for Sarsa to learn the optimal value function Q^* and optimal policy π^* , it must gradually modify the policy π in a way that brings it closer to the optimal policy. To achieve this, π can be made greedy with respect to the value estimates Q , similarly to the DP policy improvement task defined in Equation 2.39. However, making π fully greedy and deterministic would violate the first condition of trying all state-action combinations infinitely often. Thus, a common technique in TD algorithms is to instead use an ϵ -greedy policy that uses a parameter $\epsilon \in [0, 1]$, defined as⁶

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|A|} & \text{if } a \in \arg \max_{a' \in A} Q(s, a') \\ \frac{\epsilon}{|A|} & \text{otherwise} \end{cases} \quad (2.55)$$

Thus, the ϵ -greedy policy chooses the greedy action with probability $1 - \epsilon$, and with probability ϵ chooses a random other action. In this way, if $\epsilon > 0$, the requirement to try all state-action combinations an infinite number of times is ensured. Now, in order to learn the optimal policy π^* , we can gradually reduce the value of ϵ to 0 during learning, such that π will gradually converge to π^* . Pseudocode for Sarsa using ϵ -greedy policies is given in Algorithm 2.

6. Equation 2.55 assumes that there is a single greedy action with maximum value under Q . If multiple actions with maximum value exist for a given state, then the definition can be modified to distribute the probability mass $(1 - \epsilon)$ among these actions.

Algorithm 3 Q-learning for MDPs (with ϵ -greedy policies)

-
- 1: Initialize: $Q(s, a) = 0$ for all $s \in S, a \in A$
 - 2: Repeat for every episode:
 - 3: **for** $t = 0, 1, 2, \dots$ **do**
 - 4: Observe current state s^t
 - 5: With probability ϵ : choose random action $a^t \in A$
 - 6: Otherwise: choose action $a^t \in \arg \max_a Q(s^t, a)$
 - 7: Apply action a^t , observe reward r^t and next state s^{t+1}
 - 8: $Q(s^t, a^t) \leftarrow Q(s^t, a^t) + \alpha [r^t + \gamma \max_{a'} Q(s^{t+1}, a') - Q(s^t, a^t)]$
-

While Sarsa is based on the Bellman equation for Q^π , one can also construct a TD update target based on the Bellman optimality equation for Q^* ,

$$Q^*(s, a) = \sum_{s' \in S} \mathcal{T}(s' | s, a) \left[\mathcal{R}(s, a, s') + \gamma \max_{a' \in A} Q^*(s', a') \right]. \quad (2.56)$$

Q-learning (Watkins and Dayan 1992) is a TD algorithm that constructs an update target based on the above Bellman optimality equation, again by replacing the summation over states s' and the reward function $\mathcal{R}(s, a, s')$ with corresponding elements from the experience tuple (s^t, a^t, r^t, s^{t+1}) :

$$\mathcal{X} = r^t + \gamma \max_{a' \in A} Q(s^{t+1}, a') \quad (2.57)$$

The complete Q-learning update rule is specified as

$$Q(s^t, a^t) \leftarrow Q(s^t, a^t) + \alpha \left[r^t + \gamma \max_{a' \in A} Q(s^{t+1}, a') - Q(s^t, a^t) \right]. \quad (2.58)$$

Pseudocode for Q-learning using ϵ -greedy policies is given in Algorithm 3.

Q-learning is guaranteed to converge to the optimal policy π^* under the same conditions as Sarsa (i.e., trying all state-action pairs infinitely often, and standard stochastic approximation conditions in Equation 2.54). However, in contrast to Sarsa, Q-learning does not require that the policy π used to interact with the environment be gradually made closer to the optimal policy π^* . Instead, Q-learning may use *any* policy to interact with the environment, so long as the convergence conditions are upheld. For this reason, Q-learning is called an *off-policy* TD algorithm while Sarsa is an *on-policy* TD algorithm, and this distinction has a number of implications which we will discuss in more detail in Chapter 8. In Section 2.7, we will use learning curves to evaluate and compare the performance of Q-learning and Sarsa on an RL problem.

2.7 Evaluation with Learning Curves

The standard approach to evaluate the performance of an RL algorithm on a learning problem is via *learning curves*. A learning curve shows the performance of the learned policy over increasing training time, where performance can be measured in terms of the learning objective (e.g., discounted returns) as well as other secondary metrics.

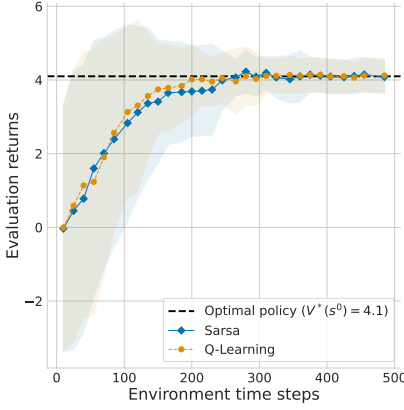
Figure 2.4 shows various learning curves for the Sarsa and Q-learning algorithms applied to the Mars Rover problem from Section 2.2 with discount factor $\gamma = 0.95$. In these figures, the x-axis shows the environment time steps across episodes, and the y-axis shows the average discounted *evaluation returns* achieved from the initial state, that is, $V^\pi(s^0)$.⁷

The term “evaluation return” means that the shown returns are for the greedy policy⁸ with respect to the learned action values after T learning time steps. Thus, the plots answer the question: If we finish learning after T time steps and extract the greedy policy, what expected returns can we expect to achieve with this policy? The results shown here are averaged over one hundred independent training runs, each using a different random seed to determine the outcomes of random events (e.g., probabilistic action selections and state transitions). Specifically, each point on a line is produced by taking the greedy policy from each training run at that time, running one hundred independent episodes with the policy and averaging over the resulting returns, and finally averaging over the average returns corresponding to the one hundred training runs. The shaded area shows the standard deviation over the averaged returns from each training run. In Figure 2.4(b), we show the average episode lengths of the greedy policy instead of the average returns.

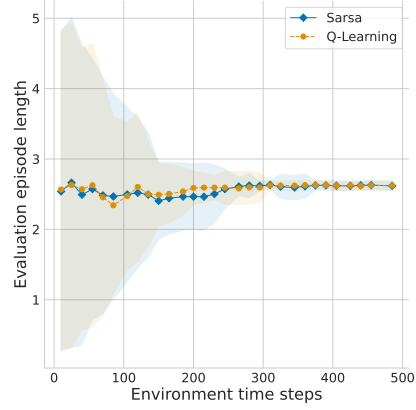
In this example, Sarsa and Q-learning both learn the optimal policy π^* which chooses action *left* in state *Start* and action *right* in states *Site A* and *Site B*. Moreover, the two algorithms basically have the same learning curves for the evaluation returns. Figures 2.4(c) and 2.4(d) (for Q-learning) show that the choice of the learning rate α and exploration rate ϵ can have an important impact on the learning. In this case, the “average” learning rate $\alpha_k(s, a) = \frac{1}{k}$ performs best for Q-learning, but for more complex MDPs with larger state and action sets, this choice of learning rate usually leads to much slower learning compared to appropriately chosen constant learning rates.

7. Recall that we use the term “return” as a shorthand for discounted return when the learning objective is to maximize discounted returns.

8. In policy gradient RL algorithms, discussed in Chapter 8, the algorithm learns a probabilistic policy for which there usually is no “greedy” version. In this case, the evaluation return just uses the current policy without modification.



(a) Average evaluation returns for Sarsa and Q-learning



(b) Average episode length for Sarsa and Q-learning

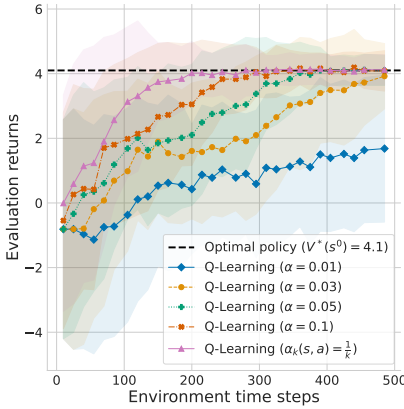
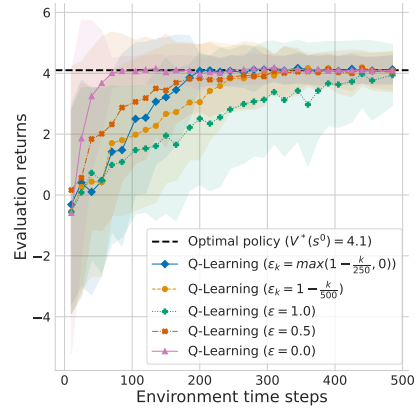
(c) Q-learning with different learning rates α (ϵ linearly decayed from 1 to 0)(d) Q-learning with different exploration rates ϵ ($\alpha = 0.1$)

Figure 2.4: Results when applying the Sarsa and Q-learning algorithms to the Mars Rover problem (Figure 2.3, page 23) with $\gamma = 0.95$, averaged over one hundred independent training runs. The shaded area shows the standard deviation over the averaged discounted returns from each training run. The dashed horizontal line marks the optimal value of the initial state ($V^*(s^0)$), computed via value iteration. In Figures 2.4(a) and 2.4(b), the algorithms use a scheduled learning rate of $\alpha_k(s, a) = \frac{1}{k}$ where k is the number of times state s has been visited and action a was applied. The exploration rate is linearly decayed from $\epsilon = 1$ to $\epsilon = 0$ over time steps $t = 0, \dots, 500$ (i.e., $\epsilon^t = 1 - t \frac{1}{500}$).

Note that the x-axis of the plots show the cumulative training time steps *across* episodes. This is not to be confused with the time steps t *within* episodes. The reason that we show cumulative time steps instead of the number of completed episodes on the x-axis is that the latter can potentially skew the comparison of algorithms. For instance, suppose we show training curves for algorithms A and B with number of episodes on the x-axis. Suppose both algorithms eventually reach the same performance with their learned policies, but algorithm A learns much faster than algorithm B (i.e., the curve for A rises more quickly). In this case, it could be that in the early episodes, algorithm A explored many more actions (i.e., time steps) than algorithm B before finishing the episode, which results in a larger number of collected experiences (or training data) for algorithm A. Thus, although both algorithms completed the same number of episodes, algorithm A will have done more learning updates (assuming updates are done after each time step, as in the TD methods presented in this chapter), which would explain the seemingly faster growth in the learning curve. If we instead showed the learning curve for both algorithms with cumulative time steps across episodes, then the learning curve for A might not grow faster than the curve for B.

Recall that at the beginning of this chapter (Figure 2.1, page 20), we defined an RL problem as the combination of a decision process model (e.g., MDP) and a learning objective for the policy (e.g., maximizing the expected discounted return in each state for a specific discount factor). When evaluating a learned policy, we want to evaluate its ability to achieve this learning objective. For example, our Mars Rover problem specifies a discounted return objective with the discount factor $\gamma = 0.95$. Therefore, our learning curves in Figure 2.4(a) show exactly this objective on the y-axis.

However, in some cases, it can be useful to show *undiscounted returns* (i.e., $\gamma = 1$) for a learned policy, even if the RL problem specified a discounted return objective with a specific discount factor $\gamma < 1$. The reason is that undiscounted returns can sometimes be easier to interpret than discounted returns. For example, suppose we want to learn an optimal policy for a video game in which the agent controls a space ship and receives a +1 score (reward) for each destroyed enemy. If in an episode the agent destroys ten enemies at various points in time, the undiscounted return will be ten but the discounted return will be less than ten, making it more difficult to understand how many enemies the agent destroyed. More generally, when analyzing a learned policy, it is often useful to show additional metrics besides the returns, such as win rates in competitive games, or episode lengths as we did in Figure 2.4(b).

When showing undiscounted returns and additional metrics such as episode lengths, it is important to keep in mind that the evaluated policy was not actually

trained to maximize these objectives — it was trained to maximize the expected discounted return for a specific discount factor. In general, two RL problems that differ only in their discount factor but are otherwise identical (i.e., same MDP) may have different optimal policies. In our Mars Rover example, a discounted return objective with discount factor $\gamma=0.95$ leads to an optimal policy that chooses action *left* in the initial state, achieving a value of $V^*(Start)=4.1$. In contrast, a discount factor of $\gamma=0.5$ leads to an optimal policy that chooses action *right* in the initial state, achieving a value of $V^*(Start)=0$. Both policies are *optimal*, but they are for two different learning problems that use different discount factors. Both learning problems are valid, and the choice of γ is part of designing the learning problem.

2.8 Equivalence of $\mathcal{R}(s, a, s')$ and $\mathcal{R}(s, a)$

Our definition of MDPs uses the most general definition of reward functions, $\mathcal{R}(s^t, a^t, s^{t+1})$, by conditioning on the current state s^t and action a^t as well as the successor state s^{t+1} . Another definition for reward functions commonly used in the RL literature is to condition only on s^t and a^t , that is, $\mathcal{R}(s^t, a^t)$. This may be a point of confusion for newcomers to RL. However, these two definitions are in fact equivalent, in that for any MDP using $\mathcal{R}(s^t, a^t, s^{t+1})$ we can construct an MDP using $\mathcal{R}(s^t, a^t)$ (all other components are identical to the original MDP) such that a given policy π produces the same expected returns in both MDPs.

Recall the Bellman equation for the state-value function V^π that we have been using so far for an MDP with reward function $\mathcal{R}(s, a, s')$:

$$V^\pi(s) = \sum_{a \in A} \pi(a|s) \sum_{s' \in S} \mathcal{T}(s'|s, a) [\mathcal{R}(s, a, s') + \gamma V^\pi(s')] \quad (2.59)$$

Assuming that $\mathcal{R}(s, a, s')$ depends only on s, a , we can rewrite the above equation as

$$V^\pi(s) = \sum_{a \in A} \pi(a|s) \sum_{s' \in S} \mathcal{T}(s'|s, a) [\mathcal{R}(s, a) + \gamma V^\pi(s')] \quad (2.60)$$

$$= \sum_{a \in A} \pi(a|s) \left[\sum_{s' \in S} \mathcal{T}(s'|s, a) \mathcal{R}(s, a) + \sum_{s' \in S} \mathcal{T}(s'|s, a) \gamma V^\pi(s') \right] \quad (2.61)$$

$$= \sum_{a \in A} \pi(a|s) \left[\mathcal{R}(s, a) + \gamma \sum_{s' \in S} \mathcal{T}(s'|s, a) V^\pi(s') \right]. \quad (2.62)$$

This is the simplified Bellman equation for MDPs using a reward function $\mathcal{R}(s, a)$. Going the reverse direction from $\mathcal{R}(s, a)$ to $\mathcal{R}(s, a, s')$, we know that by

defining $\mathcal{R}(s, a)$ to be the expected reward under the state transition probabilities

$$\mathcal{R}(s, a) = \sum_{s' \in \mathcal{S}} \mathcal{T}(s' | s, a) \mathcal{R}(s, a, s') \quad (2.63)$$

and plugging this into Equation 2.62, we recover the original Bellman equation that uses $\mathcal{R}(s, a, s')$, given in Equation 2.59. Analogous transformations can be done for the Bellman equation of the action-value function Q^π , as well as the Bellman optimality equations for V^*/Q^* . Therefore, one may use either $\mathcal{R}(s, a, s')$ or $\mathcal{R}(s, a)$.

In this book, we use $\mathcal{R}(s, a, s')$ for the following pedagogical reasons:

1. Using $\mathcal{R}(s, a, s')$ can be more convenient when specifying examples of MDPs, since it is useful to show the differences in reward for multiple possible outcomes of an action. In our Mars Rover MDP shown in Figure 2.3, when using $\mathcal{R}(s, a)$ we would have to show the expected reward (Equation 2.63) on the transition arrow for action *right* from the initial state *Start* (rather than showing the different rewards for the two possible outcomes), which would be less intuitive to read.
2. The Bellman equations when using $\mathcal{R}(s, a, s')$ show a helpful visual congruence with the update targets used in TD algorithms. For example, the Q-learning update target, $r^t + \gamma \max_{a'} Q(s^{t+1}, a')$, appears in a corresponding form in the Bellman optimality equation (in the [] brackets):

$$Q^*(s, a) = \sum_{s' \in \mathcal{S}} \mathcal{T}(s' | s, a) \left[\mathcal{R}(s, a, s') + \gamma \max_{a' \in \mathcal{A}} Q^*(s', a') \right] \quad (2.64)$$

And similarly for the Sarsa update target and the Bellman equation for Q^π (see Section 2.6).

Finally, we mention that most of the original MARL literature presented in Chapter 6 in fact defined reward functions as $\mathcal{R}(s, a)$. To stay consistent in our notation, we will continue to use $\mathcal{R}(s, a, s')$ in the remainder of this book, noting that equivalent transformations from $\mathcal{R}(s, a)$ to $\mathcal{R}(s, a, s')$ always exist.

2.9 Summary

This chapter has provided a concise introduction to the basic concepts of single-agent RL. The most important concepts are the following:

- The *Markov decision process* (MDP) is the standard model to define the environment in which the agent chooses actions over a number of time steps. It defines the possible states of the environment, the actions available to the agent, how the environment state changes in response to the different actions,

and the rewards received by the agent. The agent uses a policy that assigns probabilities to the available actions in each state.

- A learning problem in RL is defined by the combination of a decision process model (e.g., MDP) and a learning objective. The most common learning objective used in single-agent RL is to find a decision policy for the agent that maximizes the *expected discounted return*, defined as the sum of rewards received over time and weighted by a discount factor.
- The *Markov property* in MDPs means that the future states and rewards are independent of past states and actions, given the current state and action. This property allows us to define recursive value functions for policies, known as *Bellman equations*, that return the “value” of a policy in each possible state of the environment. The value is the expected return when starting in the given state and following the policy to select actions. Value functions can also be defined for state-action pairs, where a given action is first chosen and subsequent actions are chosen by the policy.
- *Dynamic programming* (DP) and *temporal-difference learning* (TD) are two families of algorithms that can learn optimal policies in MDPs. DP algorithms require complete knowledge of the MDP specification and use procedures such as value iteration to learn optimal value functions. TD algorithms build on DP theory but do not need complete knowledge of the MDP, instead learning optimal policies by repeatedly exploring actions in the environment and observing the outcomes.
- RL algorithms are evaluated via *learning curves* that show improvement in expected returns over an increasing number of agent-environment interactions during learning.

The following chapters in this part of the book will build on the above concepts in multiple ways. First, Chapter 3 will extend the MDP model by introducing game models in which multiple agents interact in a shared environment. Chapter 4 will use the notion of discounted returns to define a range of solution concepts for multi-agent games. Finally, Chapters 5 and 6 will introduce several families of MARL algorithms that build on and extend the DP and TD algorithms introduced in this chapter.

3 Games: Models of Multi-Agent Interaction

Chapter 1 introduced the general idea of agents that interact in a shared environment to achieve specified goals. In this chapter, we will define this idea formally via models of multi-agent interaction. These models are rooted in game theory, and are hence called *games*. The game models we will introduce define a hierarchy of increasingly complex models, shown in Figure 3.1.

The most basic model is the normal-form game, in which there are multiple agents but there is no evolving environment state. Further up the hierarchy are stochastic games, which define environment states that change over time as a result of the agents' actions and probabilistic state transitions. At the top of the game hierarchy are partially observable stochastic games, in which agents do not directly observe the full environment but instead observe incomplete and/or noisy information about the environment. Games may also use different assumptions about what the agents know about the game. We will introduce each of the game models in turn and provide examples.¹

Note that this chapter focuses on defining models of multi-agent interaction, but does not define what it means to *solve* the games — that is, what it means for agents to act optimally. There are many possible solution concepts which define optimal policies for agents in games. We will introduce a range of solution concepts for games in Chapter 4.

1. There exists another type of game model, not covered in this chapter, called “extensive-form game.” The primary distinction is that in extensive-form games, agents choose actions in turns (one after another) while in the games we introduce, agents choose actions simultaneously (at the same time). We focus on simultaneous-move games since most MARL research uses this type of game model, and they are a more natural extension of the MDP-type models used in RL. Transformations between extensive-form games and (sequential) simultaneous-move games are possible (Shoham and Leyton-Brown 2008).

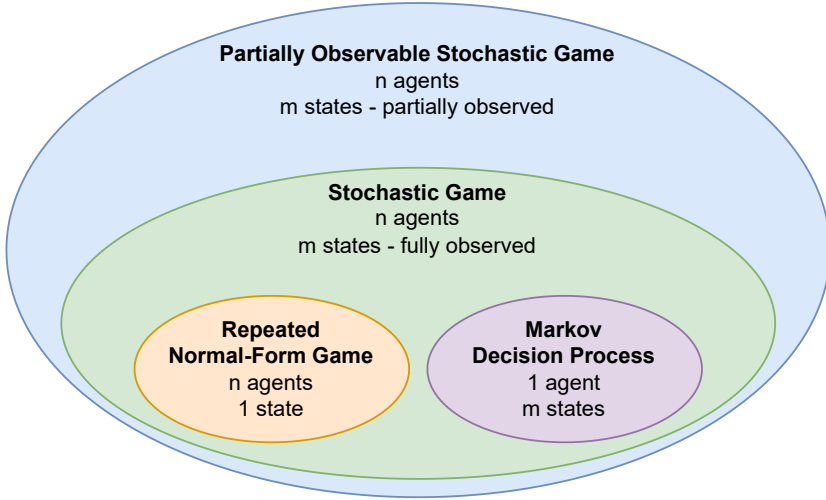


Figure 3.1: Hierarchy of game models used in this book. Partially observable stochastic games (POSGs) include stochastic games as a special case in which the states and agents’ chosen actions are fully observable by all agents. Stochastic games include repeated normal-form games as a special case in which there is only a single environment state, and they include Markov decision processes (MDPs) as a special case in which there is only a single agent.

3.1 Normal-Form Games

A *normal-form game* (also known as “strategic-form” game) defines a single interaction between two or more agents. Similar to how the multi-armed bandit problem (Section 2.2) can be seen as the basic kernel of the MDP, the normal-form game can be seen as the basic building block of all game models introduced in this chapter.

Definition 2 (Normal-form game) A *normal-form game* consists of:

- Finite set of agents $I = \{1, \dots, n\}$
- For each agent $i \in I$:
 - Finite set of actions A_i
 - Reward function $\mathcal{R}_i : A \rightarrow \mathbb{R}$, where $A = A_1 \times \dots \times A_n$

A normal-form game proceeds as follows: First, each agent $i \in I$ selects a policy $\pi_i : A_i \rightarrow [0, 1]$, which assigns probabilities to the actions available to the agent, so that $\sum_{a_i \in A_i} \pi_i(a_i) = 1$. Each agent then samples an action $a_i \in A_i$ with

probability $\pi_i(a_i)$ given by its policy. The resulting actions of all agents form a *joint action*, $a = (a_1, \dots, a_n)$. Finally, each agent i receives a reward based on its reward function and the joint action, $r_i = \mathcal{R}_i(a)$.

Normal-form games can be classified based on the relationship between the reward functions of agents:

- In a *zero-sum* game, the sum of the agents' rewards is always 0, i.e., $\sum_{i \in I} \mathcal{R}_i(a) = 0$ for all $a \in A$.² In zero-sum games with two agents, i and j ,³ one agent's reward function is simply the negative of the other agent's reward function, i.e., $\mathcal{R}_i = -\mathcal{R}_j$.
- In a *common-reward* game, all agents receive the same reward, i.e., $\mathcal{R}_i = \mathcal{R}_j$ for all $i, j \in I$.
- In a *general-sum* game, there are no restrictions on the relationship of reward functions.

Normal-form games with two agents are also referred to as *matrix games* because, in this case, the reward function can be compactly represented as a matrix.⁴ Figure 3.2 shows three example matrix games. Agent 1's action is to choose the row position, and agent 2's action is to choose the column position. The values (r_1, r_2) in the matrix cells show the agents' rewards for each possible joint action. Figure 3.2(a) shows the Rock-Paper-Scissors game in which each agent chooses one of three possible actions (R,P,S). This is a zero-sum game (i.e., $r_1 = -r_2$) since for each action combination, one agent wins (+1 reward) and the other agent loses (-1 reward), or there is a draw (0 reward to both). Figure 3.2(b) shows a "coordination" game with two actions (A,B) for each agent, and agents must select the same action to receive a positive reward. This game is common-reward (i.e., $r_1 = r_2$), hence it suffices to show a single reward in the matrix cells which both of the agents receive. Lastly, Figure 3.2(c) shows a widely studied game called the Prisoner's Dilemma, which is a general-sum game. Here, each agent chooses to either cooperate (C) or defect (D). While mutual cooperation would give both agents the second-highest reward, each agent is individually incentivized to defect since this is the "dominant action," meaning that D always achieves higher rewards compared to C. What makes

2. Zero-sum games are a special case of *constant-sum* games, in which the agents' rewards sum to a constant.

3. For games with two agents, we often use i and j to refer to the two agents in formal descriptions, and we use 1 and 2 to refer to the agents in specific examples.

4. Some game theory literature uses the term "bimatrix game" to specifically refer to general-sum games that require two matrices to define the agents' reward functions, and "matrix game" for games that require only a single matrix to define the reward functions. This book uses "matrix game" in both cases.

	R	P	S
R	0,0	-1,1	1,-1
P	1,-1	0,0	-1,1
S	-1,1	1,-1	0,0

(a) Rock-Paper-Scissors

	A	B
A	10	0
B	0	10

(b) Coordination Game

	C	D
C	-1,-1	-5,0
D	0,-5	-3,-3

(c) Prisoner's Dilemma

Figure 3.2: Three normal-form games with two agents (i.e., matrix games).

these and many other games interesting is that one agent's reward depends on the choices of the other agents, which are not known in advance.

Much effort has been devoted to developing classifications of normal-form games and understanding the relationship between different games (Rapoport and Guyer 1966; Kilgour and Fraser 1988; Walliser 1988; Robinson and Gorforth 2005; Bruns 2015; Marris, Gemp, and Piliouras 2023). For instance, in Section 11.2, we provide a comprehensive listing of all structurally distinct and strictly ordinal 2×2 normal-form games (meaning games with two agents and two actions each), of which there are seventy-eight in total.

In the remainder of the book, we will use “normal-form game” when making general statements about n -agent normal-form games, and we will sometimes use “matrix game” when discussing specific examples of normal-form games for two agents.

3.2 Repeated Normal-Form Games

A normal-form game, as presented in Section 3.1, defines a single interaction between two or more agents. The most basic way to extend this to *sequential* multi-agent interactions is by repeating the same normal-form game for a finite or infinite number of times, giving rise to the *repeated normal-form game*. This type of game model is among the most widely studied models in game theory. For example, the repeated Prisoner's Dilemma has been extensively studied in the game theory literature (e.g., Axelrod 1984) and still serves as an important example of a sequential social dilemma.

Given a normal-form game $\Gamma = (I, \{A_i\}_{i \in I}, \{\mathcal{R}_i\}_{i \in I})$, a repeated normal-form game repeats the same game Γ over T time steps, $t = 0, 1, 2, \dots, T - 1$, where T is either finite or infinite. At each time step t , each agent $i \in I$ samples an action $a_i^t \in A_i$ with probability given by its policy, $\pi_i(a_i^t | h^t)$. The policy is now conditioned on the *joint-action history*, $h^t = (a^0, \dots, a^{t-1})$, which contains all

joint actions before the current time step t (for $t=0$, the history is the empty set). Given the joint action $a^t = (a_1^t, \dots, a_n^t)$, each agent i receives a reward $r_i^t = \mathcal{R}_i(a^t)$.

Besides the time index t , the important addition in repeated normal-form games is that policies can now make decisions based on the entire history of past joint actions, giving rise to a complex space of policies that agents can use. Typically, policies are conditioned on an *internal state* which is a function of the history, $f(h^t)$. For example, a policy may be conditioned on just the most recent joint action a^{t-1} , or on summary statistics such as the action counts of other agents in the history. “Tit-for-Tat” is a famous policy for the game of repeated Prisoner’s Dilemma that simply conditions on the most recent action of the other agent, choosing to cooperate if the other agent cooperated and defecting if the other agent defected (Axelrod and Hamilton 1981).

It is important to note that a game with finite repetitions is not in general equivalent to the same game with infinite repetitions. In finitely repeated games, there can be “end-game” effects: if the agents know that a game will finish after T time steps, they may choose different actions closer to the end of the game compared to earlier in the game (see Section 6.3.3 for an example in Prisoner’s Dilemma). For infinitely repeated games, one may specify a probability with which each time step terminates the game. This termination probability is related to the discount factor $\gamma \in [0, 1]$ used in the discounted-return learning objective in RL, in that $1 - \gamma$ specifies the probability of termination in each time step (see the discussion in Section 2.3). For $\gamma < 1$, the game still counts as “infinite” since any finite number $T > 0$ of time steps will have a non-zero probability of occurring.

In the remainder of this book, we will use the term *non-repeated normal-form game* to refer to the special case when $T = 1$, while repeated normal-form game refers to the case when $T > 1$.

3.3 Stochastic Games

While the relative simplicity of normal-form games is useful to study interactions between agents, they lack the notion of an environment state which is affected by the actions of the agents. Moving closer to the full multi-agent system described in Section 1.1, *stochastic games* define a state-based environment in which the state evolves over time based on the agents’ actions and probabilistic state transitions (Shapley 1953).

Definition 3 (Stochastic game) *A stochastic game consists of:*

- *Finite set of agents* $I = \{1, \dots, n\}$
- *Finite set of states* S , with subset of terminal states $\bar{S} \subset S$

- For each agent $i \in I$:
 - Finite set of actions A_i
 - Reward function $\mathcal{R}_i : S \times A \times S \rightarrow \mathbb{R}$, where $A = A_1 \times \dots \times A_n$
- State transition probability function $\mathcal{T} : S \times A \times S \rightarrow [0, 1]$ such that

$$\forall s \in S, a \in A : \sum_{s' \in S} \mathcal{T}(s, a, s') = 1 \quad (3.1)$$

- Initial state distribution $\mu : S \rightarrow [0, 1]$ such that

$$\sum_{s \in S} \mu(s) = 1 \quad \text{and} \quad \forall s \in \bar{S} : \mu(s) = 0 \quad (3.2)$$

A stochastic game proceeds as follows: The game starts in an initial state $s^0 \in S$ sampled from μ . At time t , each agent $i \in I$ observes the current state $s^t \in S$ and chooses an action $a_i^t \in A_i$ with probability given by its policy, $\pi_i(a_i^t | h^t)$, resulting in the joint action $a^t = (a_1^t, \dots, a_n^t)$. The policy is conditioned on the *state-action history*, $h^t = (s^0, a^0, s^1, a^1, \dots, s^t)$, which contains the current state s^t as well as the states and joint actions from the previous time steps. This history is observed by all agents, which is a property known as *full observability*. Given the state s^t and joint action a^t , the game transitions into a next state $s^{t+1} \in S$ with probability given by $\mathcal{T}(s^t, a^t, s^{t+1})$, and each agent i receives a reward $r_i^t = \mathcal{R}_i(s^t, a^t, s^{t+1})$. We also write this probability as $\mathcal{T}(s^{t+1} | s^t, a^t)$ to emphasize that it is conditioned on the state-action pair s^t, a^t . These steps are repeated until reaching a terminal state $s^t \in \bar{S}$ or completing a maximum number of T time steps,⁵ after which the game terminates; or it may continue for an infinite number of time steps if the game is non-terminating.

Similar to MDPs, stochastic games have the Markov property in that the probability of the next state and reward is conditionally independent of the past states and joint actions, given the current state and joint action

$$\Pr(s^{t+1}, r^t | s^t, a^t, s^{t-1}, a^{t-1}, \dots, s^0, a^0) = \Pr(s^{t+1}, r^t | s^t, a^t) \quad (3.3)$$

where $r^t = (r_1^t, \dots, r_n^t)$ is the joint reward at time t . For this reason, stochastic games are also sometimes called *Markov games* (e.g., Littman 1994).

As a concrete example of a stochastic game, we can model the level-based foraging environment shown in Figure 1.2 (page 4). Each state is a vector that specifies the x-y integer positions of all agents and items, as well as binary flags for each item to indicate whether it has been collected. The agents' action spaces include actions for moving up/down/left/right, collecting an item, and doing nothing (noop). The effect of joint actions is specified in the transition

5. Recall also Footnote 2 (page 22).

probability function $\mathcal{T}$. For example, two agents that together collect an existing item will modify the state by switching the binary flag associated with that item (meaning that the item has been collected and no longer exists). In a common-reward version of the game, every agent will receive a reward of +1 whenever any of the items has been collected by any agents. A general-sum version may specify individual rewards for agents, such as +1 reward for agents which were actually involved in the collection of an item and 0 reward for all other agents. The game terminates after a fixed number of time steps, or when a terminal state has been reached in which all items have been collected.

Stochastic games include repeated normal-form games as a special case in which there is only a single state in S and there are no terminal states (i.e., $\bar{S} = \emptyset$). More generally, if we define reward functions as $\mathcal{R}_i(s, a)$, then each state $s \in S$ in the stochastic game can be viewed as a non-repeated normal-form game with rewards given by $\mathcal{R}_i(s, \cdot)$, as shown in Figure 3.3. (For a comment on the equivalence of $\mathcal{R}_i(s, a, s')$ and $\mathcal{R}_i(s, a)$, see Section 2.8.) It is in this sense that normal-form games form the basic building block of stochastic games. Stochastic games also include MDPs as the special case in which there is only a single agent in I . And, like MDPs, while the state and action sets in our definition of stochastic games are finite (as per the original definition of Shapley (1953)), a stochastic game can be analogously defined for continuous states and actions. Finally, the classification of normal-form games into zero-sum games, common-reward games, and general-sum games also carry over to stochastic games (Section 3.1). That is, stochastic games may specify zero-sum rewards, common rewards, or general-sum rewards.

Chapter 11 gives many more examples of state-based multi-agent environments, many of which can be modeled as a stochastic game.

3.4 Partially Observable Stochastic Games

Sitting at the top of the game model hierarchy shown in Figure 3.1, the most general model we use in this book is the *partially observable stochastic game*, or POSG for short (Hansen, Bernstein, and Zilberstein 2004). While in stochastic games the agents can directly observe the environment state and the chosen actions of all agents, in a POSG, the agents receive “observations” that carry some incomplete information about the environment state and agents’ actions. This allows POSGs to represent decision processes in which agents have limited ability to sense their environment, such as in autonomous driving and other robot control tasks or strategic games where players have private information not seen by other players (e.g., card games).

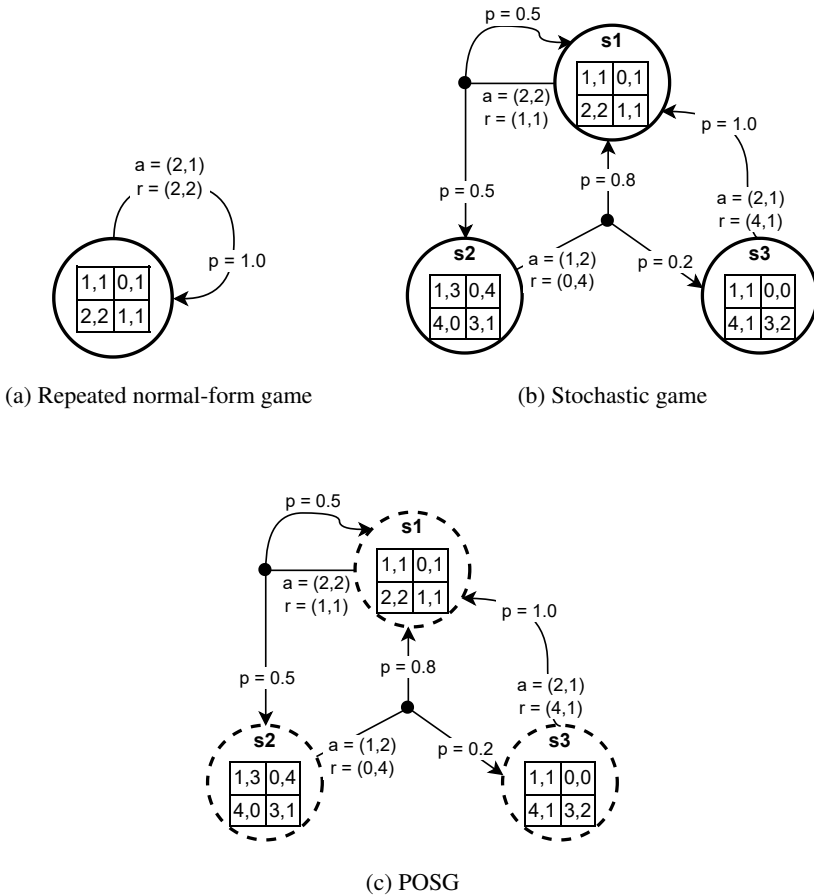


Figure 3.3: Normal-form games are the basic building block of all game models described in Chapter 3. This figure shows one example game for each type of game model, shown as a directed cyclic graph. Each game is for two agents with two actions each. Each node in the graph corresponds to a state and shows a normal-form game being played in that state. We show one possible joint action choice for each state, along with the rewards received and the probabilities of reaching the respective next state in the outgoing arrows. For POSG, the states are dashed to represent that agents do not directly observe the current state of the game; instead, agents receive partial/noisy observations about the state.

In its most general form, a POSG defines state-observation probabilities $\Pr(s^t, o^t | s^{t-1}, a^{t-1})$, where $o^t = (o_1^t, \dots, o_n^t)$ is the joint observation at time t that contains the agents' individual observations o_i^t . However, it is often the case that observations only depend on the new environment state s^t and the joint action a^{t-1} that led to this state (and not on the previous state s^{t-1}). Thus, it is common to define for each agent i an individual *observation function* $\mathcal{O}_i$ that specifies probabilities over the agent's possible observations o_i^t given the state s^t and joint action a^{t-1} . We give the full definition of POSGs next:

Definition 4 (Partially observable stochastic game) *A partially observable stochastic game (POSG) is defined by the same elements of a stochastic game (Definition 3) and additionally defines for each agent $i \in I$:*

- *Finite set of observations O_i*
- *Observation function $\mathcal{O}_i : A \times S \times O_i \rightarrow [0, 1]$ such that*

$$\forall a \in A, s \in S : \sum_{o_i \in O_i} \mathcal{O}_i(a, s, o_i) = 1 \quad (3.4)$$

A POSG proceeds similarly to a stochastic game: The game starts in an initial state $s^0 \in S$ sampled from μ . At each time t , the game is in a state $s^t \in S$ with previous joint action $a^{t-1} \in A$ (for $t=0$ we set $a^{t-1} = \emptyset$), and each agent $i \in I$ receives an observation $o_i^t \in O_i$ with probability given by its observation function, $\mathcal{O}_i(a^{t-1}, s^t, o_i^t)$. We also write this as $\mathcal{O}_i(o_i^t | a^{t-1}, s^t)$ to emphasize that the probability is conditioned on the state s^t and joint action a^{t-1} . Each agent then chooses an action $a_i^t \in A_i$ based on action probabilities given by its policy, $\pi_i(a_i^t | h_i^t)$, resulting in the joint action $a^t = (a_1^t, \dots, a_n^t)$. The policy π_i is conditioned on agent i 's *observation history* $h_i^t = (o_i^0, \dots, o_i^t)$, which includes all of the agent's past observations up to and including the most recent observation. Note that an agent's observation o_i^t may or may not include the actions a_j^{t-1} from the previous time step, as we will further discuss in the next paragraphs. Given the joint action a^t , the game transitions into the next state $s^{t+1} \in S$ with probability $\mathcal{T}(s^{t+1} | s^t, a^t)$, and each agent i receives a reward $r_i^t = \mathcal{R}_i(s^t, a^t, s^{t+1})$. These steps are repeated until reaching a terminal state $s^t \in \bar{S}$ or completing a maximum number of T time steps, after which the game terminates; or it may continue for an infinite number of time steps if the game is non-terminating.

Again, the classification of zero-sum reward, common reward, and general-sum reward also applies to the POSG model. POSGs with common rewards, in which all agents have the same reward function, are also known as “Decentralized POMDP” (Dec-POMDP) and have been widely studied in the area of multi-agent planning (see, for example, the textbook by Oliehoek and Amato

2016). POSGs can also be defined analogously with continuous-valued (or mixed discrete-continuous) observations.

POSGs include stochastic games as the special case in which $o_i^t = (s^t, a^{t-1})$. They also include POMDPs as the special case in which there is only a single agent in I . In general, the observation functions in POSGs can be used to represent diverse observability conditions of interest. Examples include:

Unobserved actions of other agents Agents may observe the state and their own previous action, but not the previous actions of the other agents, that is, $o_i^t = (s^t, a_i^{t-1})$. An example of this scenario is robot soccer, in which the robots may observe the full playing field but do not directly communicate their chosen actions to other players (especially players from the opponent team). In this case, agents may need to infer with some uncertainty the possible actions of other agents based on changes in the observed environment state (e.g., inferring a pass action between two players based on location, direction, and velocity of the ball). Another example is a market setting in which the asset prices are observed by all agents, but the agents' buy/sell actions are private.

Limited view region Agents may observe a subset of the state and joint action, that is, $o_i^t = (\bar{s}^t, \bar{a}^t)$ where $\bar{s}^t \subset s^t$ and $\bar{a}^t \subset a^t$. Such a scenario may arise if agents have a limited view of their surrounding environment, such that an agent may only see those parts of the state s^t and joint action a^{t-1} that are (or took place) within its view region. For example, in the partially observable version of level-based foraging shown in Figure 3.4, the agents have local vision fields and can only see items, other agents, and their actions within their own vision fields. Another example is agents playing a real-time strategy game in which parts of the game map are concealed by “fog of war” (e.g., Vinyals et al. 2019).

Observation functions can model uncertainty in observations (i.e., *noise*) by assigning non-zero probability to multiple possible observations. For example, such uncertainty may be caused by imperfect sensors in robotics applications. Observation functions can also model communication between agents, which may be limited by range and can be unreliable. A communication message can be modeled as a multi-valued vector (e.g., a bit vector) that does not modify the environment state but can be received by other agents within their own observation if they are within a certain range of the sender. Unreliable communication can be modeled by specifying a probability with which messages are lost (i.e., not received by an agent even if within sending range) or by randomly changing parts of the message. One may even specify combined actions $a_i = (a_{i,s}, a_{i,c})$

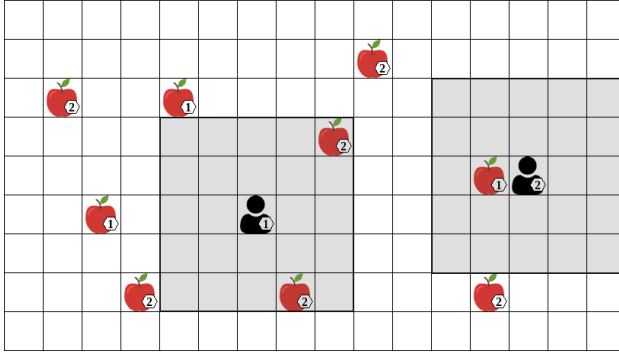


Figure 3.4: Level-based foraging environment with partial observability. Agents observe the world through their local vision fields (shown as gray rectangles around the agents).

where $a_{i,s}$ affects the state s (such as moving around in a physical world) while $a_{i,c}$ is a communication message which may be received by other agents j within their observation o_j .

In Chapter 11 (Section 11.3), we list several multi-agent environments with partial observability used in MARL research.

3.4.1 Belief States and Filtering

In a POSG, since an agent's observation only provides partial information about the current state of the environment, it is typically not possible to choose optimal actions based only on the current observation. For example, in the level-based foraging environment shown in Figure 3.4, the optimal action for the level-1 agent may be to move toward the level-1 item to its left. However, this item is not included in the agent's current observation within its vision field, and so the agent cannot infer the optimal action from its current observation alone. In general, if the environment is only partially observed, the agents must maintain estimates of the possible current states and their relative likelihoods based on the history of observations. Continuing with the example, the level-1 agent may have seen the level-1 item to its left in a previous observation, and may thus remember its location within the observation history.

One way of defining such estimates about environment states from the perspective of agent i is as a *belief state* b_i^t , which is a probability distribution over the possible states $s \in S$ that the environment may be in at time t . Consider for simplicity a POSG with only a single agent i — that is, a POMDP. The initial belief state of the agent is given by the distribution of initial states, that is,

$b_i^0 = \mu$. After taking action a_i^t and observing o_i^{t+1} , the belief state b_i^t is updated to b_i^{t+1} by computing a Bayesian posterior distribution

$$b_i^{t+1}(s') \propto \sum_{s \in S} b_i^t(s) \mathcal{T}(s' | s, a_i^t) \mathcal{O}_i(o_i^{t+1} | a_i^t, s'). \quad (3.5)$$

The resulting belief state is exact in the sense that it retains all relevant information from the observation history. This belief state is known as a “sufficient statistic” because it carries enough information needed to choose optimal actions and to make predictions about the future. The process of updating belief states based on observations is also known as (*belief state*) *filtering*.

Unfortunately, the space complexity of storing such exact belief states and the time complexity of updating them using the Bayesian update from Equation 3.5 are each exponential in the number of variables that define the state, making it intractable for complex environments. Hence, developing algorithms for efficient *approximate* filtering has been the subject of much research; see, for example, Albrecht and Ramamoorthy (2016) and discussions therein.

In a POSG with more than one agent, the definition of belief states and how to update them becomes significantly more complex. In particular, since agents may not observe the chosen actions of other agents and their resulting observations, the agents now also have to infer probabilities over the possible observations and actions of other agents, which in turn requires knowledge of their observation functions and policies (e.g., Gmytrasiewicz and Doshi 2005; Oliehoek and Amato 2016). However, as we will discuss in Section 3.6, in MARL, we typically assume that agents do not possess complete knowledge about the elements of the POSG, such as S , $\mathcal{T}$, and $\mathcal{O}_i$ (including their own observation function), all of which are required in Equation 3.5. Thus, we will not dwell further on the intricacies of defining exact belief states in multi-agent contexts.

To achieve some form of filtering without knowledge of these elements, deep RL algorithms often use recurrent neural networks (discussed in Section 7.5.2) to process observations sequentially. The output vector of the recurrent network learns to encode information about the current state of the environment and can be used to condition other functions of the agent such as value and policy networks. Part II of this book will discuss how MARL algorithms can integrate such deep learning models in order to learn and operate in POSGs. See also Section 8.3 for a further discussion on this topic.

3.5 Modeling Communication

The game models introduced in this chapter are very general and can represent various multi-agent environments, such as the application examples discussed in Section 1.3. Stochastic games and POSGs can also model *communication* between agents by including communication actions that agents can use to send messages to other agents. This section will discuss several ways in which communication actions can be modeled.

Intuitively, we can view communication as a type of action that can be observed by other agents but does not affect the state of the environment. For an agent i , we can model the action space as a combination of two sets

$$A_i = X_i \times M_i \quad (3.6)$$

where X_i includes environment actions that affect the environment state (such as the move and collect actions in level-based foraging) and M_i includes the communication actions. Thus, each action $(x_i, m_i) \in A_i$ specifies both an environment action and a communication action at the same time. At a basic level, M_i may be defined as a set of possible messages $\{m1, m2, m3, \dots\}$, which may be discrete symbols or continuous values. It is also possible to define more complex messages as multi-valued vectors, in which the vector elements may specify discrete or continuous values, or a combination of both. We could also include $\emptyset$ in M_i to represent an empty message.

In a stochastic game, each agent observes the current state s^t and the previous joint action a^{t-1} . Thus, each agent i 's communication action (message) $m_i^{t-1} \in M_i$ is observed (i.e., received) by all other agents as part of the joint action a^{t-1} . In contrast to environment actions X_i , communication actions M_i do not affect the next state s^{t+1} of the environment. Formally, let $M = \times_{i \in I} M_i$, then the state transition probabilities are independent of M , that is,

$$\forall s, s' \in S \forall a \in A, m \in M: \mathcal{T}(s' | s, a) = \mathcal{T}(s' | s, \langle (a_1^x, m_1), \dots, (a_n^x, m_n) \rangle) \quad (3.7)$$

where a_i^x refers to the environment action component of agent i in joint action a and the tuple $\langle (a_1^x, m_1), \dots, (a_n^x, m_n) \rangle$ is a joint action in which the agents use the same environment actions as in a but their communication actions are replaced by $m = (m_1, \dots, m_n)$. Note that, based on this definition, communications are ephemeral in that they last only for a single time step. However, since agents can see the state-action history h that includes all past states and joint actions, they can in principle remember past messages.

In a POSG, we can also model noisy and unreliable communication via the agents' observation functions $\mathcal{O}_i(o_i^t | a^{t-1}, s^t)$. For example, an agent i 's observation could be defined as a vector $o_i^t = [\bar{s}^t, w_1^{t-1}, \dots, w_n^{t-1}]$, where $\bar{s}^t$ contains

some partial information about the state s^t , and w_j^{t-1} is the communication message m_j^{t-1} sent by agent j at time $t - 1$. To model noisy communication, $\mathcal{O}_i$ could specify $w_j^{t-1} = f(m_j^{t-1})$ where f can modify the message in some way. For example, if m_j^{t-1} is continuous-valued, then we could specify $f(m_j^{t-1}) = m_j^{t-1} + \eta$ where η is a random noise component sampled from a Gaussian distribution. $\mathcal{O}_i$ can also model message loss, by specifying a probability with which w_j^{t-1} is set to $\emptyset$ to represent the case that the message from agent j was not received. Additional communication constraints can be modeled via $\mathcal{O}_i$, such as limited communication ranges, by setting w_j^{t-1} to $\emptyset$ if agents i and j are not within a communication range. Again, since agents have access to the history of observations, they can in principle remember past received messages.

The agents may use messages to communicate a variety of information, such as sharing their own observations about the state of the environment or communicating their intended actions and plans. In the level-based foraging example, the set M_i may contain a message corresponding to each possible location in the grid-world, and these messages could be used by the agents to communicate their intended goal locations to other agents or to tell other agents about item locations they have discovered. However, in reinforcement learning, the standard assumption is that agents do not know the *meaning* of the actions in A_i , including the communication actions. (See Section 3.6 for further discussion of knowledge assumptions.) Therefore, each communication action in M_i is an abstract action for agent i just like any environment action in X_i , and the agents have to learn the meaning of actions via repeated trials and observations. For communication actions, there is also the added difficulty that agents need to learn how to *interpret* the messages sent by other agents. This opens the possibility that the agents can learn and evolve a shared language or communication protocol (Foerster et al. 2016; Sukhbaatar, Szlam, and Fergus 2016; Wang, He, et al. 2020; Guo et al. 2022).

The learning algorithms we will present in the later chapters are general in that they learn policies over any given action sets A_i , which may or may not include communication actions. While communication actions may be modeled as part of A_i (such as discussed earlier), in the remainder of this book we will keep our descriptions general and will not give special consideration to communication actions.

3.6 Knowledge Assumptions in Games

What do the agents know about the environment they interact in and how it works? In other words, what do agents know about the game they are playing?

In game theory, the standard assumption is that all agents have knowledge of all components that define the game; this is referred to as a “complete knowledge game” (Owen 2013). For normal-form games, this means that the agents know the action spaces and reward functions of all agents (including their own). For stochastic games and POSGs, agents also know the state space and state transition function, as well as the observation functions of all agents. Knowledge of these components can be utilized in different ways. For example, if agent i knows the reward function $\mathcal{R}_j$ of another agent j , then agent i may be able to estimate the best-response action (Section 4.2) of agent j , which in turn could inform the optimal action for agent i . If an agent knows the transition function $\mathcal{T}$, it may predict the effects of its actions on the state and plan its actions several steps into the future.

However, in most real-world applications of interest (such as those mentioned in Chapter 1), it is infeasible to obtain accurate and complete specifications of these components. For such applications, often the best we can obtain is a simulator of these components that can generate samples of states, (joint) rewards, and (joint) observations. For example, we may have access to a simulator $\widehat{\mathcal{T}}$ that, given an input state s and joint action a , can produce samples of joint rewards and successor states $(r = (r_1, \dots, r_n), s') \sim \widehat{\mathcal{T}}(s, a)$, such that

$$\Pr \left\{ \widehat{\mathcal{T}}(s, a) = (r, s') \right\} \approx \mathcal{T}(s' | s, a) \prod_{i \in I} [\mathcal{R}_i(s, a, s') = r_i]_1 \quad (3.8)$$

where $[x]_1 = 1$ if x is true, else $[x]_1 = 0$.

In MARL, we usually operate at the other end of the knowledge spectrum: agents typically do not know the reward functions of other agents, nor even their own reward function; and agents have no knowledge of the state transition and observation functions (in game theory, this is also known as “incomplete information game” (Harsanyi 1967)). Instead, an agent i only experiences the immediate effects of its own actions, via its own reward r_i^t and (in stochastic games) the joint action a^t and resulting next state s^{t+1} or (in POSG) an observation o_i^{t+1} . Agents may use such experiences to construct models of the unknown components in a game (such as $\mathcal{T}$) or the policies of other agents.

Additional assumptions may pertain to whether certain knowledge about the game is held mutually by all agents or not (symmetric vs. asymmetric knowledge) and whether agents are aware of what other agents know about the game. Are the reward functions in a game known to all agents or only to some of the agents? If all agents have knowledge of the reward functions of each agent, are they also aware of the fact that all agents have this knowledge, the fact that all agents knowing the reward functions is known by all agents, and so on (common knowledge)? While such questions and their impact on optimal decision making

have been the subject of much research in game theory (Shoham and Leyton-Brown 2008; Perea 2012) and agent modeling (Albrecht and Stone 2018), in MARL they have played a relatively lesser role since the standard assumption is that agents have no knowledge of most game components. Exceptions can be found in MARL research for zero-sum or common-reward games, where the specific reward structure is exploited in some way in the algorithm design. Chapters 6 and 9 will describe several MARL algorithms of this latter category.

Lastly, it is usually assumed that the number of agents in a game is fixed, and that this fact is known by all agents. While outside the scope of this book, it is interesting to note that recent research in MARL has begun to tackle *open* multi-agent environments in which agents may dynamically enter and leave the environment (Jiang et al. 2020; Rahman et al. 2021; Rahman, Carlucho, et al. 2023).

3.7 Dictionary: Reinforcement Learning $\leftrightarrow$ Game Theory

This book lies at the intersection of reinforcement learning and game theory. These distinct fields share a number of concepts but use different terminology, and this book primarily uses the terminology that is common in reinforcement learning. We conclude this chapter by providing a small “dictionary” in Figure 3.5 that shows some synonymous terms between the two fields.

3.8 Summary

This chapter introduced a hierarchy of increasingly complex game models, in which multiple agents interact in a shared environment. In summary, the main concepts are the following:

- The *normal-form game* is the most basic type of game model, defining a single interaction between two or more agents. Each agent uses a policy that assigns probabilities to the actions available to the agent. Given a joint action that specifies an action for each agent, each agent receives a reward and the interaction ends. Normal-form games with two agents are also called *matrix games*, since the agent’s reward functions can be represented as matrices.
- *Repeated normal-form games* repeat the same normal-form game over multiple time steps. The policies of agents can be conditioned on the history of past joint actions chosen by the agents.

RL (this book)	GT	Description
environment	game	Model specifying the possible actions, observations, and rewards of agents, and the dynamics of how the state evolves over time and in response to actions.
agent	player	An entity which makes decisions. “Player” can also refer to a specific role in the game that is assumed by an agent, for example: • “row player” in a matrix game
reward	payoff, utility	• “white player” in chess Scalar value received by an agent/player after taking an action.
policy	strategy	Function used by an agent/player to assign probabilities to actions. In game theory, “(pure) strategy” is also sometimes used to refer to actions.
deterministic X	pure X	X assigns probability 1 to one option, for example: • deterministic policy • pure strategy
probabilistic X	mixed X	• deterministic/pure Nash equilibrium X assigns probabilities ≤ 1 to options, for example: • probabilistic policy • mixed strategy
joint X	X profile	• probabilistic/mixed Nash equilibrium X is a tuple, typically with one element for each agent/player, for example: • joint reward • deterministic joint policy • payoff profile
		• pure strategy profile

Figure 3.5: Synonymous terms in reinforcement learning (RL) and game theory (GT). This book uses RL terminology.

- *Stochastic games* additionally define an environment state that can change in response to the agents' actions. Agents can see the full environment state and the previous actions of all other agents. The game terminates once it reaches a terminal state or a maximum number of time steps; or it may continue for an infinite number of time steps. Stochastic games include repeated normal-form games as a special case in which there is only a single environment state, and they include MDPs as a special case in which there is only a single agent.
- In *partially observable stochastic games* (POSGs), the agents make decisions based on potentially incomplete and noisy observations about their environment. Instead of observing the complete environment state and the past actions of other agents, POSGs define observation functions that generate individual observations for each agent that depend on the state and past joint action. The POSG model includes all of the other game models covered in this book as special cases.
- Games can be categorized based on the relationship between the agents' rewards. In *zero-sum* games, the agents' rewards always sum up to zero. In *common-reward* games, each agent receives the same reward. *General-sum* games are the most general games and do not define any restrictions on the relationship of reward functions.
- Communication can be modeled in stochastic games and POSGs by including special *communication actions*. These actions can be observed by other agents but do not affect the environment state. POSGs can also model noisy and unreliable communication channels with limited range.
- Different assumptions can be made about what knowledge the agents have about the different elements that define a game, such as the state transition probabilities and reward functions. In MARL, the standard assumption is that the agents do not know these elements.

Many of the dimensions of MARL listed in Figure 1.4 (page 8) are determined in the specification of a game. Similarly to single-agent RL, a learning problem in MARL is given by the combination of a game model and a learning objective for the agents. In Chapter 4, we will introduce a range of solution concepts that can be used as learning objectives in games.

4 Solution Concepts for Games

What does it mean for agents to interact optimally in a multi-agent system? In other words, what is a *solution* to a game? This is a central question of game theory, and many different *solution concepts* have been proposed that specify when a collection of agent policies constitutes a stable or desirable outcome. While Chapter 3 introduced the basic game models to formalize multi-agent environments and interaction, this chapter will introduce a series of solution concepts. Together, a game model and solution concept define a learning problem in MARL (Figure 4.1).

For common-reward games, in which all agents receive the same reward, a straightforward definition for a solution is to maximize the expected return received by all agents (but finding such a solution may not be simple at all, as we will see in later chapters). However, if the agents have differing rewards, the definition of a solution concept becomes more complicated.

In general, a solution to a game is a *joint policy* that consists of one policy for each agent and satisfies certain properties. These properties are expressed in terms of the expected returns yielded to each agent under the joint policy and the relations between the agents' returns. Thus, this chapter will begin by giving a universal definition of expected returns that applies to all of the game models introduced in Chapter 3. Using this definition, we will introduce a hierarchy of increasingly general equilibrium solution concepts that include the minimax equilibrium, Nash equilibrium, and correlated equilibrium. We will also discuss solution refinements such as Pareto-optimality and social welfare and fairness, and alternative solution concepts such as no-regret. We finish the chapter with a discussion of the computational complexity of computing Nash equilibria.

Note that our definitions of solution concepts and their stated existence properties assume *finite* game models, as defined in Chapter 3. In particular, we assume finite state, action, and observation spaces, and a finite number of agents. Games with infinite elements, such as continuous actions and observations, can

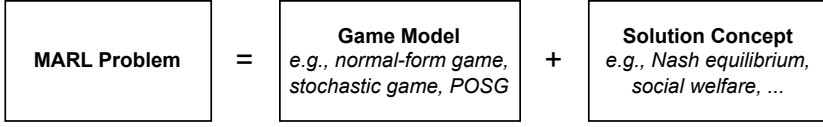


Figure 4.1: A MARL problem is defined by the combination of a game model, which defines the mechanics of the multi-agent system and interactions, and a solution concept, which specifies the desired properties of the joint policy to be learned. (See also Figure 2.1, page 20.)

use analogous definitions (e.g., by using densities and integrals) but may have different existence properties for solutions.¹

Game theory has produced a wealth of knowledge on foundational questions about solution concepts, including: For a given type of game model and solution concept, is a solution guaranteed to exist in the game? Is the solution unique (i.e., only one solution exists) or are there potentially many solutions, even infinitely many? And is a given learning and decision rule, when used by all agents, guaranteed to converge to a solution? For a more in-depth treatment of such questions, we recommend the books of Fudenberg and Levine (1998), Young (2004), Shoham and Leyton-Brown (2008), and Owen (2013).

4.1 Joint Policy and Expected Return

A solution to a game is a *joint policy*, $\pi = (\pi_1, \dots, \pi_n)$, which satisfies certain requirements defined by the solution concept in terms of the expected return, $U_i(\pi)$, yielded to each agent i under the joint policy. We seek a universal definition of the expected return $U_i(\pi)$, which applies to all of the game models introduced in Chapter 3, so that our definitions of solution concepts also apply to all game models. Thus, we will define the expected return in the context of the POSG model (Section 3.4), which is the most general game model used in this book and includes both stochastic games and normal-form games.

We start by introducing some additional notation. In a POSG, let $\hat{h}^t = \{(s^\tau, o^\tau, a^\tau)_{\tau=0}^{t-1}, s^t, o^t\}$ denote a *full history* up to time t , consisting of the states, joint observations, and joint actions of all agents in each time step before t , and the state s^t and joint observation o^t at time t . The function $\sigma(\hat{h}^t) = (o^0, \dots, o^t)$

1. For example, every two-agent zero-sum normal-form game with finite action spaces has a unique minimax game value, but zero-sum games with continuous actions exist that do not have a minimax game value (Sion and Wolfe 1957). Sufficient conditions for equilibrium existence in continuous games have been studied in works such as Debreu (1952), Fan (1952), Glicksberg (1952), and Dasgupta and Maskin (1986).

returns the history of joint observations from the full history $\hat{h}^t$, and we sometimes abbreviate $\sigma(\hat{h}^t)$ to h^t if it is clear from context. We define the probability of a joint observation, $\mathcal{O}(o^t | a^{t-1}, s^t)$, as the product $\prod_{i \in I} \mathcal{O}_i(o_i^t | a^{t-1}, s^t)$.

To make our definitions valid for both finite and infinite time steps, we use discounted returns² and assume the standard convention of absorbing states, which we defined in Section 2.3. That is, once an absorbing state has been reached, the game will subsequently always transition into that same state with probability 1 and give a reward of 0 to all agents. To simplify our definitions below, we also assume that the observation functions and policies of all agents become deterministic (i.e., assign probability 1 to a certain observation and action, respectively) once an absorbing state has been reached. Thus, our definitions encompass infinite episodes as well as episodes that terminate after a finite number of time steps, including non-repeated normal-form games that always terminate after a single time step.³

In the following, we provide two equivalent definitions of expected returns. The first definition is based on enumerating all full histories in the game, while the second definition is based on a Bellman-style recursion of value computations. These definitions are equivalent, but can provide different perspectives and have been used in different ways. In particular, the first definition resembles a linear sum and may be easier to interpret, while the second definition uses a recursion that can be operationalized such as in value iteration for games (introduced in Section 6.1).

History-based expected return: Given a joint policy π , we can define the expected return for agent i under π by enumerating all possible full histories and summing the returns for agent i in each history, weighted by the probability of generating the history under the POSG and joint policy π . Formally, define the set $\hat{H}$ to contain all full histories $\hat{h}^t$ for $t \rightarrow \infty$.⁴ Then, the expected return for agent i under joint policy π is given by

$$U_i(\pi) = \lim_{t \rightarrow \infty} \mathbb{E}_{\hat{h}^t \sim (\mu, \mathcal{T}, \mathcal{O}, \pi)} \left[u_i(\hat{h}^t) \right] \quad (4.1)$$

$$= \sum_{\hat{h}^t \in \hat{H}} \Pr(\hat{h}^t | \pi) u_i(\hat{h}^t) \quad (4.2)$$

2. Alternative definitions based on average rewards instead of discounted returns are also possible for games (Shoham and Leyton-Brown 2008).

3. In the context of non-repeated normal-form games, the expected return for an agent simply becomes the expected reward of that agent, so we use the term “expected return” in all game models including non-repeated normal-form games.

4. Recall from Chapter 3 that we assume finite game models, and that a^{t-1} before time $t=0$ is the empty set.

where $\Pr(\hat{h}^t | \pi)$ is the probability of full history $\hat{h}^t$ under π ,

$$\Pr(\hat{h}^t | \pi) = \mu(s^0) \mathcal{O}(o^0 | \emptyset, s^0) \prod_{\tau=0}^{t-1} \pi(a^\tau | h^\tau) \mathcal{T}(s^{\tau+1} | s^\tau, a^\tau) \mathcal{O}(o^{\tau+1} | a^\tau, s^{\tau+1}) \quad (4.3)$$

and $u_i(\hat{h}^t)$ is the discounted return for agent i in $\hat{h}^t$,

$$u_i(\hat{h}^t) = \sum_{\tau=0}^{t-1} \gamma^\tau \mathcal{R}_i(s^\tau, a^\tau, s^{\tau+1}) \quad (4.4)$$

with discount factor $\gamma \in [0, 1]$.

We use $\pi(a^\tau | h^\tau)$ to denote the probability of joint action a^τ under joint policy π after joint-observation history h^τ . If we assume that agents act independently, then we can define

$$\pi(a^\tau | h^\tau) = \prod_{j \in I} \pi_j(a_j^\tau | h_j^\tau). \quad (4.5)$$

If agents do not act independently, such as in correlated equilibrium (Section 4.6) and methods such as central learning (Section 5.3.1), then $\pi(a^\tau | h^\tau)$ can be defined accordingly.

Recursive expected return: Analogous to the Bellman recursion used in MDP theory (Section 2.4), we can define the expected return for agent i under joint policy π via two interlocked functions V_i^π and Q_i^π , defined below. In the following, we use $s(\hat{h})$ to denote the last state in $\hat{h}$ (i.e., $s(\hat{h}^t) = s^t$), and we use $\langle \rangle$ to denote the concatenation operation.

$$V_i^\pi(\hat{h}) = \sum_{a \in A} \pi(a | \sigma(\hat{h})) Q_i^\pi(\hat{h}, a) \quad (4.6)$$

$$Q_i^\pi(\hat{h}, a) = \sum_{s' \in S} \mathcal{T}(s' | s(\hat{h}), a) \left[\mathcal{R}_i(s(\hat{h}), a, s') + \gamma \sum_{o' \in \mathcal{O}} \mathcal{O}(o' | a, s') V_i^\pi(\langle \hat{h}, a, s', o' \rangle) \right] \quad (4.7)$$

We can understand $V_i^\pi(\hat{h})$ as the expected return, also called *value*, for agent i when agents follow the joint policy π after the full history $\hat{h}$. Similarly, $Q_i^\pi(\hat{h}, a)$ is the expected return for agent i when agents execute the joint action a after $\hat{h}$ and then follow π subsequently. Given Equations 4.6 and 4.7, we can define the expected return for agent i from the initial state of the game as

$$U_i(\pi) = \mathbb{E}_{s^0 \sim \mu, o^0 \sim \mathcal{O}(\cdot | \emptyset, s^0)} [V_i^\pi(\langle s^0, o^0 \rangle)] \quad (4.8)$$

The equivalence of the history-based and recursive definitions for U_i can, intuitively, be seen by viewing the former as enumerating all possible infinite full histories, while the latter recursively constructs an infinite tree rooted in the initial state s^0 (or, one tree for each possible initial state $s^0 \sim \mu$) in which the

branching corresponds to the different possible full histories. In this chapter, we will define most of the presented solution concepts based on U_i , with the understanding that U_i can be defined via the two equivalent definitions given above.

4.2 Best Response

Many existing solution concepts, including most of the solution concepts introduced in this chapter, can be expressed compactly based on *best responses*. Given a set of policies for all agents other than agent i , denoted by $\pi_{-i} = (\pi_1, \dots, \pi_{i-1}, \pi_{i+1}, \dots, \pi_n)$, a best response for agent i to π_{-i} is a policy π_i that maximizes the expected return for i when played against π_{-i} . Formally, the set of best-response policies for agent i is defined as

$$\text{BR}_i(\pi_{-i}) = \arg \max_{\pi_i} U_i(\langle \pi_i, \pi_{-i} \rangle) \quad (4.9)$$

where $\langle \pi_i, \pi_{-i} \rangle$ denotes the complete joint policy consisting of π_i and π_{-i} . For convenience and to keep our notation lean, we will sometimes drop the $\langle \rangle$ (e.g., $U_i(\pi_i, \pi_{-i})$).

Note that the best response to a given π_{-i} may not be unique, meaning that $\text{BR}_i(\pi_{-i})$ may contain more than one best-response policy. For example, in a non-repeated normal-form game, there may be multiple actions for agent i that achieve equal maximum expected return against a given π_{-i} , in which case any probability assignment to these actions is also a best response. (We will see a concrete example in the Rock-Paper-Scissors game in Section 4.3.)

Besides being useful for compact definitions of solution concepts, best response operators have also been used in game theory and MARL to iteratively compute solutions. Two example methods we will see include fictitious play (Section 6.3.1) and joint-action learning with agent modeling (Section 6.3.2), among others.

4.3 Minimax

Minimax is a solution concept defined for two-agent zero-sum games, in which one agent's reward is the negative of the other agent's reward (Section 3.1). A classical example of such games is the matrix game Rock-Paper-Scissors (see Figure 3.2(a), page 46). More complex examples with sequential moves include games such as chess and Go. The existence of minimax solutions for normal-form games was first proven in the foundational game theory work of von Neumann (1928) (see also von Neumann and Morgenstern (1944)).

Definition 5 (Minimax solution) *In a zero-sum game with two agents, a joint policy $\pi = (\pi_i, \pi_j)$ is a minimax solution if*

$$U_i(\pi) = \max_{\pi'_i} \min_{\pi'_j} U_i(\pi'_i, \pi'_j) \quad (4.10)$$

$$= \min_{\pi'_j} \max_{\pi'_i} U_i(\pi'_i, \pi'_j) \quad (4.11)$$

$$= -U_j(\pi).$$

Every two-agent zero-sum normal-form game has a minimax solution (von Neumann and Morgenstern 1944). Minimax solutions also exist in every two-agent zero-sum stochastic game with finite episode length, and two-agent zero-sum stochastic games with infinite episode length using discounted returns (Shapley 1953). Moreover, while more than one minimax solution may exist in a game, all minimax solutions yield the same unique value $U_i(\pi)$ for agent i (and, thus, agent j). This value is also referred to as the (*minimax*) *value of the game*.⁵ Minimax values can also be defined for zero-sum games with more than two agents (see, for example, Equation 4.17 in Section 4.4).

In a minimax solution, each agent uses a policy that is optimized against a worst-case opponent that seeks to minimize the agent's return. Formally, there are two parts in the definition of minimax. Equation 4.10 is the minimum expected return that agent i can guarantee against *any* opponent. Here, π_i is agent i 's *maxmin* policy and $U_i(\pi)$ is i 's maxmin value. Conversely, Equation 4.11 is the minimum expected return that agent j can force on agent i . Here, we refer to j 's policy π_j as the *minmax* policy against agent i , and $U_i(\pi)$ is agent i 's minmax value. In a minimax solution, agent i 's maxmin value is equal to its minmax value. Another way to interpret this is that the order of the min/max operators does not matter: agent i first announcing its policy followed by agent j selecting its policy, is equivalent to agent j first announcing its policy followed by agent i selecting its policy. Neither agent gains from such policy announcements.

A minimax solution can be understood more intuitively as each agent using a best-response policy against the other agent's policy. That is, (π_i, π_j) is a minimax solution if $\pi_i \in \text{BR}_i(\pi_j)$ and $\pi_j \in \text{BR}_j(\pi_i)$. In the non-repeated Rock-Paper-Scissors game, there exists a unique minimax solution which is for both agents to choose actions uniformly randomly (i.e., assign equal probability to all actions). This solution gives an expected return of 0 to both agents. In

5. Recall that we assume finite game models. For zero-sum games with continuous actions, examples exist that do not have a minimax game value (Sion and Wolfe 1957).

fact, it can be verified⁶ in this game that if any agent i uses a uniform policy π_i , then *any* policy π_j for the other agent j is a best-response policy to π_i , and all best-response policies $\pi_j \in \text{BR}_j(\pi_i)$ yield an expected reward of 0 to agent j (and to agent i). This example shows that best responses need not be unique, and there can be many (even infinitely many) possible best-response policies. However, the joint policy (π_i, π_j) in which both policies are uniform policies is the *only* joint policy in which *both* policies are best responses to each other, making it a minimax solution. In Section 4.4, we will see that this best-response relation can also be applied to the more general class of general-sum games.

4.3.1 Minimax Solution via Linear Programming

For non-repeated zero-sum normal-form games, we can obtain a minimax solution by solving two linear programs, one for each agent. Each linear program computes a policy for one agent by minimizing the expected return of the other agent. Thus, agent i minimizes the expected return of agent j , and vice versa. We provide the linear program to compute the policy π_i for agent i ; agent j 's policy is obtained by constructing a similar linear program in which the indices i and j are swapped. The linear program contains variables x_{a_i} for each action $a_i \in A_i$ to represent the probability of selecting action a_i (thus, $\pi_i(a_i) = x_{a_i}$ defines the policy of agent i), as well as a variable U_j^* to represent the expected return of agent j , which is minimized as follows:

$$\text{minimize } U_j^* \quad (4.12)$$

$$\text{subject to } \sum_{a_i \in A_i} \mathcal{R}_j(a_i, a_j) x_{a_i} \leq U_j^* \quad \forall a_j \in A_j \quad (4.13)$$

$$x_{a_i} \geq 0 \quad \forall a_i \in A_i \quad (4.14)$$

$$\sum_{a_i \in A_i} x_{a_i} = 1 \quad (4.15)$$

In this linear program, the constraints in Equation 4.13 mean that no single action of agent j will achieve an expected return greater than U_j^* against the policy specified by $\pi_i(a_i) = x_{a_i}$. This implies that no probability distribution over agent j 's actions will achieve a higher expected return than U_j^* . Finally, the constraints in Equations 4.14 and 4.15 ensure that the values of x_{a_i} form a valid

6. Assume agent 1's policy assigns probability $\frac{1}{3}$ to each action R, P, S . The expected reward to agent 2 for any policy π_2 is: $\pi_2(R)(\frac{1}{3}\mathcal{R}_2(R, R) + \frac{1}{3}\mathcal{R}_2(P, R) + \frac{1}{3}\mathcal{R}_2(S, R)) + \pi_2(P)(\frac{1}{3}\mathcal{R}_2(R, P) + \frac{1}{3}\mathcal{R}_2(P, P) + \frac{1}{3}\mathcal{R}_2(S, P)) + \pi_2(S)(\frac{1}{3}\mathcal{R}_2(R, S) + \frac{1}{3}\mathcal{R}_2(P, S) + \frac{1}{3}\mathcal{R}_2(S, S)) = \pi_2(R)(-\frac{1}{3} + \frac{1}{3}) + \pi_2(P)(+\frac{1}{3} - \frac{1}{3}) + \pi_2(S)(-\frac{1}{3} + \frac{1}{3}) = 0$. The same can be shown when switching the roles of agents 1 and 2.

probability distribution. Note that in a minimax solution, we will have $U_i^* = U_j^*$ (see Definition 5).

Linear programs can be solved using well-known algorithms, such as the simplex algorithm, which runs in exponential time in the worst case but often is very efficient in practice; or interior-point algorithms, which are provably polynomial-time.

4.4 Nash Equilibrium

The *Nash equilibrium* solution concept applies the idea of a mutual best response to general-sum games with two or more agents. That such a solution exists in any general-sum non-repeated normal-form game was first proven in the celebrated work of Nash (1950).

Definition 6 (Nash equilibrium) *In a general-sum game with n agents, a joint policy $\pi = (\pi_1, \dots, \pi_n)$ is a Nash equilibrium if*

$$\forall i, \pi'_i : U_i(\pi'_i, \pi_{-i}) \leq U_i(\pi). \quad (4.16)$$

In a Nash equilibrium, no agent i can improve its expected return by changing its policy π_i specified in the equilibrium joint policy π , assuming the policies of the other agents stay fixed. This means that each agent's policy in the Nash equilibrium is a best response to the policies of the other agents, that is, $\pi_i \in \text{BR}_i(\pi_{-i})$ for all $i \in I$. Thus, Nash equilibrium generalizes minimax in that, in two-agent zero-sum games, the set of minimax solutions coincides with the set of Nash equilibria (Owen 2013). Nash (1950) first proved that every finite normal-form game has at least one Nash equilibrium.

Recall the matrix games shown in Figure 3.2 (page 46). In the non-repeated Prisoner's Dilemma matrix game, the only Nash equilibrium is for both agents to choose D. It can be seen that no agent can unilaterally deviate from this choice to increase its expected return. In the non-repeated coordination game, there exist three different Nash equilibria: (1) both agents choose action A, (2) both agents choose action B, and (3) both agents assign probability 0.5 to each action. Again it can be checked that no agent can increase its expected return by deviating from its policy in each equilibrium. Lastly, in the non-repeated Rock-Paper-Scissors game, the only Nash equilibrium is for both agents to choose actions uniform-randomly, which is the minimax solution of the game.

The previous examples illustrate two important aspects of Nash equilibrium as a solution concept. First, a Nash equilibrium can be *deterministic* in that each policy π_i in the equilibrium π is deterministic (i.e., $\pi_i(a_i) = 1$ for some $a_i \in A_i$), such as in Prisoner's Dilemma. However, in general, a Nash equilibrium may

be *probabilistic* in that the policies in the equilibrium use randomization (i.e., $\pi_i(a_i) < 1$ for some $a_i \in A_i$), such as in the coordination game and Rock-Paper-Scissors. In fact, some games only have probabilistic Nash equilibria but no deterministic Nash equilibria, such as the Rock-Paper-Scissors game. As we will see in Chapter 6, this distinction is important in MARL because some algorithms are unable to represent probabilistic policies, and hence cannot learn probabilistic equilibria. In the game theory literature, deterministic and probabilistic equilibria are also called “pure equilibria” and “mixed equilibria,” respectively; see also Section 3.7 for terminology.

Second, a game may have multiple Nash equilibria, and each equilibrium may entail different expected returns for the agents. In the coordination game, the two deterministic equilibria give an expected return of 10 to each agent, while the probabilistic equilibrium gives an expected return of 5 to each agent. The Chicken game (Section 4.6) and Stag Hunt game (Section 5.4.2) are other examples of games with multiple Nash equilibria that each give different expected returns to the agents. This leads to the important question of which equilibrium the agents should converge to during learning and how this may be achieved. We will discuss this *equilibrium selection* problem in more depth in Section 5.4.2.

The existence of Nash equilibria has also been shown for stochastic games (Fink 1964; Filar and Vrieze 2012). In fact, for games with (infinite) sequential moves, there are various “folk theorems”⁷ which essentially state that any set of feasible and enforceable expected returns $\hat{U} = (\hat{U}_1, \dots, \hat{U}_n)$ can be achieved by an equilibrium solution if agents are sufficiently far-sighted (i.e., the discount factor γ is close to 1). The assumptions and details in different folk theorems vary and can be quite involved, and here we only provide a rudimentary description for intuition (for more specific definitions, see, e.g., Fudenberg and Levine 1998; Shoham and Leyton-Brown 2008). Broadly speaking, $\hat{U}$ is *feasible* if it can be realized within the game by some joint policy π ; that is, there exists a π such that $U_i(\pi) = \hat{U}_i$ for all $i \in I$. And $\hat{U}$ is *enforceable* if each $\hat{U}_i$ is at least as large as agent i ’s minmax value⁸

$$v_i = \min_{\pi_{-i}^{mm}} \max_{\pi_i^{mm}} U_i(\pi_i^{mm}, \pi_{-i}^{mm}). \quad (4.17)$$

(Think: other agents $-i$ minimize the maximum achievable return for agent i .) Under these two conditions, we can construct an equilibrium solution that uses

7. The name “folk theorem” apparently came about because the general concept was known before it was formalized.

8. We first encountered minmax in Section 4.3 where it was defined for two agents, while in Equation 4.17 we define it for n agents (the *mm* superscript stands for minmax).

π to achieve $\hat{U}$, and if at any time t any agent i deviates from its policy π_i in π , the other agents will limit i 's return to v_i by using their corresponding minmax policy π_i^{mm} from Equation 4.17 indefinitely after t . Thus, since $v_i \leq \hat{U}_i$, i has no incentive to deviate from π_i , making π an equilibrium.

Given a joint policy π , how can one check whether it is a Nash equilibrium? Equation 4.16 suggests the following procedure, which reduces the multi-agent problem to n single-agent problems. For each agent i , keep the other policies π_{-i} fixed and compute an optimal best-response policy π'_i for i . If the optimal policy π'_i achieves a higher expected return than agent i 's policy π_i in the joint policy π , that is, $U_i(\pi'_i, \pi_{-i}) > U_i(\pi_i, \pi_{-i})$, then we know that π is not a Nash equilibrium. For non-repeated normal-form games, π'_i can be computed efficiently using a linear program (e.g., Albrecht and Ramamoorthy 2012). For games with sequential moves, π'_i may be computed using a suitable single-agent RL algorithm.

4.5 ϵ -Nash Equilibrium

The strict requirement of Nash equilibrium — that no agent can gain *anything* by unilaterally deviating from the equilibrium — can lead to practical issues when used in a computational system. It is known that for games with more than two agents, the action probabilities specified by the policies in the equilibrium may be irrational numbers (i.e., cannot be represented as a fraction of two integers). Nash himself pointed this out in his original work (Nash 1950). However, computer systems cannot fully represent irrational numbers using finite-precision floating-point approximations. Moreover, in many applications, reaching a strict equilibrium may be too computationally costly. Instead, it may be good enough to compute a solution that is sufficiently close to a strict equilibrium, meaning that agents could technically deviate to improve their returns but any such gains are sufficiently small.

The ϵ -Nash equilibrium relaxes the strict Nash equilibrium by requiring that no agent can improve its expected returns by more than some amount $\epsilon > 0$ when deviating from its policy in the equilibrium. Formally:

Definition 7 (ϵ -Nash equilibrium) *In a general-sum game with n agents, a joint policy $\pi = (\pi_1, \dots, \pi_n)$ is an ϵ -Nash equilibrium for $\epsilon > 0$ if*

$$\forall i, \pi'_i : U_i(\pi'_i, \pi_{-i}) - \epsilon \leq U_i(\pi). \quad (4.18)$$

Since the action probabilities specified by π'_i are continuous and we consider expected returns U_i , we know that every Nash equilibrium is surrounded by a region of ϵ -Nash equilibria for any $\epsilon > 0$. The exact Nash equilibrium corresponds

	C	D
A	100,100	0,0
B	1,2	1,1

Figure 4.2: Matrix game to illustrate that an ϵ -Nash equilibrium (B,D) ($\epsilon = 1$) may not be close to a real Nash equilibrium (A,C).

to $\epsilon = 0$. However, although it may be tempting to view ϵ -Nash equilibrium as an approximation of Nash equilibrium, it is important to note that an ϵ -Nash equilibrium may not be close to any real Nash equilibrium, in terms of the expected returns produced by the equilibrium. In fact, the expected returns under an ϵ -Nash equilibrium may be arbitrarily far away from those of any Nash equilibrium, even if the Nash equilibrium is unique.

Consider the example game shown in Figure 4.2. This game has a unique Nash equilibrium at (A,C).⁹ It also has an ϵ -Nash equilibrium at (B,D) for $\epsilon = 1$, in which agent 2 could deviate to action C to increase its reward by ϵ . First, note that neither agent's return under the ϵ -Nash equilibrium is within ϵ of its return under the Nash equilibrium. Second, we can arbitrarily increase the rewards for (A,C) in the game without affecting the ϵ -Nash equilibrium (B,D). Thus, in this example, the ϵ -Nash equilibrium does not meaningfully approximate the unique Nash equilibrium of the game.

To check that a joint policy π constitutes an ϵ -Nash equilibrium for some given ϵ , we can use essentially the same procedure for checking Nash equilibrium described at the end of Section 4.4, except that we check for $U_i(\pi'_i, \pi_{-i}) - \epsilon > U_i(\pi_i, \pi_{-i})$ to determine that π is not an ϵ -Nash equilibrium.

4.6 (Coarse) Correlated Equilibrium

A restriction of Nash equilibrium is that the agent policies must be probabilistically independent (as per Equation 4.5), which can limit the expected returns that can be achieved by the agents. *Correlated equilibrium* (Aumann 1974) generalizes Nash equilibrium by allowing for correlation between policies. In the general definition of correlated equilibrium, each agent i 's policy is additionally conditioned on the outcomes of a private random variable d_i for the agent, which are governed by a joint probability distribution over $(d_1, \dots, d_n)$ that is commonly known by all agents. Here, we will present a common version of correlated

9. Saying that a joint action a is an X-equilibrium (e.g., Nash) is a shorthand for saying that a deterministic joint policy π that assigns probability 1 to this joint action is an X-equilibrium.

equilibrium for non-repeated normal-form games, in which d_i corresponds to an action recommendation for agent i given by a joint policy π_c . At the end of this section, we will mention possible extensions to sequential-move games.

Definition 8 (Correlated equilibrium) *In a general-sum normal-form game with n agents, let $\pi_c(a)$ be a joint policy that assigns probabilities to joint actions $a \in A$. Then, π_c is a correlated equilibrium if for every agent $i \in I$ and every action modifier $\xi_i : A_i \rightarrow A_i$:*

$$\sum_{a \in A} \pi_c(a) \mathcal{R}_i(\langle \xi_i(a_i), a_{-i} \rangle) \leq \sum_{a \in A} \pi_c(a) \mathcal{R}_i(a) \quad (4.19)$$

Equation 4.19 states that in a correlated equilibrium, in which every agent knows the probability distribution $\pi_c(a)$ and its own recommended action a_i (but not the recommended actions for other agents), no agent can unilaterally deviate from its recommended actions in order to increase its expected return. Here, deviating from the recommended actions is represented by the action modifier ξ_i . An example of a correlated joint policy can be seen in the central learning approach discussed in Section 5.3.1, which trains a single policy π_c directly over the joint-action space $A_1 \times \dots \times A_n$ and uses this policy to dictate actions to each agent.

It can be shown that the set of correlated equilibria contains the set of Nash equilibria (e.g., Osborne and Rubinstein 1994); namely, Nash equilibrium is a special case of correlated equilibrium in which the joint policy π_c is factored into independent agent policies $\pi_1, \dots, \pi_n$ with $\pi_c(a) = \prod_{i \in I} \pi_i(a_i)$. In this case, since the agents' policies in a Nash equilibrium are independent, knowing one's own action a_i does not give any information about the action probabilities for the other agents $j \neq i$. Similarly to ϵ -Nash equilibrium defined in Equation 4.18, we may add $-\epsilon$ in the left part of Equation 4.19 to obtain ϵ -correlated equilibrium, in which no agent can unilaterally deviate from its recommended actions to increase its expected return by more than ϵ .

To see an example of how correlated equilibrium can achieve greater returns than Nash equilibrium, consider the Chicken matrix game shown in Figure 4.3. This game represents a situation in which two vehicles (agents) are on a collision course and can choose to either stay on course (S) or leave (L). In the non-repeated game, there are the following three Nash equilibria with associated expected returns for the two agents shown as pairs (U_i, U_j) :

- $\pi_i(S) = 1, \pi_j(S) = 0 \rightarrow (7, 2)$
- $\pi_i(S) = 0, \pi_j(S) = 1 \rightarrow (2, 7)$
- $\pi_i(S) = \frac{1}{3}, \pi_j(S) = \frac{1}{3} \rightarrow \approx (4.66, 4.66)$

	S	L
S	0,0	7,2
L	2,7	6,6

Figure 4.3: Chicken matrix game.

Now, consider the following joint policy π_c that uses correlated actions:

- $\pi_c(L, L) = \pi_c(S, L) = \pi_c(L, S) = \frac{1}{3}$
- $\pi_c(S, S) = 0$

The expected return under π_c to both agents is: $7 \cdot \frac{1}{3} + 2 \cdot \frac{1}{3} + 6 \cdot \frac{1}{3} = 5$. It can be verified that no agent has an incentive to unilaterally deviate from the action recommended to it by π_c , assuming knowledge of π_c (and without knowing the actions recommended to other agents). For example, say agent i received action recommendation L. Then, given π_c , i knows that agent j will choose S with probability 0.5 and L with probability 0.5. Thus, the expected return for i when choosing L is $2 \cdot \frac{1}{2} + 6 \cdot \frac{1}{2} = 4$, which is higher than the expected return for choosing S ($0 \cdot \frac{1}{2} + 7 \cdot \frac{1}{2} = 3.5$). Thus, i has no incentive to deviate from L.

The previous example also illustrates how a correlated equilibrium can again be described as mutual best responses between the agents. In correlated equilibrium, each action recommendation a_i given by π_c is a best response (i.e., achieves maximum expected return for i) to the conditional distribution $\pi_{-i}(a_{-i} | a_i) = \frac{\pi_c(\langle a_i, a_{-i} \rangle)}{\sum_{a'_i \in A_i} \pi_c(\langle a'_i, a_{-i} \rangle)}$ over actions a_{-i} recommended to the other agents by π_c given a_i .

We can obtain an even more general class of equilibrium solutions, called *coarse correlated equilibrium* (Moulin and Vial 1978),¹⁰ by requiring that the inequality in Equation 4.19 only needs to hold for unconditional action modifiers, which satisfy $\forall a'_i \in A_i : \xi_i(a'_i) = a_i$ for some action a_i . In other words, each unconditional action modifier is just a constant action, with one ξ_i corresponding to each action $a_i \in A_i$. This solution concept means that each agent has to decide upfront, *before* seeing its recommended action, whether to follow the joint policy π_c assuming that the other agents follow it. If no agent can select a constant action (i.e., unconditional action modifier) to obtain a higher expected return compared to its expected return under π_c , as per Equation 4.19, then π_c is a coarse correlated equilibrium. Coarse correlated equilibria include correlated equilibria as a special case in which Equation 4.19 must hold for *all* possible action modifiers, not just unconditional action modifiers.

10. Moulin and Vial (1978) proposed this solution concept but did not name it in their work.

For sequential-move games, various definitions of correlated equilibrium exist (e.g., Forges 1986; Solan and Vieille 2002; von Stengel and Forges 2008; Farina, Bianchi, and Sandholm 2020). These definitions vary in a number of design choices and can be relatively complex. For instance, the private signals d_i may specify actions a_i that are revealed at each decision point, or they may specify entire policies π_i revealed once at the start of the game. The joint distribution over $d_1, \dots, d_n$ and the action/policy modifier ξ_i may be conditioned on different types of information, such as the current game state, agent observation histories, or previous values of d_i (Solan and Vieille 2002). The sampled outcomes of d_i may or may not be revealed to agents, such as in coarse correlated equilibria, which assume that the outcomes of d_i are only revealed to agents if they “commit” to the equilibrium (e.g., Farina, Bianchi, and Sandholm 2020). Definitions of correlated equilibrium can also vary in how agents are treated if they deviate from the equilibrium; for example, no further action recommendations may be issued to an agent after it deviates from the action recommended by the equilibrium (von Stengel and Forges 2008).

4.6.1 Correlated Equilibrium via Linear Programming

For non-repeated normal-form games, we can compute a correlated equilibrium by solving a linear program. The linear program computes a joint policy π such that no agent can improve its expected return by deviating from the joint actions sampled from π . Thus, the linear program contains variables x_a for each joint action $a \in A$, to represent the probability of selecting a under the joint policy π (i.e., $\pi(a) = x_a$). To select between different possible equilibria, we here use an objective that maximizes the sum of the agents’ expected returns (i.e., social welfare, see Section 4.9), but other objectives could be used such as maximizing expected returns for individual agents. This gives rise to the following linear program:

$$\text{maximize } \sum_{a \in A} \sum_{i \in I} x_a \mathcal{R}_i(a) \quad (4.20)$$

$$\text{subject to } \sum_{a \in A: a_i = a'_i} x_a \mathcal{R}_i(a) \geq \sum_{a \in A: a_i = a''_i} x_a \mathcal{R}_i(a'_i, a_{-i}) \quad \forall i \in I, a'_i, a''_i \in A_i \quad (4.21)$$

$$x_a \geq 0 \quad \forall a \in A \quad (4.22)$$

$$\sum_{a \in A} x_a = 1 \quad (4.23)$$

The constraints in Equation 4.21 ensure the property that no agent can gain by deviating from the action a'_i sampled under the joint policy $\pi(a) = x_a$ to a different action a''_i . The constraints in Equations 4.22 and 4.23 ensure that the

values of x_a form a valid probability distribution. A similar linear program can be solved to compute a coarse correlated equilibrium, by replacing the constraints from Equation 4.21 with the following constraints:

$$\sum_{a \in A} x_a \mathcal{R}_i(a) \geq \sum_{a \in A} x_a \mathcal{R}_i(a''_i, a_{-i}) \quad \forall i \in I, a''_i \in A_i \quad (4.24)$$

Note that these linear programs can contain many more variables and constraints than the linear programs for minimax solutions (Section 4.3.1). We now have variables corresponding to each joint action $a \in A$, which can grow exponentially in the number of agents (see also the discussion in Section 5.4.4). For n agents with k actions each, Equation 4.22 specifies k^n constraints, Equation 4.21 specifies nk^2 constraints, and Equation 4.24 specifies nk constraints.

4.7 Conceptual Limitations of Equilibrium Solutions

While equilibrium solutions, in particular the Nash equilibrium, have been adopted as the standard solution concept in MARL, they are not without certain shortcomings. Besides the practical issues noted in Section 4.5, we list some of the most important conceptual limitations in equilibrium solutions:

Sub-optimality In general, finding equilibrium solutions is not synonymous with maximizing expected returns. The only thing we know about a given equilibrium solution is that each agent's policy is a best response to the policies of the other agents, but this does not mean that the agent's expected returns are the best they could be. A simple example of this can be seen in the Prisoner's Dilemma game (Figure 3.2(c)), in which the only Nash equilibrium is the joint action (D,D) giving each agent an expected return of -3 , while the joint action (C,C) gives a higher expected return of -1 to each agent but is not a Nash equilibrium (each agent can deviate to improve its return). Similarly, in the Chicken game (Section 4.6), the shown correlated equilibrium π_c achieves an expected return of 5 for each agent, while the joint action (L,L) achieves expected returns of 6 for each agent but is neither a Nash equilibrium nor a correlated equilibrium.

Non-uniqueness Equilibrium solutions may not be unique, which means that there may exist multiple, even infinitely many, equilibria. Each of these equilibria may entail different expected returns for different agents, as can be seen in the three Nash equilibria in the Chicken game. This leads to a difficult challenge: Which of these different equilibria should the agents adopt, and how can they agree on a specific equilibrium? This challenge is known as the "equilibrium selection" problem and has been widely studied

in game theory and economics. In MARL, equilibrium selection can pose a serious challenge for agents that learn concurrently from local observations, which we will discuss further in Section 5.4.2. One approach to tackle equilibrium selection is to use additional criteria such as Pareto optimality (Section 4.8) and social welfare and fairness (Section 4.9) to differentiate between different equilibria.

Incompleteness For games with sequential moves, an equilibrium solution π is incomplete in that it does not specify equilibrium behaviors for *off-equilibrium paths*. An off-equilibrium path is any full history $\hat{h}$ which has probability $\Pr(\hat{h} | \pi) = 0$ (Equation 4.3) under the equilibrium π . For example, this could occur due to some temporary disturbance in the agents' executed policies leading to actions not normally prescribed by the policies. In such cases, π does not prescribe actions to bring the interaction back to an on-equilibrium path, meaning a full history $\hat{h}$ with $\Pr(\hat{h} | \pi) > 0$ under the equilibrium π . To address such incompleteness, game-theorists have developed refinement concepts such as subgame perfect equilibrium and trembling-hand perfect equilibrium (Selten 1988; Owen 2013).

4.8 Pareto Optimality

As we saw in the previous sections, an equilibrium solution in which agents use mutual best-response policies may be of limited value, because different equilibria may entail very different expected returns for different agents (e.g., Chicken game; see Section 4.7). Moreover, under certain conditions, any feasible and enforceable expected returns can be realized by an equilibrium solution, making the space of equilibrium solutions very large or infinite (folk theorems; see Section 4.4). Therefore, we may want to narrow down the space of equilibrium solutions by requiring additional criteria that a solution must achieve. One such criterion is *Pareto optimality*,¹¹ defined next:

Definition 9 (Pareto domination and optimality) A joint policy π is Pareto-dominated by another joint policy π' if

$$\forall i : U_i(\pi') \geq U_i(\pi) \text{ and } \exists i : U_i(\pi') > U_i(\pi). \quad (4.25)$$

A joint policy π is Pareto-optimal¹² if it is not Pareto-dominated by any other joint policy. We then also refer to the expected returns of π as Pareto-optimal.

11. Named after Italian economist Vilfredo Pareto (1848–1923).

12. Some of the game theory literature uses the terms “Pareto-efficient/inefficient” for Pareto-optimal/dominated, respectively.

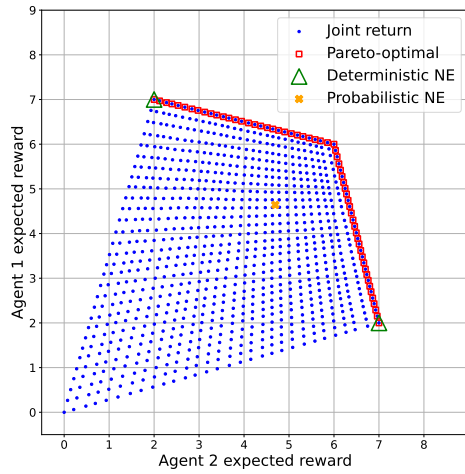


Figure 4.4: Feasible expected joint rewards and Pareto frontier in the Chicken matrix game. Each blue dot shows the expected joint reward obtained by a joint policy. Red squares show joint rewards of Pareto-optimal joint policies. Also shown are the joint rewards corresponding to the deterministic and probabilistic Nash equilibria of the non-repeated game.

Intuitively, a joint policy is Pareto-optimal if there is no other joint policy that improves the expected return for at least one agent, without reducing the expected return for any other agent. In other words, no agent can be better off without making other agents worse off. Every game must have at least one joint policy that is Pareto-optimal. In common-reward games, all Pareto-optimal joint policies achieve the same expected return, which by definition is the maximum possible expected return that any joint policy can achieve in the game.

We illustrate Pareto optimality using the non-repeated Chicken matrix game from Figure 4.3 (page 73). Figure 4.4 shows the convex hull of feasible expected joint returns in this game. Each dot corresponds to the expected joint return achieved by some joint policy (π_1, π_2) . In this figure, we discretized the space of joint policies by stepping each policy from $\pi_i(S)=0$ to $\pi_i(S)=1$ using a step size of $\frac{1}{30}$, resulting in 30 different policies for each agent, and a total of $30 \cdot 30 = 900$ joint policies. The corners of the convex hull correspond to the four possible deterministic joint policies (i.e., the four joint actions from the game matrix). The expected joint returns obtained by Pareto-optimal joint policies are marked by the red squares. The figure also shows the expected joint returns corresponding to the deterministic and probabilistic Nash equilibria of the game (see Section 4.6 for details).

For infinitely repeated games and under the average reward,¹³ there exists a folk theorem that shows that any expected joint return in this hull that is equal to or larger than the minmax value of the agents (Equation 4.17) can be realized by an equilibrium. In the Chicken game, it can be seen from the game's reward matrix that the minmax value for both agents is 2, and so any expected joint return (U_1, U_2) with $U_1, U_2 \geq 2$ in the hull can be realized by means of an equilibrium. This is where Pareto optimality comes in: we can narrow down the space of desirable equilibria by requiring that the expected joint return achieved by an equilibrium also be Pareto-optimal. The Pareto-optimal joint returns are those that reside on the *Pareto frontier* shown by the red squares. Thus, for any given joint policy π , we can project its corresponding expected joint return into the convex hull and detect Pareto optimality if the joint return lies on the Pareto frontier.

We have presented Pareto optimality as a concept to refine equilibrium solutions. However, note that a joint policy can be Pareto-optimal *without* being an equilibrium solution. Yet, Pareto optimality on its own may not be a very useful solution concept. In particular, all joint policies in zero-sum games are Pareto-optimal by definition. In general-sum games, many joint policies may be Pareto-optimal without being a desirable solution, such as joint policies that are Pareto-optimal but result in large differences between the agents' expected returns. We will discuss this latter case further in Section 4.9.

4.9 Social Welfare and Fairness

Pareto optimality states that there is no other solution in which at least one agent is better off without making other agents worse off. However, it does not make any statements about the total amount of rewards and their distribution among the agents. For example, the Pareto frontier in Figure 4.4 contains solutions with expected joint returns ranging from $(7, 2)$ to $(6, 6)$ to $(2, 7)$. Thus, we may consider concepts of *social welfare and fairness* to further constrain the space of desirable solutions.

The study of social welfare and fairness has a long history in economics, and many criteria and social welfare functions have been proposed (Moulin 2004; Fleurbaey and Maniquet 2011; Sen 2018; Amanatidis et al. 2023). The term welfare usually refers to some notion of totality of the agents' returns, while the term fairness relates to the distribution of returns among agents. In this section, we consider two basic definitions of welfare and fairness:

13. Average returns (or average rewards) correspond to discounted returns with $\gamma \rightarrow 1$.

Definition 10 (Welfare and Welfare Optimality) *The (social) welfare of a joint policy π is defined as*

$$W(\pi) = \sum_{i \in I} U_i(\pi). \quad (4.26)$$

A joint policy π is welfare-optimal if $\pi \in \arg \max_{\pi'} W(\pi')$.

Definition 11 (Fairness and Fairness Optimality) *The (social) fairness of a joint policy π is defined as¹⁴*

$$F(\pi) = \prod_{i \in I} U_i(\pi). \quad (4.27)$$

A joint policy π is fairness-optimal if $\pi \in \arg \max_{\pi'} F(\pi')$.

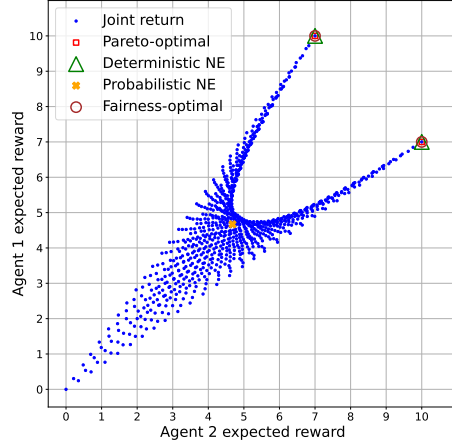
A welfare-optimal joint policy maximizes the sum of the agents' expected returns, while a fairness-optimal joint policy maximizes the product of the agents' expected returns. This definition of fairness promotes a type of equity between the agents, in the following sense: if we consider a set of joint policies $\pi \in \Pi$ that achieve equal welfare (sum of returns) $W(\pi)$, then the joint policy $\pi \in \Pi$ with the greatest fairness according to $F(\pi)$ is that which gives equal expected return for each agent, that is, $U_i(\pi) = U_j(\pi)$ for all i, j . For example, three different joint policies in a two-agent game that yield expected joint returns of (1, 5), (2, 4), (3, 3) achieve equal welfare of 6 and a respective fairness of 5, 8, and 9.¹⁵ When applying these definitions of welfare and fairness to the example game in Figure 4.4, it can be seen that the only solution that is both welfare-optimal and fairness-optimal is the joint policy that achieves expected joint return of (6, 6). Thus, in this example, we have narrowed the space of desirable solutions to a single solution. Another example of fairness-optimality, in the Battle of the Sexes matrix game, is shown in Figure 4.5.

Social welfare and fairness, such as defined here, can be useful in general-sum games but are not so useful in common-reward games and zero-sum games. In common-reward games, in which all agents receive identical reward, welfare and fairness are maximized if and only if the expected return of each agent is maximized; hence, welfare and fairness do not add any useful criteria in this class of games. In two-agent zero-sum games, in which one agent's reward is the negative of the other agent's reward, we know that all minimax solutions

14. This type of fairness is also known as *Nash social welfare*, defined as the geometric mean $(\prod_{i \in I} U_i(\pi))^{\frac{1}{n}}$ (Caragiannis et al. 2019; Fan et al. 2023).

15. The careful reader may have noticed some limitations of the simple definition of fairness in Definition 11. For example, if $U_i(\pi) = 0$ for any agent, then it does not matter what returns the other agents obtain under π . And if we allow $U_i(\pi) < 0$, then the expected joint return $(-0.1, 1, 1)$ would be less fair than the expected joint return $(-0.1, -100, 100)$, which is counter-intuitive.

	A	B
A	10,7	2,2
B	0,0	7,10



(a) Battle of the Sexes matrix game. (b) Joint rewards and fairness-optimal outcomes.

Figure 4.5: Feasible joint rewards and fairness-optimal outcomes in the Battle of the Sexes matrix game. This game models a situation where two people (classically, a man and a woman, hence the name of the game) want to meet in one of two places (A or B) but have different preferences about the best place. Agent 1 prefers place A and agent 2 prefers place B. Joint action (A,B) is preferred over (B,A), since in the latter case both agents end up at their respective least-preferred place. The two deterministic joint policies corresponding to joint actions (A,A) and (B,B) are the only joint policies (in the non-repeated game) that are both Pareto-optimal and fairness-optimal. The figure also shows the only probabilistic Nash equilibrium, which is neither Pareto-optimal nor fairness-optimal.

π have the same unique value $U_i(\pi) = -U_j(\pi)$ for agents i, j . Therefore, all minimax solutions achieve equal welfare and fairness, respectively (besides, the welfare as defined here will always be zero).

It is easy to prove that welfare optimality implies Pareto optimality. To see this, suppose a joint policy π is welfare-optimal but not Pareto-optimal. Since π is not Pareto-optimal, there exists a joint policy π' such that $\forall i: U_i(\pi') \geq U_i(\pi)$ and $\exists i: U_i(\pi') > U_i(\pi)$. However, it follows that $\sum_i U_i(\pi') > \sum_i U_i(\pi)$ and, therefore, π cannot be welfare-optimal (contradiction). Thus, π must also be Pareto-optimal if it is welfare-optimal. Note that Pareto optimality does not in general imply welfare optimality, hence welfare optimality is a stronger requirement. Moreover, fairness optimality does not imply Pareto optimality, and vice versa.

4.10 No-Regret

The equilibrium solution concepts discussed in the previous sections are based on mutual best responses between agents, and are thus a function of the agents' policies. Another category of solution concepts is based on the notion of *regret*, which measures the difference between the rewards an agent received and the rewards it could have received if it had chosen a different action (or policy) in past episodes against the observed actions (or policies) of the other agents in these episodes. An agent is said to have *no-regret* if, in the limit of infinitely many episodes, the agent's average regret across the episodes is at most zero. Therefore, no-regret considers the performance of learning agents across multiple episodes, which is in contrast to the other solution concepts introduced in this chapter that only consider a single joint policy (and not *how* this joint policy was learned). In this sense, no-regret can be viewed as an example of the prescriptive agenda discussed in Section 1.5, which is concerned with the performance of agents during learning.

There are multiple ways in which regret can be defined. We will first give a standard definition of regret for non-repeated normal-form games, which is based on comparing the rewards of different actions in the episodes. This definition will then be extended to sequential-move games. Let a^e denote the joint action from episodes $e = 1, \dots, z$. Agent i 's regret for not having chosen the best single action across these episodes is defined as

$$\text{Regret}_i^z = \max_{a_i \in A_i} \sum_{e=1}^z [\mathcal{R}_i(\langle a_i, a_{-i}^e \rangle) - \mathcal{R}_i(a^e)] . \quad (4.28)$$

An agent is said to have no-regret if its average regret in the limit of $z \rightarrow \infty$ is at most zero. As a solution concept, no-regret requires that all agents in the game have no-regret.

Definition 12 (No-regret) *In a general-sum game with n agents, the agents have no-regret if*

$$\forall i: \lim_{z \rightarrow \infty} \frac{1}{z} \text{Regret}_i^z \leq 0. \quad (4.29)$$

Similar to ϵ -Nash equilibrium (Section 4.5), we may replace the ≤ 0 in Equation 4.29 with $\leq \epsilon$, for $\epsilon > 0$, to obtain an ϵ -no-regret.

As a concrete example, Figure 4.6 shows ten episodes of two agents in the non-repeated Prisoner's Dilemma matrix game. After the episodes, agent 1 has received a total reward of -21 . Always choosing C in the episodes (against the observed actions of agent 2) would have resulted in a total reward of -30 , while always choosing D would have resulted in a total reward of -15 . Thus,

Episode e	1	2	3	4	5	6	7	8	9	10
Action a_1^e	C	C	D	C	D	D	C	D	D	D
Action a_2^e	C	D	C	D	D	D	C	C	D	C
Reward $\mathcal{R}_1(a^e)$	-1	-5	0	-5	-3	-3	-1	0	-3	0
Reward $\mathcal{R}_1(\langle C, a_2^e \rangle)$	-1	-5	-1	-5	-5	-5	-1	-1	-5	-1
Reward $\mathcal{R}_1(\langle D, a_2^e \rangle)$	0	-3	0	-3	-3	-3	0	0	-3	0

Figure 4.6: Ten episodes between two agents in the non-repeated Prisoner’s Dilemma matrix game. The bottom two rows show agent 1’s rewards for always choosing actions C/D against agent 2’s observed actions in the episodes.

action D was the “best” action against the observed actions of agent 2, and so agent 1’s regret is $\text{Regret}_1^{10} = -15 + 21 = 6$ with an average regret (dividing by 10) of 0.6. Indeed, in Prisoner’s Dilemma, D is a dominant action in that it is a best response against both D and C (see the reward matrix in Figure 3.2(c) (page 46) to verify this). In order for agent 1 to achieve no-regret, the episodes would need to continue in such a way that agent 1’s average regret across the episodes goes to zero.

We can generalize the definition of no-regret to stochastic games and POSGs, by redefining the regret over policies rather than actions. For each agent $i \in I$, let Π_i be a finite space of policies that agent i can select from. Let π^e denote the joint policy from episodes $e = 1, \dots, z$, with $\pi_i^e \in \Pi_i$ for all $i \in I$. Then, agent i ’s regret for not having chosen the best policy across these episodes is defined as

$$\text{Regret}_i^z = \max_{\pi_i \in \Pi_i} \sum_{e=1}^z [U_i(\langle \pi_i, \pi_{-i}^e \rangle) - U_i(\pi^e)]. \quad (4.30)$$

With this definition of regret, Definition 12 applies to all of the game models introduced in Chapter 3. (For non-repeated normal-form games, Equation 4.30 is equivalent to Equation 4.28 if we define each Π_i to be the set of deterministic policies corresponding to each action $a_i \in A_i$.)

Our previous example in Prisoner’s Dilemma illustrates an important conceptual limitation of regret, which is that it assumes that the actions or policies of other agents $-i$ remain fixed in the episodes. This assumption is sensible if the other agents use constant policies that do not change between episodes. However, if the other agents adapt their policies based on past episodes, then this assumption is of course violated. Therefore, regret does not actually quantify what would have happened under counterfactual situations. The second limitation, which is a result of the first limitation, is that minimizing regret is

not necessarily equivalent to maximizing returns (Crandall 2014). For example, in non-repeated and finitely repeated Prisoner’s Dilemma, the only joint policy that has no-regret is for both agents to always choose D. This is analogous to the fact that mutual best-response policies do not necessarily entail maximum returns for the agents, as we discussed in Section 4.7.

Various alternative definitions of regret exist (e.g., de Farias and Megiddo 2003; Lehrer 2003; Chang 2007; Zinkevich et al. 2007; Arora, Dekel, and Tewari 2012; Crandall 2014). For example, for normal-form games in which agents can choose from more than two actions, rather than replacing all of agent i ’s past actions as in Equation 4.28, we may replace only the specific occurrences of a given action a_i' in the history with a different action a_i . This latter definition of regret is also known as *conditional* (or internal) regret, while the definition in Equation 4.28 is known as *unconditional* (or external) regret. Furthermore, no-regret solutions have connections to equilibrium solutions. In particular, in two-agent zero-sum normal-form games, the empirical distribution of joint actions produced by agents that have no external regret converges to the set of minimax solutions; and in general-sum normal-form games, the empirical distribution of joint actions converges to the set of coarse correlated equilibria if the agents have no external regret, and to the set of correlated equilibria if the agents have no internal regret (Hart and Mas-Colell 2000; Young 2004). (See also Section 5.2 for a discussion of convergence types.) We will revisit these two regret definitions and their connection to correlated equilibrium in Section 6.5.

4.11 The Complexity of Computing Equilibria

Before we turn to MARL algorithms in Chapters 5 and 6 as a method to compute solutions for games, it is instructive to ask: How difficult is it, in terms of computational complexity, to compute an equilibrium solution for a game? Do algorithms exist that can compute equilibria *efficiently*, meaning in polynomial time in the size of the game?

These and related questions are studied in algorithmic game theory, which is a research area at the interface of computer science and game theory. Many complexity results exist for various special types of games, and we recommend the books of Nisan et al. (2007) and Roughgarden (2016) for a broad discussion. Here, we focus on non-repeated normal-form games, which are the building block of the more complex (partially observable) stochastic game models. Thus, we can expect any complexity results for normal-form games to be a lower bound on the complexity for the more complex game models.

Most computer scientists will have some familiarity with the complexity classes P and NP. P includes all decision problems whose solutions (if they exist) can be computed efficiently in polynomial time in the size of the problem instance. NP includes all decision problems whose solutions (if they exist) can be computed in polynomial time by a non-deterministic Turing machine. Unfortunately, these familiar complexity classes are not a good fit for the problem of computing an equilibrium, since P/NP characterize *decision problems* which may or may not have solutions, while we know that games *always* have at least one equilibrium solution. On the other hand, computing an equilibrium that satisfies additional properties *is* a decision problem, since such solutions may or may not exist. Such problems include computing equilibria that:

- are Pareto optimal;
- achieve a certain minimum expected return for each agent;
- achieve a certain minimum social welfare (sum of returns);
- assign zero or positive probability to certain actions of certain agents.

All of these problems are known to be NP-hard (Gilboa and Zemel 1989; Conitzer and Sandholm 2008).

Some types of games and equilibria do admit polynomial-time algorithms. Computing a minimax solution in a two-agent zero-sum non-repeated normal-form game can be formulated via linear programming (Section 4.3.1), which can be solved in polynomial time. Similarly, computing a correlated equilibrium in a general-sum non-repeated normal-form game can be done in polynomial time via a linear program (Section 4.6.1). However, computing a Nash equilibrium in general-sum non-repeated normal-form games — henceforth simply called NASH — cannot be solved via linear programming, due to the independence assumption between policies in a Nash equilibrium.

Problems that always have solutions, such as NASH, are known as *total search problems*. Section 4.11.1 will present one subclass of total search problems, called PPAD. It turns out that NASH is a *complete* problem in PPAD, meaning that any other problem in PPAD can be reduced to NASH. We will discuss the implications for MARL in Section 4.11.2.

4.11.1 PPAD Complexity Class

PPAD (short for “polynomial parity argument for directed graphs”) describes a special class of total search problems. We define PPAD by giving one of its

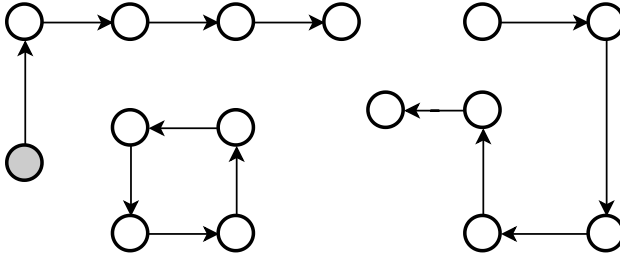


Figure 4.7: Any instance of END-OF-LINE consists of a set of paths and cycles, such as the instance shown here. Given the *Parent* / *Child* functions and a source node (a node with no parent node; here shaded in gray), is there an efficient polynomial-time algorithm to find a sink node (a node with no child node) or a different source node?

complete problems to which all other problems in PPAD can be reduced.¹⁶ This PPAD-complete problem is called END-OF-LINE and is defined as follows:

Definition 13 (END-OF-LINE) Let $G(k) = (V, E)$ be a directed graph consisting of

- a finite set V containing 2^k nodes (each node is represented as a bit-string of length k)
- a finite set $E = \{(a, b) \mid a, b \in V\}$ of directed edges (from node a to node b , for $a, b \in V$) such that:

$$\text{if } (a, b) \in E \text{ then } \nexists a' \neq a : (a', b) \in E \text{ and } \nexists b' \neq b : (a, b') \in E$$

Assume functions $\text{Parent}(v)$ and $\text{Child}(v)$ that, respectively, return the parent node (if any) and child node (if any) of node $v \in V$. These functions are represented as boolean circuits with k input bits and k output bits, and run in time polynomial in k . Given access to functions Parent and Child (but not E), and a node $s \in V$ such that $\text{Parent}(s) = \emptyset$, find a node $e \neq s$ such that either $\text{Child}(e) = \emptyset$ or $\text{Parent}(e) = \emptyset$.¹⁷

Figure 4.7 shows an illustration of a END-OF-LINE problem instance. The restriction on E in Definition 13 means that any node in the graph can have at most one parent node and at most one child node. A node with no parent is

16. Problem A can be “reduced” to problem B if there exist polynomial-time algorithms to transform any instance of A into an equivalent instance of B, and any solution of the instance of B to a solution of the original instance of A.

17. To indicate that a node has no parent/child, the respective circuit function simply outputs the same input node.

also called a source node, while a node with no child is called a sink node. The “parity argument” in PPAD refers to the fact that any source node in this graph always has a corresponding sink node. Therefore, if a source node s is given, then we know that a node e must exist. The node e can, in principle, be found by following the directed path starting at the given source node s . However, since we are only given the functions *Parent* and *Child* (and not E), the only obvious way to follow this path is by repeatedly calling the function *Child* starting at the source node s . Therefore, finding a sink node may require exponential time in the worst case, since there are 2^k nodes.

Why should we care about PPAD? Just as it is unknown whether efficient polynomial-time algorithms exist to solve NP-complete problems (the big “ $P = NP?$ ” question), it is also unknown whether efficient algorithms exist for PPAD-complete problems (“ $P = \text{PPAD}?$ ”). Indeed, PPAD contains problems for which researchers have tried for decades to find efficient algorithms, including the classic Brouwer fixed-point problem and finding Arrow-Debreu equilibria in markets (see Papadimitriou (1994) for a more complete list). PPAD has also been shown to be hard under cryptographic assumptions (Bitansky, Paneth, and Rosen 2015; Garg, Pandey, and Srinivasan 2016; Choudhuri et al. 2019). There are currently no known algorithms that can efficiently solve END-OF-LINE (i.e., in time polynomial in k) and, thus, any other problem in PPAD. Therefore, establishing that a problem is PPAD-complete is a good indicator that no efficient algorithms exist to solve the problem.

4.11.2 Computing ϵ -Nash Equilibrium Is PPAD-Complete

We return to our initial question, “Do algorithms exist that can compute equilibria *efficiently*, in polynomial time in the size of the game?” Unfortunately, the answer is very likely negative. It has been proven that NASH is PPAD-complete, at first for games with three or more agents (Daskalakis, Goldberg, and Papadimitriou 2006, 2009), and shortly after even for games with two agents (Chen and Deng 2006). This means that finding a Nash equilibrium in a non-repeated normal-form game can be described as finding an e -node in an equivalent END-OF-LINE instance. The completeness also means that any other problem in PPAD, including Brouwer fixed-points, can be reduced to NASH.

More precisely, the PPAD-completeness of NASH was proven for approximate ϵ -Nash equilibrium (Section 4.5) for certain bounds on $\epsilon > 0$; and for

exact equilibria (when $\epsilon=0$) in games with two agents.¹⁸ The use of ϵ -Nash equilibrium is to account for the fact that a Nash equilibrium may involve irrational-valued probabilities in games with more than two agents. Therefore, the PPAD-completeness of NASH also includes approximation schemes for computing Nash equilibria, such as MARL algorithms, which may only learn approximate solutions given a finite number of interactions in the game.

The implication for us is that MARL is unlikely to be a magic bullet for solving games: since no efficient algorithms are known to exist for PPAD-complete problems, it is unlikely that efficient MARL algorithms exist to compute Nash equilibria in polynomial time. Much of the research in MARL has focused on identifying and exploiting structures (or assumptions) in certain game types that lead to improved performance. However, the PPAD-completeness of NASH tells us that, in general and without such assumed structures, it is likely that *any* MARL algorithm still requires exponential time in the worst case.

4.12 Summary

This chapter introduced a range of solution concepts for games, to define optimal policies for agents in the game. The main concepts are the following:

- A *solution* for a game is a joint policy (typically including one policy for each agent) that satisfies certain conditions which are expressed in terms of the expected returns for agents and the relationship between the returns.
- Many solution concepts can be compactly represented based on the notion of *best responses*. A best-response policy maximizes the expected returns of one agent against a set of given policies for the other agents.
- There exists a series of increasingly general equilibrium solution concepts, including *minimax* for two-agent zero-sum games, and *Nash equilibrium* and (*coarse*) *correlated equilibrium* for general-sum games with two or more agents. These equilibrium solutions are all based on the idea that every agent is best-responding to all other agents under the equilibrium, and hence no agent can unilaterally deviate from the equilibrium to increase its returns.
- Games may have a unique equilibrium solution, or they may have multiple (even infinitely many) equilibria that can yield different expected returns for

18. As discussed in Section 4.5, an ϵ -Nash equilibrium may not be close to any actual Nash equilibrium, even if the Nash equilibrium is unique. The problem of approximating an actual Nash equilibrium within a desired distance, as measured in the policy space via norms such as L1 and L2, is in fact much harder than computing an ϵ -Nash equilibrium. This latter problem, for general-sum normal-form games with three or more agents, is complete for a different complexity class called FIXP (Etessami and Yannakakis 2010).

the agents. Thus, learning an equilibrium solutions is not necessarily the same as maximizing the expected returns for all agents.

- Different refinement concepts exist that can be combined with equilibrium solutions to narrow the space of desirable solutions, such as *Pareto optimality* and concepts based on *social welfare and fairness*. For example, we may seek a Nash equilibrium that is also Pareto-optimal.
- *No-regret* is an alternative solution concept that considers the alternative returns an agent could have received if it had used different policies in past episodes of the game against the observed policies of the other agents. A joint policy achieves no-regret if the average regret for each agent goes to zero in the limit of infinitely many episodes.
- Computing a Nash equilibrium in general-sum normal-form games is a complete problem for the *PPAD complexity class*. There are currently no known efficient (i.e., polynomial time) algorithms to solve PPAD-complete problems. The implication is that there probably do not exist efficient MARL algorithms that learn Nash equilibria for general games.

Now that we are equipped with different solution concepts for the game models introduced in Chapter 3, the next two chapters in this part of the book will describe different families of MARL algorithms that are designed to learn such solutions under certain conditions. Chapter 5 will begin by outlining a general learning process and different convergence types with which MARL algorithms can learn or approximate solutions, as well as the key challenges when trying to learn game solutions using MARL methods.

5 Multi-Agent Reinforcement Learning in Games: First Steps and Challenges

The preceding chapters introduced game models as a formalism of multi-agent interaction, and solution concepts to define what it means for the agents to act optimally in a game. In this chapter, we will begin to explore methods to *compute* solutions for games. The principal method by which we seek to compute solutions is via reinforcement learning (RL), in which the agents repeatedly try actions, make observations, and receive rewards. Analogous to the standard RL terminology introduced in Chapter 2, we use the term *episode* to refer to each independent run of a game starting in some initial state. The agents learn their policies based on data (i.e., observations, actions, and rewards) obtained from multiple episodes in a game.

To set the context for the algorithms presented in this book, this chapter will begin by outlining a general learning framework for MARL, as well as different types of convergence definitions used in the analysis and evaluation of MARL algorithms. We will then introduce two basic approaches of applying RL in games, called central learning and independent learning, both of which reduce the multi-agent problem to a single-agent problem. Central learning applies single-agent RL directly to the space of joint actions to learn a central policy that chooses actions for each agent, while independent learning applies single-agent RL to each agent independently to learn agent policies, essentially ignoring the presence of other agents.

Central and independent learning serve as a useful starting point to discuss several important challenges faced by MARL algorithms. One characteristic challenge of MARL is environment non-stationarity due to multiple learning agents, which can lead to unstable learning. Equilibrium selection is the problem of what equilibrium solution the agents should agree on and how they may achieve agreement. Another challenge is multi-agent credit assignment, in which agents must infer *whose* actions contributed to a received reward. Finally,

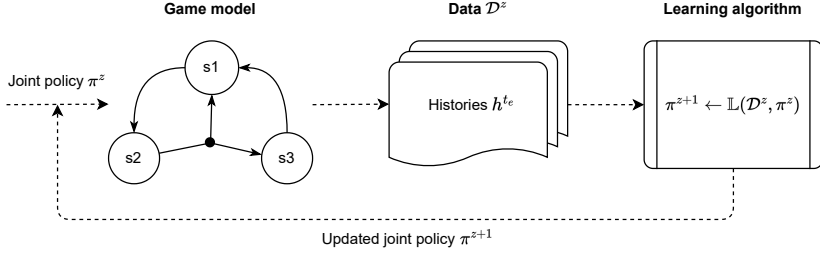


Figure 5.1: Elements of a general learning process in MARL.

MARL algorithms are typically faced with an exponential growth of the joint-action space as the number of agents is increased, leading to scalability problems. We will discuss each of these challenges and provide examples.

The idea that agents can use their own algorithms to learn policies, such as in independent learning, leads to the possibility that agents may use the same learning algorithm or different algorithms. This chapter will conclude with a discussion of such self-play and mixed-play settings in MARL.

5.1 General Learning Process

We begin by defining *learning*¹ in games and the intended learning outcome. In machine learning, learning is a process that optimizes a model or function based on *data*. In our setting, the model is a joint policy usually consisting of policies for each agent, and the data (or “experiences”) consist of one or more histories in the game. The learning goal is a solution of the game, defined by a chosen solution concept. Thus, this learning process involves several elements, shown in Figure 5.1 and detailed next.

Game model: The game model defines the multi-agent environment and how agents may interact. Game models introduced in Chapter 3 include non-repeated normal-form games, repeated normal-form games, stochastic games, and partially observable stochastic games (POSG).

Data: The data used for learning consist of a set of z histories,

$$\mathcal{D}^z = \{h^{t_e} \mid e = 1, \dots, z\}, \quad z \geq 0. \quad (5.1)$$

1. Another commonly used term is “training.” We use the terms learning and training interchangeably, for example, as in “learning/training a policy.”

Each history h^e was produced by a joint policy π^e used during episode e . These histories may or may not be “complete” in the sense of ending in a terminal state of the game, and different histories may have different lengths t_e . Often, $\mathcal{D}^z$ contains the history so far from the current ongoing episode z and the histories from previous episodes $e < z$.

Learning algorithm: The learning algorithm $\mathbb{L}$ takes the collected data $\mathcal{D}^z$ and current joint policy π^z , and produces a new joint policy,

$$\pi^{z+1} = \mathbb{L}(\mathcal{D}^z, \pi^z). \quad (5.2)$$

The initial joint policy π^0 is typically random.

Learning goal: The goal of learning is a joint policy π^* which satisfies the properties of a chosen solution concept. Chapter 4 introduced a range of possible solution concepts, such as Nash equilibrium.

We note several nuances in the above elements:

The chosen game model determines the conditioning of the learned joint policy. In a non-repeated normal-form game (where episodes terminate after one time step), policies π_i are not conditioned on histories, that is, they are simple probability distributions over actions. In a repeated normal-form game, policies are conditioned on action histories $h^t = (a^0, \dots, a^{t-1})$. In a stochastic game, policies are conditioned on state-action histories $h^t = (s^0, a^0, s^1, a^1, \dots, s^t)$. In a POSG, policies are conditioned on observation histories $h_i^t = (o_i^0, \dots, o_i^t)$. These conditionings are general and may be constrained depending on the desired form of policies. For example, in a stochastic game we may condition policies only on the current state of the game; and in a POSG we may condition policies using only the most recent k observations.

The histories in $\mathcal{D}^z$ may in general be *full* histories (i.e., contain all states and joint observations/actions; see Section 4.1) or one of the other types of histories listed earlier. Therefore, the histories in $\mathcal{D}^z$ may contain more information than the histories used to condition the agents’ policies. For example, this can be the case in centralized training with decentralized execution regimes (discussed in Section 9.1), in which a learning algorithm has access to the observations of all agents during learning, while the agents’ policies only have access to the local observations of agents.

The learning algorithm $\mathbb{L}$ may itself consist of multiple learning algorithms that learn individual agent policies, such as one algorithm $\mathbb{L}_i$ for each agent i . Each of these algorithms may use different parts of the data in $\mathcal{D}^z$, or use its own data $\mathcal{D}_i^z$ such as in independent learning (Section 5.3.2). Furthermore, an important characteristic of RL is that the learning algorithm is actively involved in the generation of the data by exploring actions, rather than just passively

consuming the data. Thus, the policies produced by the learning algorithm may actively randomize over actions to generate useful data for learning.

5.2 Convergence Types

A number of different criteria can be used to evaluate the learning performance of MARL algorithms. The main theoretical evaluation criterion for learning that we use in this book is convergence of the joint policy π^z to a solution π^* of the game (e.g., Nash equilibrium) in the limit of infinite data,

$$\lim_{z \rightarrow \infty} \pi^z = \pi^*. \quad (5.3)$$

As was discussed in Chapter 4, a game may have more than one solution π^* and this depends on the specific solution concept. When we say “convergence to a solution π^* ” the emphasis is on “a,” meaning that π^* is *some* valid solution under the relevant solution concept.

When we make statements about the theoretical convergence of MARL algorithms, unless specified otherwise, we mean convergence as per Equation 5.3.² Of course, in practice we cannot collect infinite data, and learning typically stops after a predefined budget is reached (such as a total allowed number of episodes or time steps) or once the changes in policies are below some predetermined threshold. Whether a learned joint policy π^z is in fact a solution can be tested using procedures such as described in Sections 4.4 and 4.5.

Several other theoretical evaluation criteria have been studied in the literature, including both weaker and stronger types of convergence. Weaker types of convergence include:

- Convergence of expected return:

$$\lim_{z \rightarrow \infty} U_i(\pi^z) = U_i(\pi^*), \quad \forall i \in I \quad (5.4)$$

This convergence type means that, in the limit of infinite data $z \rightarrow \infty$, the expected joint return under the learned joint policy π^z will converge to the expected joint return of a solution π^* .

- Convergence of empirical distribution:

$$\lim_{z \rightarrow \infty} \bar{\pi}^z = \pi^* \quad (5.5)$$

2. To be fully precise, some convergence statements in the literature add the additional qualifier that convergence will happen “with probability 1” or “almost surely.” This qualifier means that, theoretically, there may be conditions (i.e., trajectories of (π^z)) under which convergence might *not* happen, but that the probability of these conditions occurring is 0 under the relevant probability measure. We omit this technicality in favor of simplicity.

where $\bar{\pi}^z(a|h) = \frac{1}{z} \sum_{e=1}^z \pi^e(a|h)$ is the averaged joint policy across episodes. We may equivalently define $\bar{\pi}^z$ as the *empirical distribution* as follows: In an episode e , let $(a^\tau, h^\tau)_{\tau=0}^t$ be the sequence of joint actions a^τ sampled from the joint policy π^e conditioned on history h^τ , that is, $a^\tau \sim \pi^e(\cdot|h^\tau)$.³ (Note that e denotes the episode number while τ denotes the time step within an episode.) Let H^z be the set containing all of the (a^τ, h^τ) -pairs from episodes $e = 1, \dots, z$. Then, the empirical distribution is given by

$$\bar{\pi}^z(a|h) = \frac{1}{|H^z(h)|} \sum_{(a^\tau, h^\tau) \in H^z(h)} [a = a^\tau]_1 \quad (5.6)$$

where $H^z(h) = \{(a^\tau, h^\tau) \in H^z \mid h^\tau = h\}$, and $[x]_1 = 1$ if x is true, else $[x]_1 = 0$. (If $H^z(h)$ is empty, then $\bar{\pi}^z(a|h) = \frac{1}{|A|}$.) These two definitions are equivalent in that the averaged joint policy is the expectation of the empirical distribution. Thus, they get arbitrarily close for increasing z , and they are identical for $z \rightarrow \infty$. This means that the convergence in Equation 5.5 holds if and only if it holds for both the averaged joint policy and the empirical distribution.

- Convergence of empirical distribution to set of solutions:

$$\forall \epsilon > 0 \exists z_0 \forall z > z_0 \exists \pi^* : d(\bar{\pi}^z, \pi^*) < \epsilon \quad (5.7)$$

In words, for any arbitrarily small (but fixed) $\epsilon > 0$, there is a point z_0 in the learning process depending on ϵ such that, for all $z > z_0$, there exists a solution π^* at a distance less than ϵ to the empirical distribution $\bar{\pi}^z$. The distance $d(\bar{\pi}^z, \pi^*)$ could be based on some L-norm, or the ϵ -versions of solution concepts (e.g., ϵ -Nash equilibrium). The difference to the pointwise convergence of Equation 5.3 is that, in Equation 5.7, the empirical distribution will eventually reach (within ϵ) the space of solutions but may then “wander” inside this space without necessarily converging to any one point π^* .

- Convergence of average return:

$$\lim_{z \rightarrow \infty} \bar{U}_i^z = U_i(\pi^*), \forall i \in I \quad (5.8)$$

where $\bar{U}_i^z = \frac{1}{z} \sum_{e=1}^z U_i(\pi^e)$ is the averaged expected return across episodes. Intuitively, this convergence type can be interpreted as saying that if we generate a new episode for each updated joint policy π^e , then the average

3. For example, in a non-repeated normal-form game, a^τ is the joint action selected by the agents and $h^\tau = \emptyset$ since the policies are not conditioned on histories. In this book, we mainly refer to empirical distributions in the context of non-repeated normal-form games.

of the joint returns from these episodes will converge to the expected joint return of a solution π^* .

These weaker convergence types are often used when a particular learning algorithm is not technically able to achieve the convergence in Equation 5.3. For example, the fictitious play algorithm presented in Section 6.3.1 learns deterministic policies, meaning that it cannot represent probabilistic Nash equilibria, such as the uniform-random Nash equilibrium in Rock-Paper-Scissors. However, in certain cases, it can be proven that the empirical action distribution of fictitious play converges to a Nash equilibrium, as per Equation 5.5 (Fudenberg and Levine 1998). The infinitesimal gradient ascent algorithm presented in Section 6.4.1 *can* learn probabilistic policies, but may still not converge to a probabilistic Nash equilibrium, while it can be shown in non-repeated normal-form games that the average rewards produced by the algorithm do converge to the average rewards of a Nash equilibrium, as per Equation 5.8 (Singh, Kearns, and Mansour 2000). Lastly, the regret-matching algorithms presented in Section 6.5 can produce abrupt changes in the joint policy π^z , which, theoretically, may cause it to not converge to any one solution π^* . However, it can be shown for normal-form games that the empirical distributions produced by the regret-matching algorithms do converge to the *set* of (coarse) correlated equilibria, as per Equation 5.7 (Hart and Mas-Colell 2000).

Note that Equation 5.3 implies all of the above weaker types of convergence. However, it makes no claims about the performance of any individual joint policy π^z for a finite z . In other words, the above convergence types leave open how the agents perform *during* learning. To address this, a stronger evaluation criterion could require additional bounds, such as on the difference between π^z and π^* for finite z . (See also the discussion and references in Section 5.5.2.)

In complex games, it may not be computationally practical to check for these convergence properties. Instead, a common approach is to monitor the expected returns $U_i(\pi^z)$ achieved by the joint policy as z increases, usually by visualizing learning curves that show the progress of expected returns during learning, as shown in Figure 2.4 (page 37). A number of such learning curves will be shown for various MARL algorithms presented in this book. However, this evaluation approach may not establish any relationship to the solution π^* of the game. For instance, even if the expected returns $U_i(\pi^z)$ for all $i \in I$ converge after some finite z , the joint policy π^z might not satisfy any of the convergence properties of Equations 5.3 to 5.8.

To reduce notation in the remainder of this chapter (and book), we will omit the explicit z -superscript (e.g., π^z , D^z) and we usually omit D^z altogether.

5.3 Single-Agent RL Reductions

The most basic approach to using RL to learn agent policies in multi-agent systems is to essentially reduce the multi-agent learning problem to a single-agent learning problem. In this section, we will introduce two such approaches: *central learning* applies single-agent RL directly to the space of joint actions to learn a central policy that chooses actions for each agent; and *independent learning* applies single-agent RL independently to each agent to learn independent policies, essentially ignoring the presence of other agents.

5.3.1 Central Learning

Central learning trains a single central policy π_c , which receives the local observations of all agents and selects an action for each agent, by selecting joint actions from $A = A_1 \times \dots \times A_n$. This essentially reduces the multi-agent problem to a single-agent problem, and we can apply existing single-agent RL algorithms to train π_c . An example of central learning based on Q-learning, called *central Q-learning* (CQL), is shown in Algorithm 4. This algorithm maintains *joint-action values* $Q(s, a)$ for joint actions $a \in A$, which is a basic concept used by many MARL algorithms presented in this book. Central learning can be useful because it circumvents the multi-agent aspects of the non-stationarity and credit assignment problems (discussed in Sections 5.4.1 and 5.4.3, respectively). However, in practice, this approach has a number of limitations.

The first limitation to note is that, in order to apply single-agent RL, central learning requires transforming the joint reward $(r_1, \dots, r_n)$ into a single scalar reward r . For the case of common-reward games, in which all agents receive identical rewards, we can use $r = r_i$ for any i . In this case, if we use a single-agent RL algorithm that is guaranteed to learn an optimal policy in an MDP (such as the temporal-difference algorithms discussed in Section 2.6), then it is guaranteed to learn a central policy π_c for the common-reward stochastic game such that π_c is a Pareto-optimal correlated equilibrium. The optimality of the single-agent RL algorithm means that π_c achieves maximum expected returns in each state $s \in S$ (as discussed in Section 2.4). Therefore, since the reward is defined as $r = r_i$ for all i , we know that π_c is Pareto-optimal because there can be no other policy that achieves a higher expected return for any agent. This also means that no agent can unilaterally deviate from its action given by π_c to improve its returns, making π_c a correlated equilibrium.

Unfortunately, for zero-sum and general-sum stochastic games, it is less clear how the reward scalarization should be done. If one is interested in maximizing social welfare (Section 4.9) in general-sum games, one option is to use $r = \sum_i r_i$.

Algorithm 4 Central Q-learning (CQL) for stochastic games

- 1: Initialize: $Q(s, a) = 0$ for all $s \in S$ and $a \in A = A_1 \times \dots \times A_n$
 - 2: Repeat for every episode:
 - 3: **for** $t = 0, 1, 2, \dots$ **do**
 - 4: Observe current state s^t
 - 5: With probability ϵ : choose random joint action $a^t \in A$
 - 6: Otherwise: choose joint action $a^t \in \arg \max_a Q(s^t, a)$
 - 7: Apply joint action a^t , observe rewards $r_1^t, \dots, r_n^t$ and next state s^{t+1}
 - 8: Transform $r_1^t, \dots, r_n^t$ into scalar reward r^t
 - 9: $Q(s^t, a^t) \leftarrow Q(s^t, a^t) + \alpha [r^t + \gamma \max_{a'} Q(s^{t+1}, a') - Q(s^t, a^t)]$
-

However, if the desired solution is an equilibrium type solution, then no scalar transformation may exist that leads to equilibrium policies.

The second limitation is that by training a policy over the joint-action space, we now have to solve a decision problem with an action space that grows exponentially in the number of agents.⁴ In the level-based foraging example shown in Figure 1.2 (page 4), there are three agents that choose from among six actions (up, down, left, right, collect, noop), leading to a joint-action space with $6^3 = 216$ actions. Even in this toy example, most standard single-agent RL algorithms do not scale easily to action spaces this large.

Finally, a fundamental limitation of central learning is due to the inherent structure of multi-agent systems. Agents are often localized entities that are physically or virtually distributed. In such settings, communication from a central policy π_c to the agents and vice versa may not be possible or desirable, for various reasons. Therefore, such multi-agent systems require local agent policies π_i for each agent i , which act on agent i 's local observations and independently from other agents.

In stochastic games, in which we assume that agents can observe the full state, it is possible to learn an optimal central policy π_c that can be decomposed into individual agent policies $\pi_1, \dots, \pi_n$. This is because solving a stochastic game via central learning amounts to solving an MDP, and MDPs always admit deterministic optimal policies that assign probability 1 to some action in each state (see Equation 2.26 in Section 2.4). Thus, we can decompose the

4. This exponential growth assumes that each additional agent comes with additional decision variables. For example, in level-based foraging (Section 1.1), each agent comes with its own action set. The reverse direction is when we have a fixed set of actions that are partitioned and assigned to agents. In the latter case, the total number of actions remains constant, regardless of the number of agents. We discuss this further in Section 5.4.4.